Build AI, Understand AI

How Today's LLM Systems Actually Work, in Small Readable Pieces

Michael Borck

2026-11-07

Build AI, Understand AI

How Modern AI Systems Actually Work, in Small Readable Pieces

© 2026 Michael Borck

Book content: Creative Commons Attribution 4.0 International (CC BY 4.0 International).
https://creativecommons.org/licenses/by/4.0/

Original code examples and accompanying software: MIT License.
https://opensource.org/licenses/MIT

These grants cover material the author has authority to license. Separately credited third-party material retains its stated terms.

ISBN: (to be assigned)

Table of contents

Preface 1

I The Model 7

1 One Call to the Model 9

2 Who the Model Is Being 23

3 No Memory Between Calls 33

4 Any Provider Will Do 43

II Grounding 53

5 What the Model Doesn't Know 55

6 Chunk, Fetch, Answer 65

III Action 75

7 Giving the Model Hands 77

8 The ReAct Loop 87

IV Control 97

9 Workflows as Folders 99

10 Loops with a Goal **107**

11 The Judge **117**

12 Typed Answers **127**

V The Senses **137**

13 Pictures **139**

14 Voice **147**

VI Inside the Box (Optional) **155**

15 The Older AI in the Room **157**

16 Train a Tiny Model **165**

17 Teach It a Dialect **173**

VII Looking Underneath **181**

18 The Shape Underneath **183**

Appendices **189**

A Just Enough Python **189**

B Model Access **201**

C The Project Gallery **207**

D Going Deeper **211**

Acknowledgments **215**

About the Author **217**

Preface

Why This Book Exists

You use LLM systems every day. You type into a chat box, upload a document and ask questions about it, watch a tool fetch a web page and reason over it. And if someone asked you what actually happened in there, you would probably say something vague about "the model". That vagueness has a cost: it is where confident nonsense gets past you.

The machinery is not that complicated. Strip away the dashboards and the marketing, and every LLM system in that first paragraph is one call to a model, wrapped in plain Python. A chatbot is that call plus a paragraph of instructions. A document question-answerer is that call plus a search. An agent is that call plus a loop that runs your functions. The package this book pairs with, `hands-on-ai`, makes each of those small enough to read in an afternoon, because when the whole thing fits in your head, the concept becomes visible.

A note on the title's word, since this book will spend eighteen chapters teaching you to check claims like it: AI is a wider field than this book. It includes search and symbolic reasoning, the classical machine learning of decision trees, support vector machines, and clustering that has filtered your spam and scored your transactions for decades, and the deep learning that produced both image classifiers and the large language models, LLMs, at this book's centre. Those older branches still run much of the world's consequential automation, often better than an LLM would. This book keeps the umbrella word for its title and earns it narrowly: Parts I to V are

about LLM systems, and the optional Part VI opens with the older AI on its own terms before turning to the training loop itself. When any vendor, this book included, says "AI" without saying which branch, you will know to ask.

This book walks those builds in a deliberate order, in five parts that need nothing beyond a model you can call. You will write a little Python and read a little more. What you will not do in those five parts is train anything, install a framework, or take anyone's word for anything. An optional sixth part, clearly fenced, then opens the box a crack: it trains toy models in notebooks, at toy scale, for readers who want to see that side too. By the end, when a demo answers fluently, you will know which parts of what you saw were the model and which were a harness someone wrote on a Tuesday.

Learn the Tide, Not the Waves

The models will keep changing. The ideas underneath them change more slowly. A retrieval system is still a search step wrapped around a model call. An agent is still a loop with tools and a stopping condition. A judge is still a separate check on an output. Those ideas are older than the current model, and they will outlast it.

So this book builds small versions instead of touring features. A feature list expires with the next release. A mental model of what the feature was doing survives the rename.

The waves are the model launches, the benchmark claims, and the product names. The tide is the set of ideas that keep returning under new labels. Once you can see the tide, a new tool is a new arrangement of parts you already recognise.

The same principle applies to trust. We trust compilers because they are stable, tested and contract-bound. AI has not earned that trust yet. Understanding the layer beneath the black box is how you decide what can be trusted and what still needs a check.

Who This Book Is For

- Daily users of AI tools who want to know what is underneath them
- Students and teachers who want the concepts without a vendor tutorial
- Speak Software graduates who directed agents and now want to see the machine
- Programmers who will build on real frameworks, but want to understand first

What This Book Is Not

It is not a Python course. Appendix A teaches every construct the book uses, in ten pages. For the language itself, read *Code Python, Consult AI*.

It is not *Speak Software*. That book puts a department of AI agents at your command, no code required. This one opens the box those agents are built on. If you want to run AI without ever touching code, read that one first; if you want to know what you were running, read this one.

It is not *Ship Python, Orchestrate AI*. You will build workflows, loops and judges here in miniature, to understand them. Production form is that book's job.

And it is mostly not a machine-learning book. Parts I to V keep the original promise: no training, no gradients, no maths beyond Appendix A, with the model treated through its one documented door. Part VI breaks that fence deliberately and in the open. It is optional, notebook-based, and simplified rather than production-ready, and it declares its extra prerequisites (PyTorch, and a free Colab GPU for one chapter) before asking anything of you. Skip it and the book is complete without it.

How This Book Is Structured

Part I: The Model: one call, the system prompt, statelessness, and providers. **Part II: Grounding**: what the model does not know, and retrieval that fixes it. **Part III: Action**: tools and the reasoning loop. **Part IV: Control**: workflows as folders, loops with a goal, and the judge. **Part V: The Senses**: pictures and voice. A short conclusion names the shape you will have seen five times.

Every chapter kills one misconception and leaves one small working build.

The Series

This book is part of a series on modern software development in the AI era.

The series has a trunk: *Conversation, Not Delegation* for method and *The Human Edge* for professional judgement. This book is the branch for understanding the machinery.

Start with	Continue with	Because
The Absolute Minimum You Must Know	anywhere	You want the free, always-current quick reference first
Conversation, Not Delegation	this book	You want the thinking-with-AI framework, then to see the machine
Speak Software	this book	You direct agents already; now look underneath them
Code Python, Consult AI	this book	You want the language and the machine it talks to

Think Python, Direct AI, *Ship Python, Orchestrate AI* and the

rest deepen the tracks. All titles are at books.borck.education (https://books.borck.education).

Ways to Engage with This Book

- **Read it online.** The full book is freely available at the companion website.
- **Read it on paper or e-reader.** Available as a paperback and ebook through Amazon KDP.
- **Run the package.** Install `hands-on-ai` and build every chapter's artifact yourself; the package documentation (https://michael-borck.github.io/hands-on-ai/) is the reference this book narrates.
- **Run the code.** Every listing is on GitHub (https://github.com/michael-borck/build-ai-understand-ai).
- **Browse all books.** See the full series at books.borck.education (https://books.borck.education).

The online version is always the most current.

Source Code & Feedback

Found an error? Have a suggestion?

- Open an issue: https://github.com/michael-borck/build-ai-understand-ai/issues
- Email: michael@borck.me

Part I

The Model

Chapter 1

One Call to the Model

> Every AI system in this book starts the same way. Your program sends text to a model, and the model sends text back.

Package companion: the `chat` module of `hands-on-ai`. Listings in this chapter were checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "AI is magic." It is one function call over HTTP.

The build: a five-line script that asks a question and prints the answer. Then the same call again, with the request it sends printed where you can read it.

Python you need: an import, a variable, a function call, and `print`. Appendix A teaches each of those. Read the code in this chapter even if you have not read the appendix yet. No listing is longer than a paragraph.

1.1 The Concept First

When you type a question into a chat window and an answer appears, three ordinary things happen. The app takes your text and sends it across the network to a program running a model. The model produces reply text. The app receives that text and shows it to you.

The window, the typing indicator and the smooth scroll are presentation. Underneath them, text went out and text came back.

Once you see the event at that size, a useful question becomes available for every AI product you meet. What did this tool add around the plain call? A chat window adds an instruction at the front and keeps a transcript. A document helper adds a search before the call. An agent adds a loop after it. You will build each of those additions in later chapters. This chapter builds the call they all add to.

1.2 Understanding Through Real Life

Think of posting a letter to a well-read stranger who answers questions by return mail.

Your envelope needs an address. In code, the address is a server URL, such as the Ollama server running on your own machine at `http://localhost:11434`. The envelope also needs a name, because one server can hold several models. The name is a string like `llama3`. Inside the envelope sits the letter itself. The letter is a short list of messages, and each message says who wrote it. One message carries your question.

The stranger reads the whole letter, writes a reply, and sends it back. Two details of the analogy matter later in this book. The stranger answers from what is in your letter and from what they learned long before you wrote. They have no other window into your life. And when the next letter arrives, they have forgotten yours completely, unless you copy the earlier exchange into the new letter. Part I takes those two details in turn.

1.3 Connect a Model Before You Build

The build needs two things installed: the package, and a model it can reach. The free local path uses Ollama, which runs a model on your own machine and needs no account or API key.

```
pip install "hands-on-ai==0.6.0"
ollama pull llama3
handsonai doctor
```

The first command installs the version of the package this book is pinned to. The second downloads a small model. Downloading takes a few minutes the first time. The third command checks the whole path at once. It prints the server URL and model the package will use, and it tries to reach the server. Run it before you run any listing in this book. When something breaks in a later chapter, run it again first. Most first-week failures are a server that is not running or a model that was never downloaded, and `doctor` names both.

Appendix B covers the other routes: hosted providers, API keys, and the three environment variables that point the package at them. You do not need any of that yet. Chapter 4 explains why swapping providers later will change your settings and leave your code alone.

> ⚠ If the doctor cannot connect
>
> Read the line it prints before you change anything. "Could not connect" with a local address usually means Ollama is installed but not running yet. Start it, then run `handsonai doctor` again. A model name in the output that you never downloaded means the `ollama pull` step was skipped or used a different name.

1.4 The Build: Five Lines

Make a new folder for this book's work and save this as `ask.py` inside it.

```
# Listing 1.1 of 4: one call, nothing else
(ask.py)
from hands_on_ai.chat import get_response

answer = get_response("In one sentence, what does
a post office do?")
```

```
print(answer)
```

Run it.

```
python ask.py
```

The first run prints a short warm-up line while the model loads, then one sentence about a post office. Your sentence will differ from mine, and it will probably differ if you run the script twice. The model writes a fresh reply each time, so this book will not pretend that model output can be quoted word for word. Where output is shown in later chapters, it is one run's output, and yours should be similar in kind.

Read the listing line by line. The import line takes one function, `get_response`, out of the package's `chat` module and makes it available in your script. The blank line is for your eyes only. Python ignores it. The next line calls the function with one string, your question, and stores whatever comes back in a variable named `answer`. Nothing appears on screen yet. The last line prints the variable. If you delete it, the script will still make the call and still receive a reply. You will just never see the reply, and the run will look like it did nothing. Try that once.

The script contains no server address, no model name and no password. `get_response` found those in the package configuration, which `handsonai doctor` printed for you earlier. The defaults point at a local Ollama server. Settings live outside your code so that the code can stay this small. Chapter 4 opens that configuration up.

1.5 Pull Back the Curtain: Print What You Sent

You still cannot see what left your machine, so print it. This script builds the request as data and shows it to you before anything is sent. It runs without a model, without a network, and it prints the same thing every time.

```
# Listing 1.2 of 4: the request, as data
(payload.py)
import json

model = "llama3"
messages = [
    {"role": "system", "content": "You are a
    helpful assistant."},
    {"role": "user", "content": "In one sentence,
    what does a post office do?"},
]
payload = {"model": model, "messages": messages}

print(json.dumps(payload, indent=2))
```

```
{
  "model": "llama3",
  "messages": [
    {
      "role": "system",
      "content": "You are a helpful assistant."
    },
    {
      "role": "user",
      "content": "In one sentence, what does a
      post office do?"
    }
  ]
}
```

This is the letter from the analogy, written as JSON. JSON is a text format for structured data, built from names, values, lists in square brackets, and groups in curly braces. You can read this one already. At the top level there are two names. `model` says which model should answer. `messages` holds a list of two messages, and each message is a group with a `role` and a `content`.

The `user` message is yours. It is the question you typed in Listing

1.1, unchanged. The `system` message above it is the interesting one. You did not write it. `get_response` put it there, because its default instruction to the model is the string "You are a helpful assistant." Every call in Listing 1.1 quietly sent two messages, and the first one was an instruction about how to behave. Chapter 2 is entirely about that first message, and about how much of a chatbot's apparent personality is one string in a list.

The list has room to grow. A conversation is this same list with more messages in it: your turns and the model's replies, in order, resent in full on every call. Chapter 3 shows why products built on this call go to that trouble.

1.6 The Same Call Without the Package

`get_response` is a convenience. Real tools talk to models through a client library, and the package uses one too. Stripped of its retry and reporting code, the centre of `get_response` does this.

```
# Listing 1.3 of 4: the call underneath,
simplified from the package source
from openai import OpenAI

client =
OpenAI(base_url="http://localhost:11434/v1",
api_key="hands-on-ai")

response = client.chat.completions.create(
    model="llama3",
    messages=messages,
)

answer = response.choices[0].message.content
print(answer)
```

You do not need to run this. Read it for shape. A client is created with an address and a key. Local Ollama ignores the key, but the client insists on having one, so the package passes a placeholder.

The `create` call takes the model name and the same `messages` list you printed in Listing 1.2. What comes back is a response object, and the reply text sits in one specific place inside it: the first choice, its message, its content. One line fishes the string out.

One more layer sits underneath the client. The client turns that call into an HTTP request: a `POST` to the address `http://localhost:11434/v1/chat/completions`, with your JSON payload as the body. The server answers with JSON, and the client tidies it into the response object. HTTP requests are how programs ask servers for things all over the web. Your browser made several hundred of them before breakfast. A model call is one of them, with a question in the body.

So what does the package add, if the bare call takes a dozen lines? Four things you have already benefited from without noticing. It reads the server, model and key from configuration instead of hard-coding them in your script. It supplies the default system message. It tries the call a second time if the first attempt fails. And it can keep a copy of each answer in a local cache, so a classroom can rerun a lesson and get the same replies. Appendix B explains the cache. They save you from typing the surrounds while you learn the middle.

> **The reading habit for this book**
>
> Every chapter ends at a function like `get_response` and asks the same question. What does this add to the plain call? The package is small enough to answer that question by reading it. The source of `get_response` is one file, `chat/get_response.py`, in the package repository (https://github.com/michael-borck/hands-on-ai). Open it once this week. You will not understand every line yet. You will recognise the messages list and the `create` call, and that recognition is the first sign you can read the machinery.

1.7 What an Answer Costs

The server counts your text and the model's text in tokens. A token is a chunk of text, often a short word or part of a long one. Providers bill by tokens, and every model has a limit on how many tokens one call can carry. You do not need exact token arithmetic. You need the habit of looking at the count, because two later chapters turn on it: conversations get expensive as the resent list grows, and retrieval works by spending tokens on evidence.

Ask `get_response` to report the count.

```
# Listing 1.4 of 4: the same call, with the
receipt (ask_with_usage.py)
from hands_on_ai.chat import get_response

answer, usage = get_response(
    "In one sentence, what does a post office
    do?",
    return_usage=True,
)

print(answer)
print(usage)
```

With `return_usage=True`, the function hands back two values instead of one. Your script stores them in two variables. The second value is a small dictionary with three counts.

```
{'prompt_tokens': 31, 'completion_tokens': 22,
'total_tokens': 53}
```

Your numbers will differ a little, with the same shape. `prompt_tokens` counts everything you sent. That includes the system message you did not write, which is why a ten-word question costs more than ten tokens. `completion_tokens` counts the reply. `total_tokens` adds them.

Two ordinary situations return `None` instead of counts, so they do not look like bugs later. A streamed reply, printed piece by piece

as it arrives, comes with no final count from most servers. And an answer served from the local cache was never sent to a model at all, so there is nothing to count. Code that reads `usage` should cope with `None`. For this chapter, printing it is enough.

1.8 When the Call Fails

A learning package has one design decision you should know about before it surprises you. `get_response` does not crash your script when something goes wrong. It returns a string that describes the problem, and your `print` shows it like any other answer. That is kind to beginners in a classroom. It also means the variable named `answer` sometimes holds an apology instead of an answer, and any code you build on top of the call has to be able to tell the difference.

You can see the gentlest case without any setup at all.

```
from hands_on_ai.chat import get_response

print(get_response("   "))
```

```
  Empty prompt.
```

The empty question never leaves your machine. The package checks for blank text before it builds a request. A harder failure looks like this.

```
  Error: Connection error.
```

That message means the request was built and sent, twice, and no server answered. The cause is nearly always one of three settings: the server address is wrong or the server is off, the model name does not exist on that server, or a hosted provider rejected the key. Run `handsonai doctor` and read what it resolves your server and model to before you touch your script. The script is five lines long and seldom the problem.

> **Why return an error as text?**
>
> Production code usually signals failure by raising an exception, which stops the program unless the programmer handles it. A teaching package makes a different trade. A string keeps a first lesson moving, and it makes the failure visible in the same place as the success. Later modules in the package keep the same habit. When you graduate to production libraries in Appendix D, expect exceptions there, and expect to write the handling yourself.

1.9 Where a Demo Can Mislead You

You are now equipped to spot three common exaggerations in AI product demos. Each one dissolves once you ask what was added around the call.

A product claims its assistant "knows your business". Sometimes it does, because someone pasted business documents into the messages list before your question. That is a real technique, and you will build it in Chapter 6. But the knowledge lives in the request, not in the model. Change the documents and the expertise changes with them.

A product claims its assistant "remembers you". It keeps your earlier messages and resends them, so the list carries your name into every call. The effect feels like memory because the transcript is doing the remembering. Chapter 3 builds that transcript by hand and saves it to a file you can open.

A product claims its assistant "checked the latest figures". Its code fetched those figures first and put them in the prompt, or it gave the model tools to fetch with. The bare call you ran in this chapter can do neither. Left alone, the model answers from training, and training ends at a date. Chapter 5 makes that limit visible with a question the model gets confidently wrong.

None of this makes the products dishonest. Knowing the bare call

lets you ask which addition a claim rests on, and what happens to the claim when the addition is removed.

1.10 Practice Exercises

Exercise 1.1 (Level 1). Run `ask.py`. Change the question to something you want explained at the level of a ten-year-old, and run it again. Keep both answers.

Exercise 1.2 (Level 2). Use Listing 1.4 with a short question and then with a question that pastes in a full paragraph of your own writing. Compare `prompt_tokens` for the two runs. Write one sentence about why the second call would cost more on a paid provider.

Exercise 1.3 (Level 3). Change `payload.py` so the `user` message holds your Exercise 1.1 question. Print the payload and point at each of these: the model name, the message you did not write, and your question. Then change the `model` value and say what would happen if that model were not on your server.

Exercise 1.4 (Level 4). Break the call on purpose. Run `handsonai doctor`, then set `HANDS_ON_AI_SERVER` to `http://localhost:9999` for one terminal session and run `ask.py` again. Record the exact message you get. Unset the variable, run the doctor, and confirm the script works. Write down which step told you the truth fastest.

Exercise 1.5 (Level 5). Open the source of `get_response` in the package repository. List three things it does around the bare client call, with the line or short phrase that gave each one away. Then list one thing a chat product does that this function, by itself, cannot.

1.11 Chapter Summary

- A model call sends a JSON payload over HTTP and receives text back. The payload names a model and carries a list of messages.

- `get_response` builds that payload for you, adds a default system message, reads its settings from configuration, and returns the reply as a string.
- Usage counts show what a call carries. Your prompt tokens include text you did not write.
- Failures come back as strings in this package. A blank prompt is refused locally, and a connection error means no server answered, which `handsonai doctor` can usually name.
- Most AI product behaviour is an addition around this call: a transcript for memory, fetched text for knowledge, tools for action.

1.12 Reflection

Before moving on, ensure you:

- ☐ Ran `ask.py` and got a reply from a model you connected yourself
- ☐ Can point at the model, the system message and your question inside a printed payload
- ☐ Checked `usage` for one call and can say why the prompt count is higher than your question's word count
- ☐ Saw one failure message and used `handsonai doctor` to explain it

1.13 Your Learning Journal

1. Which part of the letter analogy broke down first when you looked at the real payload?
2. What did the system message you never wrote make you want to change?
3. Pick an AI product you used this week. Which addition around the call explains its most impressive moment?

1.14 Next Steps

The payload you printed had a first message with the role `system`, and Listing 1.1 never mentioned it. In the next chapter you will write that message yourself, twice, and watch one model become two different assistants.

Chapter 2

Who the Model Is Being

> A chatbot with a personality is the call from Chapter 1 plus one string you wrote.

Package companion: the personality bots in `hands_on_ai.chat`. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "The AI has a personality." A system prompt is text in the request, and you can read it, change it, and count what it costs.

The build: one question asked three ways: with no instruction, with your own system prompt, and through a personality bot. Then your own bot, written as a function.

Python you need: passing a named argument, and defining a small function with `def` and `return`. Both are in Appendix A.

2.1 The Concept First

In Chapter 1 you printed the payload and found a message you had not written. It had the role `system` and the content "You are a helpful assistant." That message goes first in the list, before anything the user says, and the model treats it as standing instructions for the whole exchange.

A system prompt is standing instructions, written by whoever built the product and sent on every call. When a help-desk bot stays polite under provocation, when a study app answers like a tutor and refuses to just hand over the answer, when a novelty app talks like a pirate, the mechanism is the same in each case. Someone wrote a paragraph, put it at the front of the list, and the model's replies bent around it.

Two consequences follow, and this chapter demonstrates both. The personality belongs to the request, so it can be swapped between one call and the next with no change to the model. And the instruction is text like any other text in the payload, so it costs tokens on every call, whether the user sees it or not.

2.2 Understanding Through Real Life

Think of briefing a relief worker who will staff your shop counter for one shift. Before the first customer arrives, you hand them a card. The card says who they are today, how to greet people, what they may promise, and what to do when they do not know something. Every customer conversation then happens inside the frame the card set.

Change the card and the same worker runs a different counter. "You are the returns desk. Be brief. Offer a refund or a replacement, nothing else." produces a different afternoon from "You are our most patient explainer. Walk people through setup one step at a time."

A human reads the card once and remembers it. A model reads the card fresh on every call, because the card is inside every request, and your business pays for the reading each time. A long, careful card can be worth every token. It is still a running cost, and in this chapter you will watch it on the receipt.

2.3 The Build: One Question, Three Answers

Start where Chapter 1 left off. Ask a question with a system prompt of your own.

```
# Listing 2.1 of 4: your instruction, in the first
message (who.py)
from hands_on_ai.chat import get_response

question = "Why is the moon sometimes visible in
the daytime?"

print(get_response(question))
print()
print(get_response(
    question,
    system="You are a witty pirate. Talk like a
    pirate from the 1700s.",
))
```

Run it. The first answer arrives in the package's default voice, because the default system string from Chapter 1 went along as usual. The second answer explains the same astronomy in nautical slang. The model, server and question string are unchanged; one argument changed.

Read the new argument closely. `system=` is a named argument. It fills the `system` slot of `get_response`, which places your string in the first message of the payload, exactly where the default used to sit. If you want proof rather than my word for it, rebuild the payload printer from Listing 1.2 with your string in the first message and look at the output. The shape is identical and the content is yours.

Try two more while you are here. "Answer in exactly one sentence." tends to hold. "You are a calm and clear teacher. You explain concepts step by step." changes the structure of the answer as well as its flavour. A system prompt can reach tone, length, structure, and what the reply refuses to do. It cannot give the model facts it does not have, and it cannot make the model remember this

conversation tomorrow. Facts arrive in Part II. Memory arrives in the next chapter, and it arrives as plumbing, not as instruction.

2.4 The Pirate in a Box

Typing a long instruction at every call gets old fast, so the package wraps common ones in functions with friendly names. These are the personality bots.

```
# Listing 2.2 of 4: the same pirate, packaged
from hands_on_ai.chat import pirate_bot,
teacher_bot

question = "Why is the moon sometimes visible in
the daytime?"

print(pirate_bot(question))
print()
print(teacher_bot(question))
```

There is no new machinery here. Open the source of `pirate_bot` in the package repository (https://github.com/michael-borck/hands-on-ai) and you will find the whole of it, apart from its documentation string, is one line.

```
# Listing 2.3 of 4: pirate_bot, complete (from the
package source)
def pirate_bot(prompt):
    return get_response(prompt, system="You are a
    witty pirate. Talk like a pirate from the
    1700s.", personality="parrot")
```

Your Listing 2.1 and the package's Listing 2.3 make the same call. The bot just spares you typing the string. The `personality=` argument is a smaller detail with a practical job. It does not go to the model at all. It chooses which set of retry messages the package prints while it reattempts a failed call, so a pirate that hits a network error apologises in character. Two different arguments, two differ-

ent audiences: `system` instructs the model, `personality` styles the package's own chatter around the call. Keeping straight which text the model sees and which it never sees is a habit that matters more as the wrappers grow.

The package ships thirteen of these bots, from `friendly_bot` and `coder_bot` to `shakespeare_bot` and `caveman_bot`. Every one of them is a function wrapping a call with a system string.

2.5 Write a Bot of Your Own

You have already written everything a bot contains. The last step is wrapping it in a function so you can reuse it by name.

```
# Listing 2.4 of 4: your own bot (my_bot.py)
from hands_on_ai.chat import get_response


def plain_english_bot(prompt):
    return get_response(
        prompt,
        system=(
            "You explain things in plain English
            for a reader of about "
            "twelve. Use short sentences. Give one
            everyday example. "
            "If you must use a technical word,
            define it in the same sentence."
        ),
    )


print(plain_english_bot("What is compound
interest?"))
```

The `def` line names the function and its one input. The indented lines are its body. `return` hands the result of the inner call back to whoever called your bot, which is why `print` can show it. The

system string here is longer than the pirate's because it has a job to do: it sets reading level, sentence length, and a rule for technical words. Instructions that can be checked, like "define it in the same sentence", hold up better than instructions about vibes, like "be clear". You can test that claim yourself in the exercises.

Your bot takes the question as an argument but keeps the instruction fixed. That split is the product pattern from real chatbots: the builder owns the system prompt, the user owns the question, and the user never sees the builder's paragraph. When you use a polished AI product and wonder why it behaves the way it does, a paragraph like this one, written by someone on the product team and sent with every request you make, is usually most of the answer.

2.6 What the Instruction Costs

Chapter 1 ended with a receipt. Use it again, this time on your own bot's inner call. Add `return_usage=True` to the `get_response` call inside a copy of Listing 2.4 and print both values.

Compare two runs of the same question: one with the package default system string, one with your plain-English paragraph. The question did not change. The reply length will wander a little, as replies do. The pattern to look for sits in `prompt_tokens`, which rises by roughly the length of your instruction, because your paragraph rides along in every request.

On a local model this costs you a sliver of time. On a paid provider it is a per-call charge that never switches off, paid by every user on every question, including the ones who ask something trivial. Product teams feel this pressure from both sides. A longer system prompt can buy steadier behaviour, safer refusals, and a voice users recognise. It also taxes every exchange and, in a long conversation, crowds the space available for the conversation itself. The next chapter shows where that space goes. When you write an instruction, ask what it earns per token.

> **Instructions compete with each other**
>
> A system prompt that demands one-sentence answers and step-by-step explanations in the same breath forces the model to choose which order to disobey. It will choose quietly, and differently on different runs. When a bot behaves inconsistently, read its card before you blame the model. Contradictions in standing instructions produce wobble that looks exactly like model randomness.

2.7 Where a Demo Can Mislead You

A vendor introduces "our empathetic support agent" or "our strict compliance reviewer" as though a new kind of AI had arrived. You now know to ask a plainer question. What does its system prompt say? Sometimes the honest answer is a well-tested paragraph, refined over months, backed by retrieval and tools from later chapters of this book. Sometimes the honest answer is three sentences written the morning of the demo. Behaviour under pressure tells you which: ask the empathetic agent something outside its script, and watch whether the empathy survives contact with a question its card never anticipated.

The same lens works in reverse when a model behaves oddly in your own builds. The model is being who the first message told it to be, inside the limits of what text can instruct. If you do not like who that is, rewrite the first message.

2.8 Practice Exercises

Exercise 2.1 (Level 1). Run Listing 2.1. Replace the pirate string with an instruction of your own choosing: a sports commentator, a cautious lawyer, a five-year-old. Keep the question fixed across all runs.

Exercise 2.2 (Level 2). Call three packaged bots with the same question. In one sentence each, write what changed: tone, structure,

length, or content. Then find each bot's system string in the package source and check your guesses.

Exercise 2.3 (Level 3). Write a bot for a real job: a study helper that gives hints before answers, a meeting summariser limited to decisions and owners, an explainer for your own workplace jargon. State one rule in your system prompt that a reader could check in the output, and check it across three runs.

Exercise 2.4 (Level 4). Measure the card. Run one question under the default system string, your Exercise 2.3 string, and a deliberately bloated version padded with repeated instructions. Record `prompt_tokens` for each. At a million calls, which version would you ship, and what would you cut first?

Exercise 2.5 (Level 5). Break a bot politely. Ask your Exercise 2.3 bot a question its instruction never anticipated, then a question that asks it to ignore its instruction. Record what it does in each case. Rewrite the system prompt once to handle the worse failure, and say what your rewrite costs in tokens.

2.9 Chapter Summary

- A system prompt is the first message in the payload: standing instructions written by the builder, sent on every call, invisible to the user.
- Personality bots are functions that wrap `get_response` with a fixed system string. Thirteen ship with the package, and yours takes four lines.
- The `system` argument instructs the model. The `personality` argument never reaches the model; it styles the package's retry messages.
- Instructions cost `prompt_tokens` on every call. Checkable rules hold better than vague ones, and contradictory rules produce inconsistent behaviour.

2.10 Reflection

Before moving on, ensure you:

- ☐ Asked one question under at least three different system prompts
- ☐ Read the source of one packaged bot and found its system string
- ☐ Wrote and ran a bot function of your own
- ☐ Compared `prompt_tokens` between a short and a long instruction

2.11 Your Learning Journal

1. Which instruction changed the answer's structure rather than just its tone?
2. What rule did you write that the model followed only sometimes? How could you make it checkable?
3. Think of an AI product you use. Draft the three sentences you suspect sit in its system prompt.

2.12 Next Steps

Your bots have character and no memory. Ask `plain_english_bot` your name in one call and it will answer warmly; ask in the next call what your name is and it will have no idea. The next chapter finds out why, and builds the transcript that fixes it.

Chapter 3

No Memory Between Calls

> The model forgets you completely between calls. Anything it appears to remember, your code sent again.

Package companion: `Conversation` in `hands_on_ai.chat`. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "It remembers me." Every call is stateless. Memory is a transcript you keep and resend.

The build: a chat that learns your name and uses it. Then its memory, saved to a JSON file you can open, edit, and reload.

Python you need: creating an object from a class, calling its methods with dot notation, and reading a list. Appendix A covers the patterns. The word class is explained where it first matters, below.

3.1 The Concept First

At the end of Chapter 2 your bot could not recall your name from one call to the next. That was not a flaw in your bot. It is how the call works. The server receives a payload, produces a reply, and keeps nothing. Your next call is, to the model, the first thing that has ever happened.

Chat products hide this with bookkeeping. The app stores every message, yours and the model's, in order. On each new turn it sends

the whole list again: the system instruction from Chapter 2 at the front, then the full conversation so far, then your new question at the end. The model reads the transcript, sees your name in an earlier message, and answers as though it remembered. The memory lives in the app and in the payload, and the model rereads it at full price on every turn.

Three familiar behaviours of chat products follow from this mechanism. Long conversations get slower and dearer, because each turn resends everything before it. Very long conversations start dropping their beginnings, because the payload has a size limit, called the context window. And a fresh chat feels like a stranger, because it is one: a new list with nothing in it but the instruction card.

3.2 Understanding Through Real Life

Return to the well-read stranger who answers your letters, from Chapter 1. Imagine now that the stranger has a strict rule: every letter they receive is shredded the moment the reply is posted. If you want a continuing correspondence, there is exactly one way to have it. Each time you write, you copy the entire exchange so far into your new letter: your first question, their reply, your follow-up, their reply, and finally today's question at the bottom.

Your letters get longer every week. Postage rises with length, so a long correspondence costs more per letter than a short one, even when today's question is a single line. Eventually the envelope reaches the largest size the post office will carry. Past that point something has to give. You can stop writing, start a new correspondence, or quietly leave the earliest pages out of the copy. The stranger will then answer as though those early exchanges never happened, because for them, they never did.

The envelope limit is the context window, measured in tokens. The rising postage is the per-call token cost you met in Chapter 1. The missing early pages are why a long chat can forget the name you gave it an hour ago while remembering what you said a minute ago.

3.3 First, Watch the Forgetting

Prove the statelessness to yourself before building the fix. Two separate calls, using the function you already know.

```
# Listing 3.1 of 3: two calls, no memory
(forgetful.py)
from hands_on_ai.chat import get_response

print(get_response("My name is Sam. Remember
it."))
print(get_response("What is my name?"))
```

The first reply will be warm and reassuring. The second will admit it does not know, or guess, or explain that it has no record of earlier conversations. The model said it would remember because that is a natural thing to say in conversation. The words were produced by the same process that produces everything else it writes. No storage happened behind them. When a person says "I will remember that", a memory forms. When a model says it inside a stateless call, only the sentence exists.

3.4 The Build: a Conversation That Resends

The package does the bookkeeping in a class named `Conversation`. A class is a template for making objects that bundle data with the operations that belong to it. In practical terms for this chapter: you create one conversation object, it holds the message list inside itself, and you talk to it through a method named `ask`.

```
# Listing 3.2 of 3: the same exchange, with a
transcript (remembering.py)
from hands_on_ai.chat import Conversation

chat = Conversation(system="You are a helpful
tutor.")

print(chat.ask("My name is Sam."))
```

```
print(chat.ask("What is my name?"))
print("Tokens so far:", chat.total_tokens)
```

Run it, and the second answer gets your name right. Read the listing slowly, and compare it with Listing 3.1. The first line after the import creates the object and hands it the instruction card. That card becomes the first and, for now, only message in its list. The first `ask` appends your message to the list, sends the entire list to the model, receives the reply, and appends the reply to the list as well, under a third role you have not met yet: `assistant`. The second `ask` appends your new question and sends all four messages. The model sees "My name is Sam." in the transcript and answers from it.

The last line prints the running token account. `Conversation` adds up the usage the server reports after each turn and keeps the total where you can read it. Ask a few more questions and print it again. Then do the arithmetic Chapter 1 prepared you for: the tenth turn of a conversation resends the nine before it, so a conversation costs more than the sum of its questions, and the gap widens with every turn. On a local model you pay in waiting time. On a metered provider you pay in money, for rereading.

> **i** Three roles, one list
>
> You have now seen every role a message can carry in this book's early chapters. `system` holds the builder's standing instructions. `user` holds what the person typed. `assistant` holds what the model replied, saved so it can be resent. When you read a saved transcript in the next section, roles are how you tell who said what, and the model relies on the same labels.

3.5 The Memory Is a File

If the memory is a list your code keeps, then it can be written down. `Conversation` will do it for you.

```
# Listing 3.3 of 3: save the transcript, load it
back (save_chat.py)
from hands_on_ai.chat import Conversation

chat = Conversation(system="You are a helpful
tutor.")
chat.ask("My name is Sam.")
chat.save("my_chat.json")

later = Conversation.load("my_chat.json")
print(later.ask("What is my name?"))
print(later.history())
```

Open `my_chat.json` in any text editor before you run the second half. It is plain JSON, the same format as the payload from Chapter 1, holding five things: the system prompt, the model setting, the personality used for retry messages, the complete `messages` list, and the token total. The object that felt like a mind a moment ago serialises into a page of structured text, and `load` rebuilds the object from that page in a later run, on a later day, on a different machine. The reloaded chat answers with your name because the file carried the transcript across the gap. The model was never consulted about any of it.

First, edit the file by hand. Change “Sam” to “Priya” inside the saved user message, reload, and ask again. The model will call you Priya with complete confidence, because you edited the only record it ever reads. Second, call `later.reset()` and ask once more. `reset` throws the transcript away and keeps only the system prompt, and the name is gone. Products dress these two operations in friendly buttons: edit history, delete conversation. Underneath, someone is keeping, editing, or discarding a list.

The `history()` call in the last line returns the transcript without the system message, which is the view a product would show its user. The instruction card stays in the payload and out of sight, as Chapter 2 described.

3.6 The Window Has Edges

The context window is the largest payload a model will accept, counted in tokens, instruction and transcript and new question together. Windows vary widely by model, from a few thousand tokens to well over a hundred thousand, and the model name in your configuration decides which one you have. Your local model will tell its limit in its documentation, and `handsonai models` can show what your server reports.

`Conversation` in this package resends everything and trims nothing. That honesty is deliberate in a learning tool. It means a very long chat with this class will eventually produce an error from the server, or a reply that behaves as though the beginning never happened, depending on how the server handles an overlong request. Production chat products choose a trimming strategy instead: keep the most recent turns, summarise the older ones into a short paragraph that rides along in place of the originals, or pin a few facts while letting chatter scroll away. Each strategy loses something different, and each is a decision a person made about what the conversation is allowed to forget, and most products never tell their users which decision was taken.

One more edge is already visible in your build. When a call fails, the package returns an error string, as Chapter 1 explained, and `Conversation` appends that string to the transcript as the assistant's turn. A failed exchange becomes part of the record the model rereads on the next turn. In a short lesson that is harmless. In a long session it can confuse matters.

> ⚠ The transcript is evidence, and it is also a liability
>
> A saved conversation is a complete record of what was said, in a file anyone with the file can read. That makes transcripts wonderful for debugging, teaching, and the audit habits later chapters build. It also means a chat about a medical worry, a legal problem, or a business secret leaves a plain-text copy wherever the app stored it. Before you paste anything sensitive

> into any AI product, ask where its transcript lives and who can open it. You now know there always is one.

3.7 Where a Demo Can Mislead You

"It remembers everything about you" is a claim about a list. The questions that follow from this chapter are concrete ones. How much of the transcript is resent, and what gets trimmed when the window fills? Where is the transcript stored, for how long, and can you read or delete the actual file rather than trusting a button? When the product says it learned your preferences, did anything change anywhere except the text prepended to your next call? A vendor with good answers built deliberate bookkeeping. A vendor who talks about the model getting to know you is describing that bookkeeping.

3.8 Practice Exercises

Exercise 3.1 (Level 1). Run `forgetful.py` and `remembering.py` with your own name. Copy the exact wording of the reassuring reply in the forgetful version into your journal.

Exercise 3.2 (Level 2). Extend `remembering.py` to five turns about a topic you choose. Print `chat.total_tokens` after each turn. Explain, in two sentences, why the jump between turns grows even when your questions stay short.

Exercise 3.3 (Level 3). Save a conversation, open the JSON, and change one fact in an earlier user message. Reload and ask about the fact. Then reset the loaded conversation and ask again. Record both answers and what each proves.

Exercise 3.4 (Level 4). Have a conversation long enough to hurt: paste a few paragraphs of text into several turns and watch the token total. Look up your model's context window. Estimate how many turns like yours would fill it, and say what you would trim first if this were a product.

Exercise 3.5 (Level 5). Write a ninety-word privacy note for a fictional chat product built on `Conversation`. It must state where the transcript is stored, how a user can read it, how a user can delete it, and what happens to failed turns. Plain English, no marketing words.

3.9 Chapter Summary

- The model keeps nothing between calls. A conversation product stores the full message list and resends it on every turn.
- `Conversation.ask` appends your turn, sends the whole list, appends the reply under the `assistant` role, and accumulates the reported token usage.
- `save` writes the system prompt, settings, messages, and token total to a JSON file. `load` rebuilds the conversation from it. Editing the file edits the memory.
- The context window caps the payload in tokens. Long conversations cost more per turn and eventually force a trimming decision.
- A failed call returns an error string that `Conversation` records as the assistant's turn, so failures become part of the transcript.

3.10 Reflection

Before moving on, ensure you:

- ☐ Watched two separate `get_response` calls forget your name
- ☐ Ran a `Conversation` that used your name correctly on a later turn
- ☐ Opened a saved chat file and found your name inside the messages list
- ☐ Changed a fact in the file and saw the reloaded chat believe the edit

3.11 Your Learning Journal

1. What did the forgetful bot's reassurance teach you about reading model output?
2. Which trimming strategy would you choose for a study tutor, and what would it lose?
3. Where do the transcripts of the AI products you already use most likely live?

3.12 Next Steps

Everything so far has run on one local model, chosen for you by a configuration file you have not opened. The next chapter opens it. You will point the same scripts at a different provider and a different model, change no code, and find out how much of this book so far was about AI in general rather than about one vendor in particular.

Chapter 4

Any Provider Will Do

> Your scripts name no vendor. Three settings decide whose model answers, and all three live outside your code.

Package companion: configuration and the response cache in `hands-on-ai`. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "I am locked into one vendor." OpenAI-compatible endpoints are a common language, and your code already speaks it.

The build: the Chapter 1 script, unchanged, run against a second provider. The only edits happen in your terminal, and the doctor confirms each switch.

Python you need: none beyond reading environment variables as named settings. One listing sets them from inside Python, for notebook users.

4.1 The Concept First

Look back at every script you have written so far. None contains a server address, and none names Ollama or any company. The most vendor-specific thing in them is a model name you chose in a payload exercise. And yet the calls worked, because something

outside your code supplied the address, the model, and the key. That something is the package configuration, and this chapter opens it.

The configuration can stay outside because one company's API shape became the common dialect. A request is a `POST` to an address ending in `/v1/chat/completions`, carrying the JSON payload from Chapter 1, with the key sent as a standard authorisation header. Dozens of providers now accept that exact shape, whatever their own models are called. Tools that speak the dialect, including this package, can be pointed at any of them by changing an address and a key. The industry calls such services OpenAI-compatible, after the company whose shape won.

Compatibility has limits, and a later section names them. Within those limits, the practical consequence is large: choosing a provider is a settings decision, reversible in a minute, and the code you write against the dialect survives the change.

4.2 Understanding Through Real Life

Think of the SIM card in a phone. The handset holds your apps, your habits, and everything you have built up. The SIM decides whose network carries your calls and whose bill arrives. Swapping networks means swapping a small card and confirming the new signal works. Nobody expects to replace the handset to change carriers, and a phone that demanded it would be returned to the shop.

Your scripts are the handset. The three settings in this chapter are the SIM: the server address, the model name, and the key. A provider who wants you locked in will try to move pieces of value out of your handset and into their network, into features only their dialect extensions can reach. That can be a fair trade when you choose it knowingly. This chapter exists so that you always know which parts of your setup would travel and which would not.

4.3 The Three Settings

The package resolves its configuration in a fixed order, weakest first: built-in defaults, then a config file at `~/.hands-on-ai/config.json`, then environment variables, which win over everything. Chapter 1 ran on the defaults: a local Ollama server at `http://localhost:11434`, the model `llama3`, and no key, because a local server does not check one.

Environment variables are named settings your terminal hands to every program it launches. Three of them steer everything in Part I:

Variable	Sets	Local default
`HANDS_ON_AI_SERVER`	The provider's base address	`http://localhost:11434`
`HANDS_ON_AI_MODEL`	Which model answers	`llama3`
`HANDS_ON_AI_API_KEY`	The key, where one is required	unset

The address does not need the `/v1` ending; the client adds it when it is missing, so addresses copied from provider documentation work either way. And when no key is set, the client still sends a harmless placeholder, because the underlying library insists on a key-shaped string even though your local server ignores it. You met that placeholder in Listing 1.3.

`handsonai doctor` prints the resolved result of all this: the config file it read, the server and model it will use, and whether a key is configured, shown masked. Whenever you change a setting, run the doctor before you run a script. It reads the same configuration your code will read, in the same order, so an argument with the doctor is an argument you do not have to have with a traceback.

4.4 The Build: Same Script, Second Provider

You need an account and a key from one hosted provider for this section. Appendix B lists routes, including free tiers and local-only alternatives, and explains what a key is and how to guard it. If you have no key and want none, read this section as a worked example and do the two-model variant in Exercise 4.2 instead.

With `ask.py` from Chapter 1 in front of you, set the three variables for a hosted provider. This example uses OpenAI's own service; the package documentation gives the same three lines for OpenRouter, Groq, Together, and Gemini's compatible endpoint, among others.

```
# Listing 4.1 of 3: point the same code at a
hosted provider (terminal)
export HANDS_ON_AI_SERVER="https://api.openai.com"
export HANDS_ON_AI_API_KEY="your-key-here"
export HANDS_ON_AI_MODEL="gpt-4o-mini"

handsonai doctor
python ask.py
```

The doctor should now report the hosted address, the new model, and a configured key. Then `ask.py` runs, the file untouched since Chapter 1, and a model on another company's computers answers your post-office question. Run `remembering.py` from Chapter 3 next. It works too, transcript and all, because nothing in it ever knew which server would answer.

Provider-specific code would have made this section a rewrite. Settings made it four lines in a terminal.

Notebook users can set the same variables from inside Python, before the first call. The package reads the environment when configuration is loaded, so the timing matters: set first, import after, or set before the first `get_response` call at the latest.

```
# Listing 4.2 of 3: the same settings, set from
Python (notebooks)
import os
```

```
os.environ["HANDS_ON_AI_SERVER"] =
"https://api.openai.com"
os.environ["HANDS_ON_AI_API_KEY"] =
"your-key-here"
os.environ["HANDS_ON_AI_MODEL"] = "gpt-4o-mini"

from hands_on_ai.chat import get_response

print(get_response("In one sentence, what does a
post office do?"))
```

> **A key is a password that spends money**
>
> Treat an API key like a card number. Anyone holding it can run up charges on your account until you revoke it. Keep keys out of scripts you share, out of screenshots, and out of any repository you publish. Environment variables exist partly so the secret can live outside the code. If a key ever appears somewhere public, revoke it first and tidy up second.

When you are done, return to local runs by unsetting the three variables or opening a fresh terminal, then run the doctor to confirm the local address is back. Settings set with `export` live only in that terminal session, which makes experiments like this one easy to undo.

4.5 Local and Hosted: an Honest Comparison

Local Ollama and a hosted provider are both legitimate places to run this book, and they trade differently. A local model is free per call, works offline, and keeps your prompts on your own machine, which matters for the transcript concerns raised in Chapter 3. Its limits are your hardware's limits: smaller models, slower replies, and a smaller context window than the big hosted offerings. A hosted model reverses the trade. It is stronger and faster, it costs money per token, and every prompt travels to someone else's computers

under their retention rules.

Neither choice is the permanent one. A classroom can run Monday's lesson locally and Friday's on a hosted model for the harder project. A prototype can start where iteration is free and move when quality demands it. What you should not do is let the choice harden into your code by accident, one vendor-specific feature at a time, until moving would mean starting over.

Compatibility also has edges. The dialect covers the request and reply shape for chat, which is everything Part I has used. It does not standardise every extra a provider offers. Model names differ and must be exact; `handsonai models` lists what your current server reports, which settles most name disputes. Usage reporting varies, so `usage` can be `None` on one provider and complete on another. And two models given identical payloads will still answer differently, in quality, tone, and how literally they follow your system prompt. Portability moves your code without making the models interchangeable, and Part IV's judge exists partly because of that gap.

4.6 The Cache: When Reproducible Beats Fresh

One more setting belongs in this chapter because it changes where answers come from. The package can keep a copy of every reply on disk and serve repeats from the copy instead of calling the model again. It is off until you switch it on:

```
# Listing 4.3 of 3: turn on the response cache
(terminal)
export HANDS_ON_AI_CACHE=~/.hands-on-ai/cache
python ask.py
python ask.py
```

The first run calls the model and stores the reply in a plain text file inside that directory, named by a hash of the three things that define the call: model, system prompt, and question. The second run finds

the file and prints the stored reply without contacting any server. You can prove it to yourself by running the second one with Ollama stopped. It still answers, instantly, in the exact words of the first run.

In a classroom, a teacher can warm the cache once, then thirty students rerun the lesson offline and see identical output, and nobody's exercise breaks because a model felt whimsical that morning. Chapter 11's judge needs stable material to judge, and the cache is how this book gets it. The costs are the mirror image of the benefits. A cached answer never improves when you switch to a better model, because the model is part of the lookup only as a name in the hash, and a stale cache can confuse you into thinking your new settings changed nothing. Cached replies also arrive with `usage` set to `None`, as Chapter 1 warned, because no tokens were spent. And the cache is one more folder of plain-text transcripts on disk, subject to the same care as the saved chats in Chapter 3. Clearing it is deleting the directory.

4.7 Where a Demo Can Mislead You

"Powered by our proprietary AI platform" sometimes means a team built something substantial: retrieval, tools, evaluation, the subjects of Parts II to IV. Sometimes it means a system prompt and an account with a hosted provider, dressed in platform language. You can now estimate the difference from the outside. Ask what would happen to the product if its model provider doubled prices tomorrow. A team that can answer "we change three settings and re-run our evaluations" owns its handset. A team that goes quiet has rented the network and the phone from the same shop, and its pricing page is hostage to someone else's.

4.8 Practice Exercises

Exercise 4.1 (Level 1). Run `handsonai doctor` and write down the resolved server, model, and key status. Then find which layer each

value came from: default, config file, or environment. `handsonai config` can help.

Exercise 4.2 (Level 2). No hosted key required. Download a second local model with `ollama pull`, switch only `HANDS_ON_AI_MODEL`, and run `ask.py` and Listing 2.1's pirate under both models. Record one difference in the answers that no setting caused.

Exercise 4.3 (Level 3). With a hosted key, run the Chapter 3 remembering script against the hosted provider with no code changes. Compare `total_tokens` and the reply wording with your local run. Switch back to local and confirm with the doctor.

Exercise 4.4 (Level 4). Turn the cache on, run `ask.py` twice, then stop your local server and run it a third time. Open the cache directory, find the file your answer lives in, and explain its name. Delete the directory and describe what the next run must do.

Exercise 4.5 (Level 5). Write a one-page provider plan for a fictional school deploying these scripts to two hundred students. Cover where prompts travel, what a term of usage plausibly costs under each option, what happens on the day the provider has an outage, and how the cache changes the plan. End with the condition under which you would switch.

4.9 Chapter Summary

- Configuration resolves in order: defaults, then `~/.hands-on-ai/config.json`, then environment variables. `handsonai doctor` shows the resolved result.
- Three variables steer Part I: `HANDS_ON_AI_SERVER`, `HANDS_ON_AI_MODEL`, and `HANDS_ON_AI_API_KEY`. The client adds `/v1` to the address when missing.
- OpenAI-compatible endpoints give many providers one request shape, so unchanged scripts run against local Ollama or hosted services. Model names, usage reporting, and answer quality still differ.
- Local runs are free, private, and hardware-limited. Hosted runs are stronger, metered, and send prompts to the provider.

- `HANDS_ON_AI_CACHE` turns on a plain-text reply cache keyed by model, system prompt, and question, for reproducible and offline reruns.

4.10 Reflection

Before moving on, ensure you:

- ☐ Can name the layer each of your current settings came from
- ☐ Ran one earlier script against a second model or provider without editing it
- ☐ Saw the doctor confirm a settings change before a script relied on it
- ☐ Turned the cache on, found your answer's file, and cleared it

4.11 Your Learning Journal

1. What surprised you most: that the code travelled unchanged, or that the answers still differed?
2. For your own likely use, which matters more: prompts staying on your machine, or the strongest available model? What would flip your answer?
3. Where in your current digital life have you rented the handset and the network from the same shop?

4.12 Next Steps

Part I is complete. You can make the call, instruct the model, keep the transcript, and choose who answers. Every reply so far, though, came from whatever the model happened to learn during training, and training stopped on a date. Part II begins with a question your model gets confidently wrong, and then builds the fix: putting evidence into the payload yourself.

Part II

Grounding

Chapter 5

What the Model Doesn't Know

> A model answers from what it learned in training and what you put in the payload. Everything else is improvisation, delivered in the same confident voice.

Package companion: `get_response` from Part I, used twice on the same question. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "It knows everything." Training has a cutoff and a domain, and confidence is a style, not a measurement.

The build: a question your model cannot possibly answer from training, asked bare, then asked again with the answer pasted into the prompt by hand.

Python you need: a multi-line string and an f-string, both in Appendix A.

5.1 The Concept First

Every answer in Part I came from one of two places. The model was trained, once, on a large collection of text, and that training ended on a particular date with a particular coverage. Call that

everything the model learned at school. The second source is your payload: the system instruction, the transcript, and your question. There is no third source. The bare call you have been making cannot look anything up, cannot check today's news, and cannot read your documents unless you put them in the request.

Trouble starts because the model does not experience the boundary between those sources as a wall. Its job is to produce a plausible continuation of the conversation, and it does that job with the same fluency whether the material under it is solid training knowledge, your supplied text, or nothing at all. Given a question it cannot answer from either source, it will often still answer, composing something that sounds like the kind of reply such a question usually gets. The industry calls an invented answer of that kind a hallucination. The word suggests a malfunction, and that suggestion misleads: the machinery did what it always does, in a place where it had nothing true to work with. Hallucination is a property of the setup, best managed by changing the setup.

If invented answers are what a model produces when the payload lacks the facts, then the fix is to put the facts in the payload and instruct the model to stay inside them. This chapter does that by hand, with copy and paste. The next chapter automates it.

5.2 Understanding Through Real Life

Think of a brilliant, well-read colleague who joined your organisation this morning. Ask her how double-entry bookkeeping works and you will get a superb answer: she learned it long before she met you. Ask her what your team's refund limit is, and a careful person says she does not know yet and asks where the policy lives. A model, asked the same question in the same bare way, frequently produces a number. The number will be formatted like a policy. It may even be a sensible number. It will not be your limit, because your limit is written in a document she has never seen, and no amount of general brilliance substitutes for reading it.

The failure sits with the request: somebody asked for an internal fact

without handing over the internal document, and accepted fluency as a substitute for the hand-over. Organisations run on facts of the kind no training set contains: this term's timetable, your pricing, your house rules, what was decided in Tuesday's meeting. Every one of those questions needs the document in the room.

5.3 The Build: Ask for What It Cannot Have

You need a fact the model cannot have from training. The surest supply is a small organisation of your own invention. This listing asks about one.

```
# Listing 5.1 of 2: a question with no source
(cannot_know.py)
from hands_on_ai.chat import get_response

question = (
    "According to the Borck Street Book Club
    borrowing rules, "
    "how many books can a member borrow at once, "
    "and what is the fine per week for a late
    return?"
)

print(get_response(question))
```

Read the reply with a pen in hand. Models vary, and yours may hedge, ask for the document, or answer outright. If it hedges well, ask a follow-up that presses for the numbers, the way an impatient user would. If it answers outright, look at what it gave you: specific figures, probably capped by a plausible policy voice. Those figures came from nowhere checkable. There is no Borck Street Book Club, or if the name collides with something real by accident, your model has never read its borrowing rules. Whatever the reply says, it is not based on the rules, because the rules were never in the payload and training never contained them.

Two features of the reply recur everywhere. The first is borrowed

confidence. The answer's tone matches the tone it uses for bookkeeping, where it is usually right. Tone is generated along with everything else and tracks the style of the question, not the quality of the evidence. The second is plausible specificity. Invented answers tend to arrive with numbers, names, and structure, because real answers have those things and the model has read a great many real answers. Specificity is cheap to produce and expensive to check, which is why invented policies, invented citations, and invented product features slip past busy readers.

Try a second family of unknowable questions while you are here: the very recent past. Ask who won a match played yesterday, or what a public figure said this morning. Training ended before those events, so a bare local model has no way to know. Some hosted products answer such questions well, and Chapter 4 taught you how to read that: something around the call fetched the news first. Your bare call has no such addition, so it guesses, hedges, or answers from stale training while sounding current. The cutoff is true of every model on the day it ships.

5.4 The Same Question, With the Evidence In the Room

Now write the rules yourself, put them in the prompt, and ask again.

```
# Listing 5.2 of 2: grounding by hand
(grounded_by_hand.py)
from hands_on_ai.chat import get_response

rules = """
Borck Street Book Club: borrowing rules (2026)
- A member may borrow up to 4 books at once.
- The loan period is 3 weeks.
- The fine for a late return is $0.50 per book per
week.
- A lost book is charged at replacement cost plus
a $5 fee.
```

```
"""

question = (
    "Based only on the rules above: "
    "how many books can a member borrow at once, "
    "and what is the fine per week for a late
    return?"
)

prompt = f"Rules:\n{rules}\n\nQuestion:
{question}"

print(get_response(
    prompt,
    system="Answer only from the text provided. "
           "If the text does not say, reply that
           the rules do not say.",
))
```

The reply now has somewhere true to stand, and you can check it against the four lines you wrote. Three changes did the work. The evidence is inside the payload, as part of the prompt string. The question tells the model to base its answer on that text. And the system prompt, the standing instruction from Chapter 2, gives it a permitted way to decline: if the text does not say, say so. A model with no graceful exit will usually improvise one, and giving it the exact sentence to use when the evidence runs out is a small, repeatable defence.

Test the exit. Add a second question your rules never address, about guest passes or meeting nights, and watch whether the reply uses your escape sentence or quietly invents a policy anyway. Models differ in obedience here, and instruction alone is a soft guardrail: it lowers invention, it does not abolish it. Chapter 11 returns to measuring how often such instructions hold. Record what yours did.

You selected relevant text. You pasted it into the prompt ahead of

the question. You instructed the model to answer from the text and to say when the text runs out. The only part you did personally, by hand, with copy and paste, is the part Chapter 6 hands to code.

5.5 What Asking Bare Questions Costs

A bare question about general knowledge is cheap and often fine. A bare question about your documents, your customers, or last week costs you a check you probably will not do, because the answer arrives formatted as though the check already happened. The expensive moment is the decision made from a fluent invention by someone who had no reason to suspect it.

Three questions sort most requests before you send them. Could the model have learned this in training, from public text written before its cutoff? Is the fact stable, or could it have changed since? And if the answer were invented, who would find out, and when? A question that fails the first two tests and embarrasses you on the third does not need a better model. It needs its evidence in the payload, which from the next chapter onward your code will supply.

> **i** Confidence is not a dial you can read
>
> People sometimes ask a model how confident it is, hoping the number separates knowledge from invention. Treat the reply as more generated text. A model can produce "I am 95% certain" for the same reason it produces any fluent sentence: such sentences appear in its training, attached to right and wrong answers alike. The checks that actually separate the two are external: evidence in the prompt, a source you can open, a second method, a judge measured against examples. This book builds those, in that order.

5.6 Where a Demo Can Mislead You

A demo question is chosen by the demonstrator, and demonstrators instinctively choose questions training covers well. The model

shines, and the audience generalises the shine to their own questions, which live in documents training has never seen. When you watch any AI demo from here on, keep one question of your own ready: a fact from your world, recent or internal, that you can verify on the spot. Watch what the system does with that question, and whether it can show you its source.

5.7 Practice Exercises

Exercise 5.1 (Level 1). Run `cannot_know.py` three times. Note whether the invented numbers stay the same across runs. What does their stability, or lack of it, tell you?

Exercise 5.2 (Level 2). Ask your bare model about an event from the last seven days and one from five years ago, both of which you can check. Record accuracy and tone for each. Which answer sounded more confident, and was it the right one?

Exercise 5.3 (Level 3). Replace the book club rules with a short real document of yours: a timetable, a price list, a policy page. Ask two questions the document answers and one it does not. Check all three replies against the document, line by line.

Exercise 5.4 (Level 4). Weaken Listing 5.2 in stages: remove the system prompt's escape sentence, then remove "based only on the rules above" from the question. Re-ask the unanswerable question at each stage. Which removal cost you more honesty?

Exercise 5.5 (Level 5). Write a five-question trap set for a bare model: one internal fact, one recent event, one stable public fact, one question about a document only you have, one trick with a false premise. Predict the outcome of each in writing before you run them. Score your predictions as well as the model.

5.8 Chapter Summary

- A bare model answers from training and from your payload. There is no third source, and training has a cutoff and a cov-

erage limit.

- Given a question neither source can answer, the model often improvises in the same fluent voice. Hallucination is a property of that setup, not a malfunction.
- Grounding by hand fixes the setup: put the evidence in the prompt, instruct the model to answer from it, and give it a permitted sentence for when the evidence runs out.
- Confidence in a reply, including a self-reported percentage, is generated text. It does not measure evidence.
- Before asking, sort the question: could training know it, is it stable, and who catches an invention?

5.9 Reflection

Before moving on, ensure you:

- ☐ Collected one answer your model could not have sourced, and can say why
- ☐ Re-asked with evidence in the prompt and checked the reply against your text
- ☐ Tested the escape sentence with a question your evidence does not answer
- ☐ Can explain a training cutoff without saying the model is broken

5.10 Your Learning Journal

1. Which invented detail in your bare answer was most believable, and what made it so?
2. Where in your own work are people asking bare questions about internal facts?
3. What escape sentence would you want every AI tool in your organisation to use?

5.11 Next Steps

Grounding by hand works for four lines of rules. It fails for four hundred pages of documents, because the context window from Chapter 3 will not hold them and your paste hand will tire. The next chapter keeps the shape and replaces your hand with code: split the documents into chunks, fetch the chunks a question needs, and assemble the prompt you built manually here.

Chapter 6

Chunk, Fetch, Answer

> Retrieval-augmented generation is the paste you did by hand in Chapter 5, with code doing the searching and the pasting.

Package companion: `hands_on_ai.rag`, plus the `rag` command-line tool. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "RAG is a framework." It is search, then a prompt with the search results in it.

The build: `first-rag-bot`, the package's first RAG project. Index a small folder of documents, then ask questions and get answers that name their sources.

Python you need: loops, lists, dictionaries, and f-strings building a long string. All are in Appendix A.

6.1 The Concept First

Chapter 5 ended with a shape: select relevant text, paste it into the prompt ahead of the question, instruct the model to answer from the text. The technique has a name in the research literature, retrieval-augmented generation, from the paper that popularised it [@lewis2020rag], and the industry shortens it to RAG. The name sounds like a product category, but nothing has changed about the

call. The model still receives one payload and returns text. What changed is who does the selecting and the pasting: code, in the seconds before the call.

The work splits into two phases that happen at different times. Indexing happens once, before any question is asked: load the documents, cut them into chunks, turn each chunk into numbers a search can compare, and save the lot to a file. Answering happens per question: turn the question into numbers the same way, fetch the chunks whose numbers sit closest, assemble them into a prompt of exactly the shape you built by hand, and make the familiar call. Every concept in this chapter lives in one of those two phases, and keeping them apart in your head prevents most confusion about RAG systems, including expensive commercial ones.

6.2 Understanding Through Real Life

Picture a reference librarian working from index cards. Long before you arrive, someone read every book in the collection and wrote a card per passage: a short excerpt, the book it came from, and a code describing what the passage is about. That is indexing, done once, at leisure. When you arrive with a question, the librarian does not reread the library. She works out which code your question carries, pulls the handful of cards whose codes sit closest, and hands you those passages with their book titles attached. You read five passages instead of five shelves, and you can check each claim against the book named on its card.

Two details of the card system carry over. The codes were designed so that passages about similar things get similar codes, even when they share no wording: a passage about returning library books late and your question about overdue fines should land near each other. And the system can only ever hand you cards that exist. A book nobody indexed is invisible, however relevant, and a card written from last year's edition keeps describing last year's rules until somebody redoes the indexing.

6.3 Connect a Second Model Before You Build

The codes on the cards are produced by an embedding model. An embedding model is a relative of the chat models you have used so far, with a narrower job: it reads a piece of text and returns a long list of numbers, called an embedding or a vector, that captures what the text is about. Texts about similar things produce lists that point in similar directions, and comparing directions is arithmetic your code can do quickly. The package configuration names an embedding model separately from the chat model, defaulting to `nomic-embed-text`. Pull it the same way you pulled your chat model:

```
ollama pull nomic-embed-text
handsonai doctor
```

The doctor prints the embedding model alongside the chat model. From this chapter on, a working setup means two models on the server: one that talks, one that codes text for search. Nearly every first RAG failure in a classroom is the second model missing, and the error it produces mentions embeddings, which is your cue.

You also need documents. The package ships sample texts about networking, and one function copies them into a folder of your own:

```
# Listing 6.1 of 4: get the sample documents
(get_docs.py)
from hands_on_ai.rag import copy_sample_docs,
list_sample_docs

print(list_sample_docs())
print(copy_sample_docs("sample_docs"))
```

The folder contains a handful of files in the formats the package can load: `.txt`, `.md`, `.docx`, and `.pdf`. Reading them needs no model at all. Text and markdown files are read as they are, Word documents surrender their paragraphs, and PDFs surrender the text of each page. Your own documents can join the folder at any time, as long as they wear one of those four endings.

6.4 Chunk: Cut the Documents Into Cards

Indexing begins with cutting. `load_text_file` reads a file into one long string, and `chunk_text` splits the string into chunks of a set number of words. The configured default is 500 words per chunk. That number is far too large to see anything at a glance, so this listing uses five:

```
# Listing 6.2 of 4: chunking, at a size you can
see (chunks.py)
from hands_on_ai.rag import chunk_text

text = (
    "A member may borrow up to four books at once.
    "
    "The loan period is three weeks."
)

for chunk in chunk_text(text, chunk_size=5):
    print(repr(chunk))
```

```
'A member may borrow up'
'to four books at once.'
'The loan period is three'
'weeks.'
```

Read that output closely. The cutting is purely mechanical: count words, slice, repeat. It knows nothing about sentences, so a rule can be severed from its number, as "four" nearly was here. Real chunk sizes soften this. At 500 words, most facts survive inside one chunk, and a question usually needs only one or two chunks to answer. The tension remains: small chunks make search precise and risk amputating the answer. Large chunks keep context intact and spend your token budget on text the question never needed, three chunks at 500 words being a fair slice of a modest context window before the conversation even starts.

`chunk_text` also does not overlap chunks. Production systems often let neighbouring chunks share a margin of text so a severed sentence

appears whole in at least one card. This package keeps the simpler cut, and the simplicity is instructive. When your bot later misses an answer that straddles a boundary, you will know a named cause to investigate first.

6.5 Fetch: Numbers That Find Their Neighbours

With chunks cut, indexing finishes the cards. `get_embeddings` sends each chunk to the embedding model, through the same server and the same kind of client as Chapter 1's call, at the related address `/v1/embeddings`. Back comes one list of numbers per chunk. `save_index_with_sources` then writes a single file, conventionally `index.npz`, holding three parallel lists: the vectors, the chunk texts, and for each chunk the path of the file it came from. That third list is the book title on the librarian's card, and it is what later lets an answer name its sources.

Fetching at question time mirrors the process. The question goes to the embedding model and comes back as one more list of numbers. Then a similarity score, cosine similarity, compares the question's direction with every chunk's direction and produces a number per chunk: near 1 for chunks about much the same thing, lower as topics drift apart. `get_top_k` sorts those scores, takes the best `k` of them (three unless you say otherwise), and returns the winning chunks paired with their sources. Stripped to its skeleton, the whole retrieval is four moves: load the index file, embed the query, score every vector against it, keep the top few. A vector database, when a vendor mentions one, is a faster and larger home for those same four moves.

Similarity search has two limits. Closeness is about topic, not about truth or importance, so a chunk can score well by discussing your subject without containing your answer. And the question's wording steers the fetch: asking with the documents' vocabulary retrieves better than asking around the subject, which is why rephrasing a failed question is a real debugging step and not superstition.

6.6 Answer: Assemble the Prompt You Already Know

The package's own `rag ask` command builds the prompt by listing the fetched chunks under a "Context:" heading, appends an instruction to answer from the context and to say so when the context lacks the answer, sets a system prompt demanding answers based only on the provided context, and calls `get_response`. It is Listing 5.2 with the paste automated. This listing does the same job in your own code, printing the sources so you can check them:

```
# Listing 6.3 of 4: fetch, assemble, answer
(ask_docs.py)
from hands_on_ai.chat import get_response
from hands_on_ai.rag import get_top_k

question = "What does TCP do?"

results = get_top_k(question, "index.npz", k=3)

prompt = f"Question: {question}\n\nContext:\n"
for chunk, source in results:
    prompt += f"- {chunk}\n"
prompt += (
    "\nAnswer the question based on the provided
    context. "
    "If the context doesn't contain the answer,
    say so."
)

answer = get_response(
    prompt,
    system="You are a helpful assistant that
    answers questions "
           "based only on the provided context.",
)
```

```
print(answer)
print("\nSources:")
for chunk, source in results:
    print("-", source)
```

Before this can run, an index must exist. The command-line tool builds one from a folder in a single step, which is how the package's `first-rag-bot` project starts:

```
# Listing 6.4 of 4: index once, ask with receipts
(terminal)
rag index sample_docs --force
rag ask "What does TCP do?" --context --scores
```

By default the index lands at `~/.hands-on-ai/index.npz`, where `rag ask` looks for it, and `--force` permits rebuilding over an older index. The two flags on the question are the chapter's debugging instruments. `--context` shows the chunks the fetch chose, and `--scores` shows how close each one scored. When an answer disappoints, look there before you touch the prompt. Most bad RAG answers are faithful summaries of the wrong chunks, and no instruction to the model can repair evidence that was never fetched.

Run both routes on the networking samples. Ask what TCP does, how HTTP works, and what distinguishes TCP from UDP, the starter questions from the project guide. Then ask something the documents cannot answer, about book club fines, say, and watch the escape instruction do the job you wrote it for in Chapter 5. Finally, repeat the grounded exercise with documents of your own: a course outline, a policy, a manual.

> ⚠ The index is a snapshot
>
> Indexing reads the documents as they were at indexing time. Edit a document afterwards and the bot keeps answering from the old text, confidently and with citations, until somebody rebuilds the index. Stale indexes are the quietest failure in RAG systems because nothing errors: the fetch works, the answer

> is fluent, the source file exists. Only the content moved. Any team running RAG in earnest needs a rebuild habit tied to how its documents change, and now you know why that sentence belongs in their design rather than their apologies.

6.7 Where a Demo Can Mislead You

"Chat with your documents" is this chapter, productised. Chunking choices, embedding quality, retrieval tuning, and fresh indexes at scale are real engineering, and good implementations answer noticeably better than naive ones. The questions that separate them are the ones you can now ask. How are documents chunked, and can the vendor show you the chunk an answer came from? How many chunks ride along per question, and what happens to token cost at your volume? How fresh is the index relative to the documents, and who rebuilds it? What does the system do, observably, when the answer is not in the collection? A vendor who answers with chunks, scores, and a rebuild schedule is giving you evidence you can check.

6.8 Practice Exercises

Exercise 6.1 (Level 1). Copy the sample documents, build the index, and ask the three starter questions with `--context`. For one answer, open the named source file and find the sentence the answer rests on.

Exercise 6.2 (Level 2). Rebuild the index at a chunk size of 50 words, then at 500. Ask the same five questions against each and keep score: answered well, answered from the wrong chunk, or declined. Which size served your questions better, and what did it cost?

Exercise 6.3 (Level 3). Index a document of your own choosing, at least a few pages. Write three questions it answers and two it does not. Run all five with `--scores` and record whether the unanswerable ones declined gracefully.

Exercise 6.4 (Level 4). Engineer a severed fact. Write a two-sentence document where a small chunk size must cut between a rule and its number, index it at that size, and ask for the number. Then fix the failure twice: once with chunk size, once with question wording. Which fix generalises?

Exercise 6.5 (Level 5). Take a RAG product or pitch you can access, demo pages included. Write the four questions from "Where a Demo Can Mislead You" against it, answer each from public material where you can, and mark the ones the vendor leaves unanswerable. Conclude with what you would pilot before paying.

6.9 Chapter Summary

- RAG automates Chapter 5's manual paste: index once, then per question, fetch relevant chunks, assemble a context prompt, and make the ordinary call.
- Indexing loads files (`.txt`, `.md`, `.docx`, `.pdf`), cuts text into word-count chunks with no overlap, embeds each chunk, and saves vectors, chunks, and source paths in one `index.npz` file.
- An embedding model turns text into numbers whose direction encodes topic. `get_top_k` embeds the question, scores every chunk by cosine similarity, and returns the best few with their sources.
- Chunk size trades precision against intact context and token cost. Retrieval closeness measures topic, not truth, and question wording steers the fetch.
- `--context` and `--scores` expose the fetch for debugging. Most bad answers are good summaries of wrong chunks, and a stale index fails silently.

6.10 Reflection

Before moving on, ensure you:

- ☐ Built an index and can name the three things saved inside it

- ☐ Asked a question and traced its answer to a chunk in a named source file
- ☐ Saw scores for a fetch and explained one disappointing ranking
- ☐ Asked a question outside the collection and watched the decline behaviour

6.11 Your Learning Journal

1. Which chunk size surprised you, and what did the scores show at each size?
2. Describe a question your wording sabotaged. How did you rephrase it?
3. For a document set you care about, how often would the index need rebuilding, and who would notice if nobody did?

6.12 Next Steps

Part II is complete: your model now answers from evidence you control, fetched by code. So far the model can only talk, though. It cannot check a file, run a calculation reliably, or fetch anything on its own initiative. Part III gives it hands, carefully, by letting it ask your code to run a function and report back.

Part III

Action

Chapter 7

Giving the Model Hands

> A tool is an ordinary Python function. The model's new power is being allowed to ask for it by name.

Package companion: `hands_on_ai.agent`, its tool registry, and the built-in calculator. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "Agents are products you buy." An agent begins the moment a model may request a function and your code decides whether to run it.

The build: one registered calculator the model can ask for, called directly so you can watch it work without the model, then a tool of your own.

Python you need: functions and function arguments from Appendix A. One system in this chapter passes a function itself as an argument, and that move is explained where it happens.

7.1 The Concept First

Everything your model has done so far ends in text: it answers, adopts a voice, rereads a transcript, and summarises fetched chunks. It cannot add two numbers reliably, read today's date, look inside a file, or check anything outside the payload. Tool use changes the shape of the exchange. Alongside your question, the payload now

carries descriptions of functions your code is willing to run. The model may reply with a request to run a named function with a given input. Your code runs it, or refuses to, and sends the result back as another message. The model then answers with the result in hand.

The model never executes anything; it cannot. It produces text, and one shape of that text is a request for a named function. Your program parses the request, checks the name against a registry you control, calls the function with the requested input, and returns the output as text. Every security question in this chapter, and most of Part IV, grows from that division of labour: the model proposes, your code disposes.

Three pieces make the arrangement work. The registry is a table of allowed functions, each with a name and a description. The description is what the model reads, so the description is the interface: a tool whose description is vague or wrong will be called wrongly, however good the function is. And the result comes back as a message in the transcript, an observation, where the model can read it on the next turn: another job for Chapter 3's list.

7.2 Understanding Through Real Life

Think of advising a colleague by phone while they stand in a workshop you cannot enter. You cannot touch anything. What you can do is say: use the torque wrench on the second bolt, and tell me what the gauge reads. Your colleague, who has the hands, decides whether that instruction makes sense, carries it out, and reports the reading. You reason again from the report.

The voice on the phone can be brilliant and can also ask for the wrong tool, the right tool with a nonsense setting, or a tool that does not exist on the bench. The person holding the tools stays responsible for what gets picked up. When a product advertises an agent that "takes action", translate it into this scene and ask who built the bench, who stocked it, and what the hands refuse to do.

7.3 The Build: One Tool the Model Can Request

The package ships a calculator tool, and registering it takes one call. Registration adds an entry to the registry table: a name, a description for the model to read, and the function itself.

```
# Listing 7.1 of 3: a model that may ask for a
calculator (first_tool.py)
from hands_on_ai.agent import list_tools,
register_calculator_tool, run_agent

register_calculator_tool()
print(list_tools())

answer = run_agent("What is 25 * 3?")
print(answer)
```

The `list_tools()` line prints the registry as the model will effectively meet it: a list of names with descriptions. Run the script and read that output before the answer arrives. Then watch what the run returns. Somewhere between your question and the printed answer, the model asked for the calculator with the input `25 * 3`, your side of the program evaluated it, and the result rode back into the conversation. Chapter 8 slows that exchange down until every hop is visible.

You will not see the request itself in this run. By default, `run_agent` chooses how the model should phrase its requests based on the model you configured. Capable models get a readable text format, the subject of Chapter 8. Small local models, including the default from Part I, get routed to a stricter JSON format, where the request arrives as structured data naming a thought, a tool, and an input, validated before anything runs. Either way, what reaches your registry is a name and a string.

7.4 Call the Tool Yourself First

Take the model out of the loop entirely to understand a tool. The calculator is a Python function, and you can call it the way you called every function in Appendix A:

```
# Listing 7.2 of 3: the tool, no model involved
(tool_directly.py)
from hands_on_ai.agent.tools.calculator import
calculator

print(calculator("2 + 2 * 10"))
print(calculator("{'expression': '2 + 2 * 10'}"))
```

```
Result: 22
Error: Could not evaluate expression. Dict is not
allowed
```

The first line is the tool working as intended. The second line fails. Look back at the description `list_tools()` printed in Listing 7.1: its example input is written as `{'expression': '2 + 2 * 10'}`, dictionary-style. The function, however, receives one string and evaluates the expression inside it, so a model that follows the description's example literally will send the dictionary text and get an error in return. The failure sits in the interface between them: one sentence of description.

When a tool fails inside an agent run, the failure does not stop the program. The error text becomes the observation sent back to the model, which gets a chance to read it and try again with a corrected input. The same happens when the model invents a tool name: the reply from your side reads `Error: Tool 'nonexistent' not found.` It is a forgiving design for learning, and a design whose risks the safety section below takes seriously, because a model that can retry can also be steered by whatever text a tool returns.

While you have the tool open in front of you, look at how it evaluates. Your first instinct for a calculator might be Python's built-in `eval`, which runs a string as code. The package refuses that route on purpose. Its calculator parses the expression into a tree and walks

only the arithmetic nodes it allows: numbers, the basic operators, a short list of named functions. A request to open a file or reach the operating system finds no path in. That is why the dictionary input failed with “Dict is not allowed” instead of being politely evaluated. The allowlist rejected a whole category of input without caring whether this particular instance was harmless.

7.5 Register a Tool of Your Own

A tool of yours needs three things the calculator already showed you: a function that takes one string and returns text, a name, and a description written for its reader, who is a model deciding from that sentence alone whether and how to call it.

```
# Listing 7.3 of 3: your own tool on the bench
(my_tool.py)
from hands_on_ai.agent import list_tools,
register_calculator_tool, register_tool, run_agent


def count_words(text: str) -> str:
    count = len(text.split())
    return f"The text has {count} words."


register_calculator_tool()
register_tool(
    "count_words",
    "Count the words in a piece of text. Input:
    the text itself, "
    "with no quotes or labels around it.",
    count_words,
)

print(list_tools())
print(run_agent("How many words are in this
sentence: the model proposes and the code
```

```
disposes"))
```

One line in that listing does something new. The third argument to `register_tool` is `count_words` with no parentheses after it. That hands the function itself to the registry without calling it, so the registry can call it later when a request arrives. Parentheses would run it immediately, with no input, and register the resulting text instead, which fails in ways worth seeing once in the exercises.

The description earns its keep by preventing the failure you just studied. It names the input's exact shape, including what not to add around it. When you write descriptions of your own, write the example input as the literal string the function should receive, then test the description the way Listing 7.2 tested the calculator's: feed the tool exactly what its description tells the model to send. If that string errors, the description is wrong, whatever the function does with well-formed input.

7.6 The Bench Rules: Safety Lives on Your Side

Everything in this section follows from one fact: text produced by a model, influenced by whoever wrote the prompt and whatever the retrieved documents say, is about to select functions in your process. Treat tool design as designing what that text can reach.

Stock the bench narrowly. A calculator that accepts arithmetic only, a word counter, a lookup that reads one file: each tool should do one small thing, take one plainly shaped input, and have no side effects you would mind explaining. The dangerous tool is the general one, a `run_command` or `evaluate_code` that turns model text into machine instructions. The package's own calculator shows the alternative pattern: parse the input, walk an allowlist, reject everything else wholesale.

Check what each tool actually returns before you trust it in a chain. The package is honest about its teaching tools, and reading two of them is a useful exercise in professional suspicion. Its weather tool is labelled simulated and returns random conditions. Its search tool

is also simulated, answering from a small set of canned passages. An agent using them will produce fluent answers grounded in theatre, and nothing in the final answer warns the reader. Tools are evidence sources like the chunks in Chapter 6, and they deserve the same question: where does this text actually come from?

The model can ask, repeatedly and creatively, and the error-as-observation design means it gets feedback and retries. Your registry is the complete list of what asking can achieve. A function that is not registered cannot be called, whatever the prompt says, and a registered function should be safe when called with the worst string a model can produce, as well as the strings you imagined. Chapter 10 adds stopping rules and budgets around the asking.

> **i** Descriptions are prompts too
>
> The tool descriptions ride inside the system prompt the agent sends on every call, so they cost prompt tokens on every iteration, exactly as Chapter 2's standing instructions do. They also compete with each other: two tools with overlapping descriptions will be confused with each other, and the fix is usually in the wording, not the model. When an agent picks the wrong tool, read the bench card before blaming the voice on the phone.

7.7 Where a Demo Can Mislead You

Agent demos love the moment the system "does something" in the real world: sends the email, updates the record, books the thing. The model requested an action in text, and somebody's code decided the request was acceptable and carried it out. What is the full registry this product exposes, and which entries change the world rather than read it? What inputs are validated against an allowlist, and what happens on the worst string, not the demo string? Which tools are real integrations and which are simulations wearing the same costume? And when a tool errors or returns hostile text, does the loop retry blindly, because a system that treats tool output as trusted

instruction has handed the phone to whoever wrote the tool's data.

7.8 Practice Exercises

Exercise 7.1 (Level 1). Run `first_tool.py`. Then change the question to one needing no tool at all, and one the calculator cannot help with. Record what the agent does in each case.

Exercise 7.2 (Level 2). Using Listing 7.2, test the calculator with five inputs: a valid expression, the dictionary-style example, a word, an empty string, and an attempt to reach beyond arithmetic. Tabulate which returned results and which returned errors.

Exercise 7.3 (Level 3). Write and register a tool of your own that does one verifiable thing: count sentences, look up a word in a small dictionary you define, or convert a temperature. Test it directly first, including with the exact example string from your own description.

Exercise 7.4 (Level 4). Deliberately register your Exercise 7.3 tool with a misleading description, then ask the agent a question that needs it. Record the failure. Rewrite only the description and ask again. What does the comparison teach you about where agent bugs live?

Exercise 7.5 (Level 5). Audit a bench. List every tool the package's `agent ask` command registers (read its command source), classify each as reads-the-world or changes-something and as real or simulated, and write the one-sentence bench rule you would impose before connecting it to a system that mattered.

7.9 Chapter Summary

- A tool is a Python function taking one string and returning text, registered under a name with a description the model reads.
- The model proposes by emitting a request in text or JSON. Your code checks the name against the registry, executes, and returns the result as an observation message.

- The description is the interface. The package calculator's dictionary-style example contradicts its function, and feeding the tool its own example proves it without any model.
- Tool failures return error strings as observations, so the model can read them and retry. Unknown tools fail the same way.
- Safety lives in the registry and in allowlist parsing. The built-in calculator walks an arithmetic tree and rejects whole categories of input; simulated tools return theatre in the same shape as evidence.

7.10 Reflection

Before moving on, ensure you:

- ☐ Registered a tool and read the registry as the model meets it
- ☐ Called a tool function directly, with good and bad input
- ☐ Registered a function you wrote, without parentheses after its name
- ☐ Can state, for any tool on your bench, whether it is real or simulated

7.11 Your Learning Journal

1. Which failure taught you more: the dictionary-style input, or an invented tool name?
2. Write the description for a tool you wish your workplace systems offered a model. What input shape did you specify?
3. Where in a product you use might a simulated tool be passing as a real one?

7.12 Next Steps

One tool request is a single step. Real work needs several: fetch a number, compute with it, check the result, answer. The next chapter puts the step in a loop with a memory, reads the package's own loop

line by line in spirit, and asks the question every loop must answer: what makes it stop?

Chapter 8

The ReAct Loop

> Thought, action, observation, repeat. The thinking is text you can read, and the loop is code you already know how to read.

Package companion: `run_agent` in `hands_on_ai.agent`, forced to its readable text format. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "The agent thinks." It reasons in text, acts through your functions, observes the results, and repeats, inside a loop with stated stopping conditions.

The build: a two-tool agent answering a question neither tool can settle alone, run in the format where every step is legible.

Python you need: the loop idea from Appendix A at reading level. The one new pattern, reasoning written out as labelled text, is the chapter itself.

8.1 The Concept First

Chapter 7 gave the model one move: request a function, receive the result. Most interesting tasks need the move repeated with judgement about what to do next. ReAct, short for reason and act, is the pattern that organises the repetition [@yao2023react].

The model writes its reasoning as text labelled `Thought:`, follows it with a requested `Action:` and `Action Input:`, your code appends the result as an `Observation:`, and the model reads the growing transcript and writes the next thought. When it judges the question answered, it writes `Final Answer:` instead. The transcript of an agent run is Chapter 3's message list wearing name tags.

The reasoning is ordinary model output, produced the same way as any paragraph in this book, which means you can read it, save it, argue with it, and catch it in an error before the error becomes an action. The loop that drives it is a short Python `for` loop in the package: call the model, scan the reply for an action or a final answer, run any requested tool, append the observation, go around again. An agent, at this size, is a loop and a labelling convention around the call from Chapter 1.

8.2 Understanding Through Real Life

Think of working a problem aloud with a notebook while a colleague fetches things. You say what you are trying to establish. You ask for one specific thing: the manual, a measurement, a date from the file. Your colleague brings it, you write down what it shows, and the written record changes what you ask for next. At some point the notebook contains enough, and you say the answer out loud.

The notebook is doing quiet work in that scene. Because each step was written, your colleague can spot that you asked for the wrong manual before the wrong part gets ordered. You can backtrack to where a wrong reading entered. And anyone later can replay how the answer was reached, step by step, instead of trusting that it was. An agent trace is that notebook. Treat products that hide the trace the way you would treat a colleague who will not show their workings.

8.3 The Build: Two Tools, One Question Each Cannot Finish

Register the calculator beside your word counter from Chapter 7, then ask a question that needs both, in sequence. To watch the steps in their readable form, this listing asks for the ReAct text format explicitly:

```
# Listing 8.1 of 2: the loop, in legible format
(two_tools.py)
from hands_on_ai.agent import
register_calculator_tool, register_tool, run_agent


def count_words(text: str) -> str:
    count = len(text.split())
    return f"The text has {count} words."


register_calculator_tool()
register_tool(
    "count_words",
    "Count the words in a piece of text. Input:
    the text itself, "
    "with no quotes or labels around it.",
    count_words,
)

answer = run_agent(
    "How many words are in the sentence 'the model
    proposes and the code "
    "disposes', and what is that number squared?",
    format="react",
    verbose=True,
)
print(answer)
```

With a capable model, the exchange inside that run reads like this.

Wording will vary by model and run, as always; the labels and the order are the fixed part:

```
Question: How many words are in the sentence 'the
model proposes and
the code disposes', and what is that number
squared?
Thought: I need a word count first, then a square.
The count_words
tool takes the text itself.
Action: count_words
Action Input: the model proposes and the code
disposes
Observation: The text has 7 words.
Thought: Now I need 7 squared, which is a job for
the calculator.
Action: calculator
Action Input: 7 ** 2
Observation: Result: 49
Thought: I have both numbers.
Final Answer: The sentence has 7 words, and 7
squared is 49.
```

Read the transcript as a work record. The first thought decomposes the question into an order of operations. The first action's input respects the description you wrote in Chapter 7: the text itself, no labels. The observation is your function's return value, word for word, entering the transcript where the model can use it. The second thought plans from a fact it did not have one turn earlier, supplied by your code rather than its training and carried by Chapter 3's resent list. Both numbers in the final answer came from tools whose work you can inspect.

The readable format is the demanding one. The model must keep the labels exact while reasoning, and small local models often drop a colon, rename a label, or answer inside a thought. That is why `run_agent` defaults to choosing the format for you, routing smaller models to the JSON format from Chapter 7 where a schema vali-

dates each reply before anything runs. Forcing `format="react"` on a small model is a fine experiment and a common first failure. Second, `verbose=True` routes the steps through the package's logging, which stays quiet at its default level. To see the steps live, run with the environment variable `HANDS_ON_AI_LOG=debug` set, the same debugging switch the configuration chapter prepared you for.

8.4 Read the Loop

The package's ReAct loop fits on a screen, and reading it in `agent/core.py` is this chapter's version of Chapter 1's payload print. In spirit, stripped of its logging and fallback handling, it does this:

```
# Listing 8.2 of 2: the loop's skeleton,
simplified from agent/core.py
messages = [system_message_with_tools,
your_question]

for i in range(max_iterations):
    response =
    get_response(prompt=transcript_so_far,
    system=system_message)

    if "Final Answer:" in response:
        return the_final_answer

    tool_calls = parse_action_lines(response)
    if not tool_calls:
        return response

    for tool_name, tool_input in tool_calls:
        result = execute_if_registered(tool_name,
        tool_input)
        append response, then f"Observation:
        {result}", to messages
```

```
return a_partial_answer_or_an_admission
```

The system message is Chapter 2's instruction card, here carrying the format rules and the tool descriptions from Chapter 7. The transcript resent on every iteration is Chapter 3's list, which means each turn of the loop is a full model call at full token cost, a fact Part IV's budgets take seriously. The parser is unglamorous text matching: lines beginning `Action:` and `Action Input:` are paired up in order, which is as fragile as it sounds and explains the JSON alternative. Execution is Chapter 7's registry check, including the error-string behaviour for unknown names and failed tools, fed back as the next observation. If someone describes an agent framework to you in terms that cannot be mapped onto this skeleton plus better furniture, ask which line of the skeleton their feature replaces.

8.5 What Makes It Stop

A loop that can always take one more step will, on some question, take all of them. This one has three exits.

The intended exit is the model's own verdict: a `Final Answer:` line, whose text `run_agent` extracts and returns. The second exit is quieter. If a reply contains no tool request and no final answer, the loop treats the reply itself as a direct answer and returns it. That covers questions needing no tools, and also covers a model that lost the format mid-run, which matters when you debug: a plausible-looking reply may be a formatting failure wearing an answer's clothes. The third exit is the budget: `max_iterations`, five by default. When the loop exhausts it, the package tries to salvage the last thought into a partial report, beginning along the lines of "I've been working on this but haven't reached a final answer", and otherwise admits it reached the maximum number of tool calls.

Each exit has a misuse to watch. A model can declare a final answer early, from training, without calling the tools you stocked for exactly that question; the trace exposes it, which is one more reason to keep traces. A model can also circle: request the same tool with the same input, receive the same observation, and think the same

thought again until the budget ends. And because every iteration resends the transcript, a looping agent's turns get dearer as they get less useful. Chapter 10 puts goals and ratchets around repetition. Before running any agent, including one in a product, ask what its three exits are and which one fired.

> **Read one trace against the grain**
>
> Take a run that produced a correct final answer and reread its thoughts critically. A right answer can travel through a wrong reason ("squaring counts the words twice") rescued by a tool computing correctly anyway. The answer grades well and the reasoning would fail the next question. Traces let you mark the reasoning, which is the part that generalises, and Chapter 11's judge automates a version of exactly that marking.

8.6 Where a Demo Can Mislead You

Agent demos are edited like cooking shows: a question goes in, a flurry of impressive steps crosses the screen, a polished result comes out. Can I read the full trace of that run, including the tool inputs and the raw observations? How many model calls did it make, and what would they cost at my volume? Which exit ended the run, and what does the trace look like on the runs that hit the iteration budget? Where the vendor says the agent "reasoned", can they show you the sentences, and do the sentences actually support the answer? A demo that cannot show a trace asks you to trust the model's own account of its work.

8.7 Practice Exercises

Exercise 8.1 (Level 1). Run the two-tool build with debug logging on. Save the full trace. Mark each line with its author: model, tool, or loop.

Exercise 8.2 (Level 2). Ask the same agent a question needing only the second tool, then one needing neither. Identify which loop exit

ended each run, from the trace.

Exercise 8.3 (Level 3). Set `max_iterations=1` and ask the two-step question again. Record exactly what comes back. Then raise the budget until the run completes, and note the smallest budget that suffices.

Exercise 8.4 (Level 4). Induce a circle: describe one of your tools so vaguely that the model keeps calling it with unhelpful inputs. Capture the trace, find the first repeated action, and fix the description rather than raising the budget. Compare traces.

Exercise 8.5 (Level 5). Write the trace policy for a fictional workplace agent: what is logged, who may read traces, how long they are kept, and what a weekly review of a sample of traces looks for. One page, ending with the failure your policy exists to catch.

8.8 Chapter Summary

- ReAct organises tool use as labelled text: `Thought`, `Action`, `Action Input`, `Observation`, repeated until `Final Answer`. The reasoning is readable output.
- The driver is a short loop over `get_response`: parse action lines, execute registered tools, append observations to the transcript, resend everything next iteration.
- `run_agent` defaults to choosing a format by model size, routing small local models to validated JSON. Forcing `format="react"` buys legibility and demands a model that can hold the format.
- Three exits stop the loop: a final answer, a reply with no tool request treated as the answer, and the `max_iterations` budget, five by default, after which the package salvages a partial report.
- Traces are work records. They expose early final answers, circling, wrong reasoning behind right answers, and the per-iteration token cost.

8.9 Reflection

Before moving on, ensure you:

- ☐ Watched a two-tool run and can attribute every trace line to model, tool, or loop
- ☐ Identified which exit ended at least two different runs
- ☐ Hit the iteration budget deliberately and read what came back
- ☐ Read your trace against the grain and found one reasoning step to argue with

8.10 Your Learning Journal

1. Where in your trace did the model's written reasoning most diverge from what the tools actually established?
2. How many iterations did your hardest question need, and what did each one cost in transcript growth?
3. Describe a task where you would refuse a correct answer because the trace could not be shown to you.

8.11 Next Steps

Part III is complete: the model can request work, and the loop can string requests into a plan. What neither chapter gave you is control at scale: work reviewed stage by stage, repetition aimed at a goal rather than merely permitted, and a way to judge output you cannot read personally. Part IV builds those three controls, starting with the most mundane one in this book: folders.

Part IV

Control

Chapter 9

Workflows as Folders

> A workflow can be a folder of numbered stages, each one a readable file, with a person reviewing the output before the next stage runs.

Package companion: `hands_on_ai.workflow`, the `Pipeline` class and `init_workspace`. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "Orchestration needs a framework." Numbered folders of reviewable stages are enough to see, steer, and restart a multi-step job.

The build: a three-stage pipeline, research then outline then draft, run one stage at a time with your review in between.

Python you need: objects and methods from Appendix A, at the level Chapter 3 used them.

9.1 The Concept First

Part III chained steps inside one run, at machine speed, in a transcript you had to go looking for. That suits a two-step calculation. It suits a report, a lesson plan, or a grant draft much worse, because the moment you most want is the pause between steps: read what the research stage gathered before the drafting stage builds on it, fix

the outline before prose hardens around it, stop the line cheaply. A workflow makes that pause part of the structure.

The package implements workflows as folders. A workspace holds numbered stage folders. Each stage folder holds a `CONTEXT.md`, plain-language instructions for that stage, and an `output` folder. Running a stage means assembling one familiar call: a shared system instruction, the stage's contract, any reference rules, and the previous stage's output file as input. The reply is written to `output/output.md`, and then everything stops until a person has read that file and asked for the next stage. The package calls the method ICM, for Interpretable Context Methodology. The name and the folder-as-agent architecture come from Jake Van Clief and David McDermott [@vanclief2026icm]. The context at every step is a file a person can open, rather than state inside a framework you must take on faith.

There is no scheduler, message queue, graph language, or agent hierarchy. The number in the folder name gives the order, the file system gives the memory (Chapter 3's saved-transcript trick at a larger grain), and you, with a text editor, do the review. Because the input to each stage is literally the previous output file, your edits become that input word for word.

9.2 Understanding Through Real Life

Think of a production line with inspection benches between stations. Work arrives at a station with its job card, gets transformed, and sits in the out-tray until an inspector passes it to the next station. The inspector can send a piece back, correct a small flaw by hand and pass it on, or stop the whole line when a station's job card turns out to be wrong. Nobody confuses the line with magic because you can walk it, bench by bench, and see the state of every piece at a glance.

The folder workspace is that line. Stage folders are stations, `CONTEXT.md` files are job cards, `output.md` in the out-tray is the work so far, and `status()` is the glance down the line. The contrast with Chapter 8's loop is deliberate: the ReAct loop optimises for a

machine deciding its own next step quickly; the pipeline optimises for a person deciding whether the last step deserves a next one. Most work worth doing with AI has a cheapest point of intervention, and it is usually between stages, before an error gets three stages of polish applied to it.

9.3 The Build: Three Stages, Two Pauses

One function lays out the line:

```
# Listing 9.1 of 2: lay out the workspace
(make_pipeline.py)
from hands_on_ai.workflow import Pipeline,
init_workspace

init_workspace(
    "report_workspace",
    stages=["research", "outline", "draft"],
    system="You are helping prepare a short
    briefing for a school board. Be accurate,
    plain-spoken, and brief.",
)

pipe = Pipeline("report_workspace")
print(pipe.status())
```

```
  [ ] 01_research
  [ ] 02_outline
  [ ] 03_draft
[('01_research', False), ('02_outline', False),
('03_draft', False)]
```

Open the workspace in your file browser and look at what was made. A top-level `CONTEXT.md` holds the shared instruction, which becomes the system prompt for every stage, as Chapter 2's card did for one call. Under `stages/`, each numbered folder holds its own `CONTEXT.md`, created as a stub asking you to describe the stage's job, and an empty `output` folder. A `references/` folder waits at the top

level for stable rules, house style, a marking rubric, anything every stage should obey; the runner folds every markdown file in it into each stage's prompt under a references heading. The package calls that folder the factory, by which it means the part of the setup you tune once while the workpieces flow through.

Your job before any model runs is to write the three job cards. In `01_research/CONTEXT.md`, describe the gathering job: the topic, what counts as a useful finding, and the shape to return, perhaps five findings each with a one-line implication. In `02_outline`, ask for an outline built only from the research output, with a named section per decision the board must make. In `03_draft`, ask for prose following the outline, at a stated length, introducing no fact the research stage did not supply. Writing those cards is Chapter 2's instruction craft and Chapter 5's grounding discipline, applied per stage. Vague cards produce vague stations, and no pipeline design rescues them. A `CONTEXT.md` job card is a skill in embryo: a reusable instruction file an agent loads when a task matches. When the same card survives a second run, it can become a reusable skill file. *Build Web, Guide AI*, "The Agent-Ready Repo and Handover", shows that progression.

Then run the line, one station at a time:

```
# Listing 9.2 of 2: run, review, run
(run_pipeline.py)
from hands_on_ai.workflow import Pipeline

pipe = Pipeline("report_workspace")

print(pipe.run_next()["stage"])   # research runs;
read its output.md now
# Open stages/01_research/output/output.md.
# Fix it, trim it, or delete it and run this stage
again.

print(pipe.run_next()["stage"])   # outline, built
from your edited research
```

```
# Open stages/02_outline/output/output.md and
review again.

print(pipe.run_next()["stage"])   # draft
print(pipe.run_next())            # None: nothing
left to run
```

Each `run_next` call finds the first stage whose output file is missing or empty, assembles its call, writes the reply, and returns a small dictionary naming the stage, the output path, and the text. When every stage has a non-empty output, it returns `None` and does nothing. That definition of done, a non-empty `output.md`, is simple enough to reason about at the bench. Edit an output and you have edited the next stage's input. Delete an output and that stage becomes the next one to run again. Clear every output, which the `reset()` method does in one call, and the line restarts from the first station with the same job cards.

9.4 Read the Assembly

One stage's call uses only Part I parts. The system prompt is the workspace `CONTEXT.md` if you wrote one, or a careful default about executing one stage exactly. The prompt is the stage contract first, then the reference rules under their heading if any exist, then the previous stage's output under an input heading, joined into one string. That string goes to `get_response`, and the reply is written to disk verbatim.

Two consequences follow. Stage boundaries are context boundaries: the drafting stage sees the outline file and its own card, not the research transcript, not the road not taken, not your reasoning at the review bench unless you wrote it into the file. Whatever must survive a boundary must be written in the workpiece, which is why good stage contracts ask outputs to carry their evidence with them. And a failed stage leaves its failure in the out-tray, literally: as Chapter 1 established, a connection error comes back as a string, so `output.md` can contain an error message, non-empty and therefore

"done" by the runner's rule, quietly feeding the next stage. On this design, the review pause is the error handling.

The runner also offers `run_all`, which fires the remaining stages in sequence with no pause, bounded by a generous step limit. The package documentation is blunt that it is for pipelines you already trust. Treat your first run of any new line as untrusted by definition. Use `run_all` only after the cards have survived a few inspected runs. Even then, the saved files mean a bad run can be rewound stage by stage rather than rerun from a blank prompt.

> **Contracts are the product**
>
> When a pipeline disappoints, the fix almost never lives in the runner, which only assembles and files. It lives in a job card: the research card did not say what a finding looks like, the outline card did not forbid new facts, the draft card never named its reader. Iterate on `CONTEXT.md` files the way Chapter 7 taught you to iterate on tool descriptions, by feeding the stage exactly what its card requests and reading what the card, as written, actually asked for.

9.5 Where a Demo Can Mislead You

"Fully autonomous multi-agent workflow" demos run the whole line at speed and show the finished piece. Can I see the workpiece between stages, as a file or its equivalent, and edit it before the next stage consumes it? If stage two of five goes wrong, do we rerun one stage or the universe? Where do the standing rules live, and can a person change them without a deployment? When a stage fails, does the failure stop the line in a visible tray, or flow downstream dressed as content? A framework that answers those well has earned its complexity over numbered folders. One that cannot show you the stages gives you nothing to inspect.

9.6 Practice Exercises

Exercise 9.1 (Level 1). Build the three-stage workspace, write the cards for a topic you know well, and run the research stage only. Read `output.md` and mark every claim you could not verify from your own knowledge.

Exercise 9.2 (Level 2). Edit the research output by hand: correct one claim, delete the weakest finding, add one finding of your own. Run the outline stage and trace which of your edits visibly shaped it.

Exercise 9.3 (Level 3). Add a `references/style.md` with three rules for the draft stage, reset the workspace, and rerun. Compare the two drafts. Which rule was obeyed most literally, and which needed rewording?

Exercise 9.4 (Level 4). Sabotage the line: empty the middle stage's output file without deleting it, then call `run_next`. Then write an error sentence into it by hand and call `run_next` again. Record how the runner's definition of done handled each case, and what your review habit must therefore include.

Exercise 9.5 (Level 5). Design a four-stage pipeline for a real recurring job: meeting notes to decisions, lesson material to quiz, incident log to board brief. Write all four cards without running anything. For each boundary, state what the output must carry so the next stage needs nothing upstream.

9.7 Chapter Summary

- A workflow workspace is numbered stage folders, each with a `CONTEXT.md` contract and an `output/output.md`. A stage counts as done when its output file exists and is non-empty.
- Running a stage assembles one ordinary call: workspace instruction as system prompt, then contract, reference rules, and the previous stage's output as the prompt.
- `run_next` runs one stage and stops for review; your edits to an output become the next stage's input verbatim. `run_all`

removes the pauses and is for trusted lines only.
- Stage boundaries are context boundaries. Anything needed downstream must be written in the output file itself.
- An error string from a failed call is non-empty, so it counts as done. The review pause is the error handling.

9.8 Reflection

Before moving on, ensure you:

- ☐ Laid out a workspace and can walk its folders station by station
- ☐ Wrote stage contracts and ran at least two stages with a review between
- ☐ Edited an output and traced the edit into the next stage's result
- ☐ Can state the runner's definition of done and its one dangerous consequence

9.9 Your Learning Journal

1. Which job card took the most rewriting, and what did its first version fail to ask for?
2. What did you write into an output file purely so the next stage would have it?
3. Describe a workflow you already do by hand in stages. Where are its inspection benches?

9.10 Next Steps

The pipeline repeats across stages, with you as the metronome. The next chapter repeats within one stage: the same improvement step applied again and again to its own output, with the stopping decision handed to a goal you state in advance.

Chapter 10

Loops with a Goal

> Do, check, repeat is a fine engine and a terrible plan, until the check is a goal you can state and the repetitions have a budget.

Package companion: `hands_on_ai.loop`, with `run_loop`, `run_ratchet`, and `judged`. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "More steps means better." Repetition needs a stopping condition, and progress needs a memory of the best so far.

The build: a rough paragraph improved by a loop until a stated standard is met, twice: once keeping whatever the last step produced, once keeping only improvements.

Python you need: passing a function as an argument, first met in Chapter 7, and the one-line functions written with `lambda`, explained below. The rest is Appendix A.

10.1 The Concept First

Chapter 8's agent loop repeated until the model declared an answer. This chapter's loop is the same shape with the roles clarified and handed to you. A `step` function does one piece of work to the

current state. A `goal` function inspects the new state and returns `True` when it is good enough. The runner alternates them, keeping a history, until the goal fires or a budget of iterations runs out. Do, check, repeat, with the check written down before the doing starts.

Without a stated goal, a loop around a model will happily polish for as many iterations as you fund, including the iterations where the text gets longer, greasier, and worse, because nothing in "improve this" contains a stopping point or a direction of improvement that two readers would agree on. With a goal, two honest outcomes become possible: the loop can report that it met the standard in this many iterations, or that it spent its budget without meeting it. A loop that always returns its last attempt gives you neither report.

The package names the two parts for you. The trigger is the starting state plus the step that fires every turn. The goal is the stopping check, and it can be anything callable that answers yes or no about a state: a word count under a limit, a required phrase present, a test suite passing in your own code, or, from the next chapter's machinery, a model judging the output against criteria and scoring it high enough.

10.2 Understanding Through Real Life

Think of kneading dough to the windowpane test. The step is kneading for a minute. The goal is not "knead until it seems right". It is a stated check: stretch a small piece until light passes through it without tearing. A baker checks after each round, stops the moment the test passes, and knows that kneading past the test does damage rather than adding quality. There is also a budget in practice: if the test has not passed after a reasonable effort, something upstream is wrong with the flour or the water, and more kneading will not rescue it. The honest report is "test not passed in the time", followed by a different decision.

A goal-checked loop is that bakery discipline for text. The windowpane test must exist before the kneading starts, must be cheap enough to run every round, and must be capable of failing.

10.3 The Build, First Without a Model

The runner does not care whether a step calls a model, which makes it possible to learn the shape with plain Python and exact output. This loop builds a sentence one word at a time until it is long enough:

```
# Listing 10.1 of 3: the loop shape, no model
(loop_shape.py)
from hands_on_ai.loop import run_loop

def add_word(state):
    return (state + " word").strip()

result = run_loop(
    step=add_word,
    goal=lambda text: len(text.split()) >= 4,
    start="start",
    max_iters=10,
)

print(result["result"])
print(result["iterations"], result["met_goal"])
print(result["history"])
```

```
start word word word
3 True
['start word', 'start word word', 'start word word
word']
```

Read the return value as a small report, because it is the same report your model versions will give you. `result` is the final state. `iterations` counts the steps that actually ran. `met_goal` tells you which exit fired: the goal's `True`, or the budget running out first. `history` holds every intermediate state in order, the loop's equivalent of Chapter 8's trace, and it is where you will later watch quality rise, stall, or decay. The `lambda` in the listing is Python's way of writing a tiny unnamed function in one line: `lambda text: ...` defines a function taking `text` and returning the value after the

colon. Chapter 7 handed named functions to a registry; loops take one-line functions so often that the shorthand earns its place here.

Set `max_iters=2` and run it again in your head: the same step, the same goal, and the report changes to three words, two iterations, `met_goal` false. Nothing failed; the loop tells the truth about a budget too small for its goal, and hands the decision back to you.

10.4 The Build, With a Model and a Stated Standard

Now the step becomes a model call and the goal becomes a judgement. The package's `judged` helper wraps next chapter's judge into a goal function: give it criteria in words and a threshold score, and it returns the `goal(state)` check `run_loop` wants. Reading Chapter 11 first is allowed and even sensible; the wiring makes sense on contact either way.

```
# Listing 10.2 of 3: improve until good enough
(improve_paragraph.py)
from hands_on_ai.chat import get_response
from hands_on_ai.loop import judged, run_loop

result = run_loop(
    step=lambda draft: get_response(f"Improve this
    paragraph:\n{draft}"),
    goal=judged("clear, concise, and friendly",
    threshold=4),
    start="loops are when you do stuff again and
    again",
    max_iters=5,
)

print(result["result"])
print("in", result["iterations"], "iterations;
goal met:", result["met_goal"])
for draft in result["history"]:
```

```
    print("-", draft)
```

Count the model calls before you run it, because Chapter 3's arithmetic has a new multiplier. Each iteration costs one improvement call from the step and one judging call hidden inside the goal, so a budget of five iterations is up to ten calls, each carrying its own prompt. On a local model that costs patience; on a metered provider it is a line item, and `max_iters` is where you signed for it.

Read the printed history with a marker in hand. A draft may reach the threshold by becoming shorter in a way that loses the point, because "concise" was in your criteria and "keeps the original meaning" was not. A later iteration may be worse than an earlier one, and here `run_loop` has a character trait you must know: it keeps the last state, whatever the history shows. The goal check happens after the step, so a run that decays through its final iteration returns the decayed text with `met_goal` honestly false, and the best draft of the run sits in `history` for you to rescue by hand. That design is simple and sometimes wrong, which is why the package ships a second shape.

10.5 The Ratchet: Keep Only Wins

`run_ratchet` changes what the loop remembers. Each iteration, the step proposes one change to the current best. A `score` function rates the proposal, and the proposal replaces the best only if it scores strictly higher. Otherwise it is discarded, the way a mechanical ratchet refuses to slip backward, and the loop logs every proposal's score and whether it was kept. The package nicknames it the Ralph Wiggum technique, and the loop it describes is a classroom-sized relative of how optimisation harnesses grind on a problem overnight.

```
# Listing 10.3 of 3: only improvements survive
(ratchet_paragraph.py)
from hands_on_ai.chat import get_response
from hands_on_ai.eval import judge
from hands_on_ai.loop import run_ratchet
```

```
CRITERIA = "clear, concise, friendly, and keeps
the original meaning"

result = run_ratchet(
    step=lambda draft: get_response(
        f"Improve this paragraph in one small
        way:\n{draft}"
    ),
    score=lambda draft: judge(draft,
    CRITERIA)["score"],
    start="loops are when you do stuff again and
    again",
    max_iters=5,
)

print(result["best"])
print("final score:", result["score"])
for row in result["log"]:
    print(row)
```

Each row of the log is one proposal: its iteration number, its score, and whether it was kept. A healthy log shows a few keeps early and discards later as the text firms up. A log of all discards says your starting text already beat everything the step proposed, or that the score cannot tell your proposals apart. A log of all keeps with climbing scores on a vague criterion should make you suspicious in the other direction: the step may be learning to please the scorer rather than to serve the reader. Note two bookkeeping facts for your budget: the ratchet scores the starting state once to set the baseline, and a proposal whose score comes back `None`, which happens when a judge's reply cannot be parsed, is never kept. Inaction on unreadable evidence is the right default.

The two shapes differ in what a mediocre run hands back. `run_loop` pursues a stated standard and returns wherever it stopped, with the history as evidence. `run_ratchet` guarantees the returned text is at least as good, by your score, as where you started, and pays for

the guarantee with a scoring call per proposal and a step that must be kept to small changes. For a paragraph, the ratchet's guarantee is worth it. For reaching a format or passing a test, the plain loop reports failure faster.

> ⚠ A loop is an amplifier
>
> A step and a goal that are each slightly wrong do not cancel out in a loop. They compound. A step that pads text and a goal that rewards thorough-sounding criteria will march a paragraph away from its reader a few confident iterations at a time, logging progress at every step. Read histories and logs, especially early on, the way Chapter 9 taught you to read stage outputs. The budget you set is also a statement about how far you trust the amplifier unsupervised.

10.6 Where a Demo Can Mislead You

"It refines itself until it is excellent" compresses every question in this chapter into a slogan. What is the goal, stated so a third party could check it, and who wrote the criteria? What is the budget in iterations and in model calls, and what does the system return when the budget expires without the goal firing? Does it keep the last attempt or the best? May I see a history or a log from a run that failed to meet its goal, since anyone can show the run that converged?

10.7 Practice Exercises

Exercise 10.1 (Level 1). Run `loop_shape.py`, then change only the goal to demand six words and only the budget to three iterations, one at a time. Predict the full report dictionary before each run.

Exercise 10.2 (Level 2). Run the paragraph improver on a rough paragraph of your own writing. Read the history aloud. Mark the first iteration you would have stopped at, and compare it with where the loop stopped.

Exercise 10.3 (Level 3). Add "keeps the original meaning" to the criteria in Listing 10.2, then remove it, running on the same start text both times. Where does the criterion change the outcome, and where does the loop simply not notice?

Exercise 10.4 (Level 4). Run the ratchet with a deliberately crude step that rewrites the whole paragraph each time instead of one small change. Read the log. Why does the ratchet's logic prefer small proposals, and what did whole rewrites do to the keep rate?

Exercise 10.5 (Level 5). Design a loop for a real task (quiz questions until they meet a rubric, summaries until they fit a limit with facts intact). Spec sheet only: start state, one-paragraph step prompt, goal criteria written to be checkable, threshold, budget in iterations and estimated calls, and the exact sentence the loop should hand back when the goal is not met.

10.8 Chapter Summary

- A goal loop alternates a `step(state)` doing one piece of work with a `goal(state)` check stated in advance, bounded by `max_iters`.
- `run_loop` returns the final state, the iteration count, whether the goal was met, and the full history. It keeps the last state even when the goal never fires.
- `judged(criteria, threshold)` turns the model-as-judge into a goal. Each judged iteration costs one step call plus one judging call.
- `run_ratchet` keeps a proposal only when it scores above the best so far, logs every proposal with its score and keep decision, and never keeps an unscorable proposal.
- Loops amplify the step and the goal together. Histories and ratchet logs are how you watch direction as well as arrival.

10.9 Reflection

Before moving on, ensure you:

- ☐ Ran a plain-Python loop and can explain all four keys of its report
- ☐ Saw `met_goal` come back false and read the history that explains why
- ☐ Ran a ratchet and accounted for every keep and discard in its log
- ☐ Can state, for a loop of your own design, its budget in model calls rather than iterations

10.10 Your Learning Journal

1. At which iteration would you have stopped your paragraph run, and what does the gap to the loop's choice say about your criteria?
2. Describe a check in your own work that functions as a windowpane test: cheap, repeatable, capable of failing.
3. When have you kept the last version of something when an earlier one in the history was better?

10.11 Next Steps

Both shapes in this chapter leaned on a scorer that has so far operated offstage: a model asked to grade text against your criteria and return a number. The next chapter puts that judge on the bench, takes it apart into the call and prompt it is, and calibrates how far its numbers deserve your trust.

Chapter 11

The Judge

> When output is cheap and plentiful, the scarce skill is telling good from fluent, at a scale your eyes cannot cover alone.

Package companion: `hands_on_ai.eval`, the `judge` function behind Chapter 10's goals. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "I can eyeball quality." At scale you cannot, the demo is not a test, and a score is a signal to calibrate rather than a verdict to obey.

The build: `judge-your-bot`, the package's evaluation project. Score a set of chatbot answers against stated criteria, then compare the judge's marks with your own and read the disagreements closely.

Python you need: dictionaries and loops from Appendix A.

11.1 The Concept First

Every chapter since Chapter 5 has widened the gap this one addresses. Production is now the cheap direction: a call produces an answer, a pipeline produces a draft, a loop produces five drafts and a log. Evaluation is where the cost now sits: somebody, or something,

has to read all of that output and separate the sound from the merely fluent, and a human reader does not scale past a few dozen items a day with their attention intact. The industry's working answer is the pattern this chapter takes apart: LLM-as-judge, in which a model is asked to score an output against criteria you state, returning a number and a reason your code can act on, as Chapter 10's goals already did.

The judge is the smallest build in the book. It is one call, with a system prompt casting the model as a strict but fair evaluator and demanding exactly two lines back, a score and a reasoning, followed by a small parser that extracts the number and clamps it into the scale. There is no special evaluation engine, which is why calibration matters more than the mechanism: a judge is a model producing text about text, with a model's failure modes, and the craft is in knowing how far its numbers track your own judgement on work you have actually read.

Judging is becoming a baseline skill of working with AI in the way estimating a search result's reliability became one a generation ago, because output volume has already passed the point where unaided reading is the control. At scale, the alternative to a calibrated judge is no review, with a demo's confidence in its place.

11.2 Understanding Through Real Life

Think of examination marking at scale. One chief examiner cannot read ten thousand scripts, so a team of markers works from a shared rubric: criteria bands, exemplar answers at each level, and moderation meetings where markers compare scores on the same scripts and argue until their standards converge. Examination boards do not treat a new marker's first hundred marks as ground truth; those marks are sampled, compared, and adjusted. Trust in a marker is earned per rubric, per season, and it expires when the rubric changes.

An LLM judge joins your marking team as an indefatigable new marker who reads at inhuman speed and never gets tired at script

nine thousand. It also arrives with no stake in being right, a susceptibility to confident presentation it inherited from training, and standards that live entirely in the rubric text you handed it. The moderation meeting is the process by which this particular marker, on this particular rubric, earns the right to mark unsampled work. This chapter's build is that meeting, held at classroom scale with your own marks as the comparison.

11.3 The Build: Put the Judge on the Bench

Start with a single verdict, printed in full, so every part of the return value is visible:

```
# Listing 11.1 of 2: one verdict, all its parts
(one_verdict.py)
from hands_on_ai.eval import judge

verdict = judge(
    "TCP retransmits lost packets so data arrives
    complete and in order.",
    criteria="accurate, concise, and answers the
    question asked",
    question="What does TCP do?",
)

print(verdict["score"])
print(verdict["reasoning"])
print(repr(verdict["raw"]))
```

The dictionary has three entries. `score` is the parsed number, an integer on the scale you chose (1 to 5 unless you pass `scale=`), or `None` when the reply contained no readable score. `reasoning` is the one-sentence justification the prompt demanded. `raw` is the judge's untouched reply, kept so that when a score surprises you, the text it was parsed from is one print away. Keep `raw` in anything you build on this.

Run the listing a few times on the same answer, and treat the wobble

as data. Scores may hold steady or drift by a point between runs, because the judge is a model call, subject to everything Part I taught you about model calls. Drift does not make the instrument useless, any more than marker variance abolishes examinations. It tells you what the numbers can and cannot carry: separating a 2 from a 5 with confidence, perhaps, while a ranking built on 4-versus-5 distinctions is reading tea leaves unless your calibration says otherwise.

11.4 The Moderation Meeting: Judge Your Bot

Now the project the package names `judge-your-bot`. Take a small set of answers from a bot you have built, a personality bot from Chapter 2 or your RAG bot from Chapter 6 answering questions you chose, and mark them twice: once yourself, once with the judge, against the same written criteria.

```
# Listing 11.2 of 2: your marks beside the judge's
(judge_your_bot.py)
from hands_on_ai.eval import judge

CRITERIA = "accurate, based only on the provided
documents, and concise"

cases = [
    ("What does TCP do?", "TCP retransmits lost
    packets so data arrives complete and in
    order.", 5),
    ("What does TCP do?", "TCP is a very important
    protocol that does many things on the
    internet.", 2),
    ("How does HTTP work?", "The rules do not
    say.", 4),
]

for question, answer, my_score in cases:
    verdict = judge(answer, CRITERIA,
    question=question)
```

```
    print(f"mine={my_score}
    judge={verdict['score']} ::
    {verdict['reasoning']}")
```

Write your scores into the listing before you run it, from your own reading, as the tuples above suggest. Then read the output disagreement first, agreement second. Each mismatch is a small investigation with exactly three possible findings. Your criteria may be underwritten: the judge marked to the words you gave it, and the words did not say what you meant, which is Chapter 2's instruction lesson in a new uniform. Your own mark may be the sloppy one, given on a skim the judge's slower pass exposed. Or the judge may simply be wrong in a way worth characterising: rewarding length, rewarding confidence, missing a factual error it had no source to check against, punishing a correct refusal because it looks unhelpful. Sort your mismatches into those three piles and you have a calibration record: where this judge, on this rubric, with this model, can be trusted, and where it must be sampled or overruled. Refresh that record when the model or criteria change.

11.5 Writing Criteria a Judge Can Use

Rubric writing for a model follows the instruction rules from earlier chapters. Criteria should name checkable properties rather than vibes: "states the borrowing limit as a number" can be marked against a document; "high quality" cannot be marked against anything. Where correctness matters, give the judge something to be correct against: the question, the source chunk, the expected fact. A judge asked to score accuracy from its own training is a second opinion from the same well the first answer was drawn from, and Chapter 5 already told you what that well contains. Keep the scale honest about the discrimination you verified in moderation: if your judge reliably separates poor, adequate, and strong, a 1-to-3 scale collects the information without pretending to five points of it. And separate criteria that pull against each other into separate judgements when the trade-off matters, because a single score

for “accurate and entertaining” hides which one failed, as Chapter 2’s contradictory system prompts hid which instruction the model disobeyed.

The parser clamps and fails visibly, by design. A judge that replies with a score off the scale has it clamped to the nearest end. A judge whose reply contains no `SCORE:` line yields `None`, which Chapter 10’s goal treats as not good enough and the ratchet treats as never an improvement. Neither behaviour fixes a confused judge, but both keep one from silently passing work.

> ⚠ The judge can be captured by its input
>
> A judge reads the output it is marking, and that output is model-generated text, which can contain sentences addressed to the marker: flattery, instructions, a forged `SCORE: 5` line of its own. The parser takes the first score it finds in the judge’s reply, and a small model quoted its input often enough for this to matter in adversarial settings. In your classroom builds the risk is mostly accidental, but the bench rule from Chapter 7 applies with roles reversed: treat the marked text as untrusted input to the marking process, and be suspicious of perfect scores on work you have not sampled.

11.6 Where a Demo Can Mislead You

Ask the moderation questions of any evaluation claim, however scientific its percentages, benchmark names, and charts look. Who or what produced these scores, against criteria I may read? Where is the calibration sample comparing the judge’s marks with marks from someone accountable, and what did the disagreements look like? What happens to the score distribution when the judge model and the judged model swap? Can I see raw verdicts, or only the averaged number that survived contact with a slide template? These questions ask whether the vendor held the moderation meeting and can show its minutes.

11.7 Practice Exercises

Exercise 11.1 (Level 1). Run one verdict five times on the same answer and record the scores and reasonings. Report the range, and state what decisions you would and would not base on this judge at this stability.

Exercise 11.2 (Level 2). Build a case list of eight answers from your own earlier chapters, at least two of which are fluent and wrong. Mark them yourself, run the judge, and sort every disagreement into rubric, my mark, or judge error.

Exercise 11.3 (Level 3). Rewrite your criteria to repair the largest rubric-caused disagreement, rerun the same cases, and report what moved and what stubbornly did not.

Exercise 11.4 (Level 4). Probe the judge's soft spots with paired answers: same content at two lengths, same content in confident and hedged tones, a correct refusal against a fluent invention. Which dimension moved the score most, and was it one your criteria mentioned?

Exercise 11.5 (Level 5). Write the moderation protocol for a course team adopting this judge for feedback drafts: sampling rate, who marks the sample, what disagreement rate suspends the judge, and when recalibration is mandatory. One page, ending with the decision this judge is never allowed to make alone.

11.8 Chapter Summary

- The judge is one model call with a strict evaluator prompt and a parser. It returns `score` (clamped to the scale, or `None` when unparsable), `reasoning`, and the untouched `raw` reply.
- Scores wobble between runs and track your rubric's wording as much as the work's quality. Calibration against your own marks on sampled work is what converts scores into a usable signal.
- Disagreements sort into rubric gaps, your own marking errors, and judge errors such as rewarding length or confidence. Each

pile has a different fix.
- Checkable criteria, evidence supplied to the judge, a scale sized to verified discrimination, and separate scores for competing virtues all raise what the numbers can carry.
- Marked text is untrusted input to the marking process. Keep raw verdicts, sample continuously, and treat averages without minutes as marketing.

11.9 Reflection

Before moving on, ensure you:

- ☐ Collected repeated verdicts on one answer and measured the wobble
- ☐ Marked a case set yourself before seeing the judge's scores
- ☐ Sorted your disagreements into the three piles and repaired one rubric gap
- ☐ Can state one decision this judge may inform and one it must never make alone

11.10 Your Learning Journal

1. Which disagreement most changed your mind: one where the judge caught you, or one where you caught it?
2. What did your criteria say that you did not mean, once a literal reader marked to them?
3. Where in your own work is quality currently judged by whoever had time to glance at it?

11.11 Next Steps

Part IV is complete: work staged in folders with review gates, repetition bounded by stated goals, and quality marked by a calibrated judge. Everything so far has moved text in and text out. Part V gives the same call two more senses, pictures and voice, and finds

out how much of the machinery you already own by changing what rides in the payload.

Chapter 12

Typed Answers

> Most programs do not need paragraphs from a model. They need one option chosen, one yes or no, one score, one record, in a shape code can trust.

Package companion: `hands_on_ai.decide` (`classify`, `decide`, `score_output`, `extract`). Listings checked against the `decide` module on `hands-on-ai` main, after 0.6.0; install from the repository until it ships in a release.

Misconception this chapter kills: "The model replies in prose, so my program must parse prose." You can ask for small typed judgments instead, and validate the shape before your code believes a word of it.

The build: a support-ticket router and a refund decider, run first on your local model, then on a hosted System One model if you have a key, with marks compared.

Python you need: dictionaries and one small class definition for extraction, both met in Appendix A. The class here is data-shaped only: field names and types, no behaviour.

12.1 The Concept First

Chapter 11's judge smuggled a new kind of call into the book. It wanted one integer and one sentence, and dug them out of the reply with a regular expression, hoping the model kept the format. Chapter 7's agent wanted the same kind of smallness: a tool name and an input string, not a meditation. A large share of what programs actually need from models has that shape. Which queue does this ticket belong to? Does this message ask for a refund? How frustrated is this customer, on a stated scale? What are the name, date, and total on this invoice? Questions with small, known answer spaces, asked thousands of times, where a paragraph of prose becomes a parsing problem.

TypeSafe gave a name and a service to this end of the trade: System One models, after Kahneman's fast, intuitive System 1, which return typed decisions and probabilities through three primitives. Choice picks one option from a set you define. Noul answers a yes/no question as a probability. Score rates something along ordered levels you describe. Their model, Jev, returns calibrated judgments your code can compose, without chat or explanations. You do not need their service to learn the pattern, because an ordinary chat model constrained to a schema can play the same role in miniature, which is what the package's `decide` module does. What the hosted model adds is calibration: probabilities trained to mean what they say across many predictions.

Schema constraining guarantees the *shape* of the answer. It says nothing about whether the answer, or its self-reported confidence, is right. A local model asked for a confidence will produce a number in range with the same fluency it produces everything else. Chapter 11's moderation habits therefore apply to every number in this chapter: calibrate on cases you marked yourself, or treat the number as decoration.

12.2 Understanding Through Real Life

Think of the difference between asking a colleague to "look into this customer email" and handing them a triage slip. The slip has boxes: department (tick one of three), refund requested (yes or no), urgency (circle 1 to 5), account number (write it here). The slip is more useful to everything downstream than the conversation, because the routing rule, the queue, and the report at the end of the week all consume boxes. Nobody asks the slip to explain the customer's feelings at length; there are other instruments for that, slower ones, reached for when the boxes say this case deserves them.

Fast typed judgments handle the bulk and the routine. Measured uncertainty routes the remainder to a person or to a slower reasoning model, the System 2 end of the shop. Most of this book has been about the slow end. Being fast takes narrow questions, defined answer spaces, and evidence that the fast answers deserve the trust your thresholds place in them.

12.3 The Build: a Router and a Decider

The module's four functions share one interface habit with the rest of the package: they return plain dictionaries and never raise for a model-side failure, putting `None` in the answer fields with an explanation under `error`. Start with Choice, wearing its package name, `classify`:

```
# Listing 12.1 of 3: route the ticket
(route_ticket.py)
from hands_on_ai.decide import classify, decide

ticket = "Help! My payouts have been failing for 3
days and I want my money back."

routed = classify(
    ticket,
    options=["billing", "technical", "sales"],
```

```
    instructions="Which team should handle this
    ticket?",
)
print(routed["choice"], routed["confidence"],
routed["error"])

refund = decide(ticket, "Does this message request
a refund?")
print(refund["answer"], refund["probability"],
refund["error"])
```

Run it on tickets of your own invention, including awkward edge cases: a technical fault that mentions money, a billing question caused by a bug, a message that fits no box at all. The option list is a closed world: the function validates the reply against it and hands back `None` rather than an invented department, so an "other" option is yours to include or to live without. "Which team should handle this?" plus three bare labels asks the model to guess your org chart; adding one line of rubric per option, the way TypeSafe's own examples pair each choice with a description, turns guessing into marking. Chapter 2's instruction craft and Chapter 11's rubric craft are the same craft here, applied to a smaller answer.

The extraction primitive has no System One counterpart, because Jev judges rather than transcribes, but it belongs in the same drawer: a typed record out of messy text, validated field by field:

```
# Listing 12.2 of 3: a record out of prose
(extract_invoice.py)
from pydantic import BaseModel

from hands_on_ai.decide import extract


class Invoice(BaseModel):
    vendor: str
    total_dollars: float
    due_date: str
```

```
result = extract(
    "Invoice from Sunrise Catering. Total due:
    $1,240.50. "
    "Please pay by 14 November 2026. Thank you for
    your business.",
    Invoice,
)
print(result["data"])
print(result["error"])
```

The `Invoice` class is a schema in Python syntax, with three field names and three types. Underneath, the local backend hands that schema to the model through Instructor's JSON-schema mode, the same machinery Chapter 7's JSON agent format used to validate tool calls before anything ran. What comes back has been checked against the schema by construction, so `total_dollars` is a number your code can add up, not the string `"$1,240.50"` waiting to break a sum. Fields the text cannot support are the interesting failure: try an invoice with no due date and read what the model does with its instruction to leave a field empty rather than invent one. Then decide, as a designer, whether empty, invented, or refused is the failure your system can afford, because the schema guarantees shape only.

12.4 The Same Judgments, Calibrated

Swap the backend and the interface holds while the meaning of the numbers changes. With a TypeSafe key in `TYPESAFE_API_KEY`, the same two calls go to Jev:

```
# Listing 12.3 of 3: local beside Jev
(compare_backends.py)
from hands_on_ai.decide import classify, decide

ticket = "Help! My payouts have been failing for 3
```

```
days and I want my money back."
options = ["billing", "technical", "sales"]

for backend in ("local", "jev"):
    routed = classify(ticket, options,
    backend=backend)
    refund = decide(ticket, "Does this message
    request a refund?", backend=backend)
    print(backend, "::", routed["choice"],
    routed["confidence"],
          "| refund:", refund["answer"],
          refund["probability"],
          "| error:", routed["error"] or
          refund["error"])
```

Without a key, the Jev row prints its error in plain sight and the local row carries the run, which is the correct classroom default. With one, compare more than the answers. Jev's Choice returns a probability for every option, summing to one, and its Noul probability is the product being sold: trained so that across many predictions, cases scored near 0.8 come out yes about eighty percent of the time. Your local model's confidence is a number it generated because the schema had a slot for one. On a single ticket both may look identical. Across fifty tickets you marked yourself, in Chapter 11's moderation style, the difference becomes measurable: bin each backend's probabilities and check the bins against reality.

Cost belongs in the same comparison, because these primitives exist to run at volume. A routed ticket locally is one small call on hardware you own. Jev is priced per input token at rates that make a million triage calls a rounding error next to one misrouted enterprise ticket, and it evaluates many questions against one state in a single request. Neither fact decides the architecture on its own; together with your calibration table, they might.

> **The judge moved house**
>
> Chapter 11's `judge` now tries this chapter's typed scorer first and falls back to its old two-line text format only when the structured call cannot produce a score. Its return shape, `{score, reasoning, raw}`, did not change, and Chapter 10's loops never noticed. Copy that pattern in your own systems: typed path preferred, text path as the documented fallback, the interface above both untouched.

12.5 When Fast Is the Wrong Instrument

For every question in this chapter there is a neighbouring question the primitives serve badly. Typed judgments do not explain, negotiate, draft, or reason through novelty; a Choice among three departments cannot tell you the ticket describes a fourth problem your org chart lacks. They do not fetch evidence; a refund decision made without the transactions and the policy in the state is a bare question from Chapter 5 with a probability attached. And their speed creates a specific organisational temptation, to let thresholds quietly become policy: auto-refund below this probability, auto-reject above that score. Thresholds are decisions about who bears the cost of the fast lane's errors. They deserve the same daylight as any other policy, set from your calibration data and revisited when the model or the mix of cases changes.

Used inside their fence, though, typed answers change what the rest of the book's machinery costs. A router decides which retrieval index a question deserves before Chapter 6 spends its tokens. An extractor turns incoming prose into fields Chapter 9's pipeline can file without a model rereading it at every stage. A decider gates Chapter 10's loop with a cheap check before an expensive judge wakes.

12.6 Practice Exercises

Exercise 12.1 (Level 1). Route ten tickets you invented, including two that fit no option. Add an "other" option and rerun. Record what changed beyond the two.

Exercise 12.2 (Level 2). Extract records from five messy texts of one kind (invoices, event announcements, job ads). Deliberately omit a field from two of them and catalogue what the model does with the gap.

Exercise 12.3 (Level 3). Build a calibration set: thirty short cases for one `decide` question, marked by you in advance. Bin the local probabilities and compare each bin with reality. If you have a Jev key, run the same set and place the tables side by side.

Exercise 12.4 (Level 4). Compose a cascade: a typed triage that routes easy cases directly and escalates low-confidence ones to Chapter 11's judge or to a human queue file. State your thresholds and the cost of each kind of error at each threshold.

Exercise 12.5 (Level 5). Write the threshold policy for a real routing decision in your world: the primitives, the state each needs as evidence, the calibration set that justifies the cutoffs, and the review rhythm when models change. One page, ending with the case type that must never be auto-decided.

12.7 Chapter Summary

- Typed primitives give programs small answers in validated shapes: `classify` picks one option, `decide` returns a yes/no probability, `score_output` rates against criteria, `extract` pulls a typed record from prose.
- The local backend constrains an ordinary model to a Pydantic schema: shape guaranteed, confidence self-reported and uncalibrated. Jev's hosted System One backend sells calibrated probabilities behind the same interface.
- Failures arrive as `None` fields with an `error`, never as exceptions or invented options; closed option lists make "other" a

design decision.
- Calibration is a table you build from your own marked cases, per backend, per question. Without it, confidence is decoration and thresholds are policy by accident.
- Typed answers compose: routers before retrieval, extractors before pipelines, cheap deciders before expensive judges, escalation where uncertainty concentrates.

12.8 Reflection

Before moving on, ensure you:

- ☐ Routed tickets including one that fit no box, and decided your "other" policy
- ☐ Extracted a record and probed a missing field deliberately
- ☐ Can state, with at least a small table behind you, how much your local confidence numbers deserve
- ☐ Know which of your calls would change meaning as well as shape, on the Jev backend

12.9 Your Learning Journal

1. Which question in your own work is secretly a triage slip waiting to be designed?
2. Where did a confident local number most mislead you, and what bin did it belong in?
3. What threshold in a system you use do you now suspect was never calibrated by anyone?

12.10 Next Steps

Part IV ends with the fast lane built and fenced. Part V returns to the slower senses, pictures and voice, where the payload grows a new kind of evidence and the clock starts charging rent. Readers headed for the optional Part VI may also proceed there directly; its notebooks assume Parts I and IV, not Part V.

Part V

The Senses

Chapter 13

Pictures

> Seeing, in this machinery, is a prompt with a picture attached. The reply is still text, and it can still be wrong in a confident voice.

Package companion: `hands_on_ai.vision`, its one function `describe`. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "Vision is a different kind of AI." Images go into the same kind of call, and descriptions come out.

The build: `talking-picture`, the package's vision project. A photograph that answers questions about itself, asked three different ways.

Python you need: one function call and a file path. The interesting work in this chapter is choosing photographs that test the model honestly.

13.1 The Concept First

A vision-language model, VLM for short, accepts a payload whose user message carries two parts instead of one string: your text question, and an image. Somewhere inside the server, the image is converted into tokens, the same internal units the model uses for words, and the machinery from Chapter 1 proceeds as always. Text

and a picture go in, and text comes out. The module docstring puts it plainly: seeing is another way of writing a prompt. This module describes images. It does not create them; image generation is a different family of models and outside this book.

How the image travels explains the costs and failures to come. A web address can pass through as a link for the server to fetch. A local file, the normal case for your own photographs, is read by your code and encoded as base64, a text representation of the file's bytes, inlined into the payload as a data URL. Your photograph becomes a long string inside the JSON from Chapter 1: a phone photograph inflates into hundreds of thousands of characters of payload before the server has done any thinking, which is one reason vision calls are slower and the package allows them a generous timeout.

13.2 Understanding Through Real Life

Think of describing a scene over the phone to someone who has never seen your street. The description is a translation, and like every translation it is shaped by the question asked. "What is happening in this photo?" earns a story about the scene. "How many people are wearing hats?" earns a count the describer may or may not have performed carefully. "What does the sign say?" earns a reading, accurate where the sign is legible and confabulated where it is not, unless the describer is the rare sort who says the text is too small to read.

A VLM is that describer with two exaggerations. It has seen vastly more photographs in training than any person, and its scene sense is correspondingly strong. And it shares the language model's reluctance to say it cannot tell: fine print, exact counts, small text, spatial relations like left-of versus behind, and anything the downscaled encoding smeared away may all come back rendered in fluent certainty. Chapter 5's rule applies unchanged, with pixels added: separate what the image could show from what this call could recover, and check the parts that matter against the picture yourself.

13.3 Connect a Vision Model Before You Build

Vision needs a model that accepts images, configured separately from your chat model, the way Chapter 6 added an embedding model. The default is `moondream`, a small vision model, changeable through the `vision_model` setting or `HANDS_ON_AI_VISION_MODEL`:

```
ollama pull moondream
handsonai doctor
```

The doctor reports the vision model alongside the others. If you point the package at a hosted provider instead, use one of its vision-capable models under the same setting; Appendix B gathers the details. Expect the first call to be slow while the model loads, and expect quality to track model size more than chat did. A small local VLM teaches well and proofreads poorly.

13.4 The Build: a Photograph That Answers

The module has one function, and its defaults need no help. Save a photograph of your own beside the script, something with a scene in it rather than a screenshot, and ask it three questions of widening difficulty:

```
# Listing 13.1 of 2: the talking picture
(talking_picture.py)
from hands_on_ai.vision import describe

photo = "market.jpg"

print(describe(photo))
print(describe(photo, "What is the main thing
being sold here?"))
print(describe(photo, "How many people are in this
picture? Count carefully."))
```

The first call uses the default prompt, “Describe this image.”, and gets you the tour. The second shows the usual pattern: questions put to a picture, such as reading a label, identifying a plant, checking

whether a gauge is in the red, or asking what is funny here. A picture plus a pointed question is the vision equivalent of Chapter 5's grounded prompt, with the image as the evidence in the room. The third call is the honesty test. Counting is where small vision models go to be humbled, along with small text and precise spatial claims, and your own model may produce a crisp number you can refute by looking at the photograph.

Underneath, `describe` assembles the two-part user message and sends it through the same `chat_completion` machinery as every chat call in the book, with the longer timeout. One behaviour will look familiar from Chapter 1 by design: point it at a file that does not exist and you get an error string back, `Error: Image file not found: ...`, rather than a crash. Your program can tell failure from description the way it learned to in Part I.

13.5 What It Can and Cannot See

A useful first hour with a VLM sorts its answers into three piles. Scene gist is strong: what kind of place this is, what is broadly happening, the salient objects and their rough relations. Reading is conditional: signage and labels come back well when they are large, level, and high-contrast, and degrade into plausible invention as they shrink, tilt, or blur. Precision work is weak on small models especially: exact counts, small differences between similar objects, charts read to values, and anything depending on detail the encoding downscaled away. Results vary with model size, and re-sorting the piles per model is the same moderation exercise Chapter 11 taught, with the photograph as ground truth sitting right there.

Vision output is text, so it can be judged: a description can be scored against criteria, and better, against a human transcript of the same image, by the machinery of Chapter 11. And a picture rarely answers the question behind the question alone. What the sign says comes from the photograph; whether the shop is open now comes from a document or a tool. Systems that pair a VLM's description with retrieval from Chapter 6 or a lookup tool from Chapter 7 combine this book's parts on one bench.

⚠ Photographs carry people

Your photographs will be uploaded, encoded, to whatever server your configuration names, and Chapter 3's transcript warning applies with faces in place of words. Local models keep the bytes on your machine; hosted ones are governed by the provider's retention terms. Photographs of other people deserve the consent habit you would want applied to your own likeness, and some workplaces and jurisdictions regulate biometric uses well beyond anything in this chapter. Description for learning is benign. Build the consent reflex anyway, while the stakes are homework.

13.6 Where a Demo Can Mislead You

Vision demos choose photographs the way Chapter 5's demonstrators chose questions: big subjects, clean light, legible signs, counts of three. The counter-demo takes a minute to assemble from your own phone. A crowd to count, a receipt to read to the cent, a dashboard at an angle, a shelf where one item differs from its neighbours. Run the product on your photographs and sort its answers into this chapter's three piles. Then ask the vendor's questions: at what resolution is my image downscaled before the model sees it, where do the bytes go and how long are they kept, and show me the runs where the count was wrong. The pile boundaries will move as models grow, so draw them yourself, per model, on your material.

13.7 Practice Exercises

Exercise 13.1 (Level 1). Run the talking picture on a photograph of your own. Ask the tour question and two pointed ones, and grade each answer against the picture.

Exercise 13.2 (Level 2). Assemble a five-photograph test set spanning the three piles: a scene, a sign or label, a crowd, a chart or receipt, a near-duplicate pair. Record where your model's pile

boundaries actually fall.

Exercise 13.3 (Level 3). Ask a question the photograph cannot answer (when it was taken, what happened next, who these people are). Record whether the model declines, hedges, or improvises, and compare with Chapter 5's escape-sentence experiment.

Exercise 13.4 (Level 4). Pair the senses: describe a photograph of a document, save the description, and ask your Chapter 6 RAG bot a question needing both the picture's content and your documents. Where did the hand-off between them lose information?

Exercise 13.5 (Level 5). Write the acceptance test for a vision feature at your workplace: the photograph set, the questions, the accuracy bar per pile, the consent and retention rules. One page, ending with the pile you would refuse to ship at this model's scores.

13.8 Chapter Summary

- A vision call is a chat call whose user message carries text plus an image, inlined as base64 for local files. Descriptions come out; images are not created.
- `describe` needs a vision-capable model, configured separately (`moondream` by default), and runs slower with a generous timeout. Missing files return an error string.
- Sort your model's work into scene gist (strong), reading (conditional on size and clarity), and precision (weak, especially counting and small text), using your own photographs as truth.
- Vision output is text and composes with the rest of the book: it can be judged, grounded against documents, and checked like any other claim.
- Photographs travel to the configured server. Local models keep them home; people in pictures deserve a consent habit.

13.9 Reflection

Before moving on, ensure you:

- ☐ Asked one photograph three questions of widening difficulty
- ☐ Caught at least one fluent error by checking against the picture
- ☐ Can say where your model's three pile boundaries fall today
- ☐ Know which server your photographs were sent to, and approve of it

13.10 Your Learning Journal

1. Which answer most surprised you by being right, and which by being wrong?
2. Describe a picture from your work where the conditional pile, reading, is exactly what matters. What would you pair the model with?
3. What is your personal rule, after this chapter, for sending photographs of people to a hosted model?

13.11 Next Steps

Pictures arrived as an attachment to the call you already had. Voice arrives the same way, twice over: one call turns speech into the text your chat loop already eats, and another turns its reply back into speech. The next chapter stitches those three calls into a conversation you can hear, and meets the one problem text never had, which is time.

Chapter 14

Voice

> A voice agent is the chat loop with ears and a mouth bolted on: speech in, text through the machinery you know, speech out.

Package companion: `hands_on_ai.voice`, with `transcribe` and `speak`, around the `Conversation` from Chapter 3. Listings checked against `hands-on-ai` 0.6.0.

Misconception this chapter kills: "Voice AI is a different technology." It is transcribe, think, speak: three plain calls, and a fight against the clock.

The build: a talk-out-loud chatbot in two forms: typed input with spoken replies first, then a recorded question carried the whole way around the loop.

Python you need: function calls and the `Conversation` object. The new material is about servers and timing, not syntax.

14.1 The Concept First

The thinking in a voice system is the text machinery of Parts I to IV, unchanged, with two translators at the edges. A speech-to-text model, STT, takes an audio file and returns the words as text. That text enters the chat loop you built in Chapter 3 and receives a reply

in text. A text-to-speech model, TTS, takes the reply and returns an audio file. That gives the loop: transcribe, think, speak. Each translator is a model call to an OpenAI-compatible endpoint under `/v1/audio/`, made with the same client patterns as Chapter 1 and Chapter 6's embeddings. Voice adds senses the way pictures did: by converting to and from the text the system already runs on.

What changes is the constraint. Text chats tolerate a slow reply gracefully; the words appear when they appear. Spoken conversation runs on human turn-taking clock, where a pause of two seconds already feels like the other party has wandered off. Every hop in the chain spends from that budget: recording and uploading the audio, transcribing it, thinking over a growing transcript, synthesising the reply, downloading and playing it. Latency, the total time from the end of your sentence to the start of the reply, is the engineering problem of voice systems.

14.2 Understanding Through Real Life

Think of a conversation conducted through two interpreters in a soundproof booth relay. You speak your language. The first interpreter renders it into the booth's working language and passes the words across. Your counterpart, who thinks in the working language, composes a reply. The second interpreter renders the reply back into yours. Everyone in the chain can be excellent and the conversation can still feel drunk, because each hand-off adds delay and small translation choices: a name heard slightly wrong enters the working language as fact, and a long, clause-heavy reply that reads beautifully becomes punishing when someone has to speak it and someone else has to stand there listening.

Transcription errors propagate inward disguised as the user's own words, and the system answers the sentence it heard rather than the one you said. Reply style propagates outward: length, lists, tables, and URLs that serve a reader well become unusable when spoken, which is why voice system prompts beg for brevity.

14.3 Two More Servers Before You Build

Audio models usually do not live on the chat server. A plain Ollama install serves chat, embeddings, and vision, but speech typically comes from a separate local server that speaks the same `/v1/audio/` dialect: tools such as speaches or whisper.cpp serving Whisper models for transcription, and Kokoro or Piper for synthesis. The package therefore carries separate settings, `HANDS_ON_AI_STT_SERVER` and `HANDS_ON_AI_TTS_SERVER`, which default to your main server, and separate model settings defaulting to `whisper-1` for transcription and `kokoro` for speech. A hosted provider with audio endpoints collapses all of this back to one address and one key, at hosted prices and with your voice recordings travelling to it.

Before building anything, point those settings at servers you actually have, local or hosted, and confirm with the doctor habits from Chapter 4. Appendix B collects the variables in one table. Then test the two translators in isolation, in the spirit of Chapter 7's call-the-tool-yourself rule:

```
# Listing 14.1 of 2: the ears and the mouth,
separately (voice_pieces.py)
from hands_on_ai.voice import speak, transcribe

print(speak("The loop is transcribe, think, and
speak.", out_path="hello.wav"))
print(transcribe("hello.wav"))
```

`speak` returns the path of the WAV file it wrote, `speech.wav` in the current folder if you do not name one, or the familiar error string in place of the path. Empty text comes back as a warning string without troubling any server, Chapter 1 manners again. Feed the file it just made back through `transcribe` and you have run a small experiment: a sentence that went text to speech to text, with both translators at their best behaviour, on clean synthesised audio. Compare the returned words with the originals. Then transcribe a recording of yourself, made on your phone in a real room, and compare again. No chat model downstream can recover words the

transcription missed.

14.4 The Build: the Loop Wearing Headphones

The package's own interactive voice loop starts with a sensible cheat: you type what you would say, and only the reply is spoken. Microphone capture is platform plumbing, left as an extension exercise. In your own code it is Chapter 3's `Conversation` with one added line, plus a system prompt that knows its output will be heard rather than read:

```
# Listing 14.2 of 2: typed question, spoken answer
(voice_chat.py)
from hands_on_ai.chat import Conversation
from hands_on_ai.voice import speak

SYSTEM = (
    "You are a helpful assistant. Keep your
    answers short and "
    "conversational, one or two sentences, because
    they will be read aloud."
)

chat = Conversation(system=SYSTEM)

question = transcribe("my_question.wav") if False
else input("You: ")
reply = chat.ask(question)
print(reply)
print(speak(reply, out_path="reply.wav"))
```

Flip the `if False`, record your question as a WAV file by whatever tool your machine offers, and the same script runs the full round trip, ears included, with `Conversation` still holding the transcript between turns if you wrap it in a loop of your own. Playback of the reply file is deliberately your problem; the package's command-line

REPL makes a best effort with `afplay` on macOS and `aplay` on Linux and otherwise tells you to open the file. That seam, getting sound out of speakers and in from microphones across operating systems and browsers, is where small demos end and voice frameworks like Pipecat begin, and Appendix D points at the bridge project, the package's *Build the World's Smallest Pipecat*.

Time one full turn with a stopwatch: question finished, reply audible. Then ask where the seconds went. Transcription of a short clip, the chat call over a transcript that grows every turn (Chapter 3's tax, now audible), synthesis of the whole reply before the first sound plays. The industry's answers are all visible from here: stream the reply in pieces and start speaking the first sentence while the rest generates, keep spoken answers short by instruction, run smaller local models for the translators, and put the servers close. None of those change the shape; they shorten the relay.

> **i** Voice is biometric in a way text is not
>
> A voice recording identifies a person in ways a typed sentence does not, and speech synthesis can be pointed at imitation with enough samples. The consent reflex from Chapter 13 applies in full: record yourself freely, record others with their knowledge, know which server your audio travelled to, and treat cloning a real person's voice as a line that needs their explicit agreement rather than your technical ability. The tools in this chapter synthesise a stock voice. Keep the ethics at stock settings too.

14.5 Where a Demo Can Mislead You

Voice demos are staged in quiet rooms with close microphones, short questions, and presenters who have learned to pause where the demo needs them to. Your counter-demo is a kitchen with a kettle on, a name the transcriber will mangle, a question whose honest answer is a list, and an interruption mid-reply. What is the end-to-end latency at the ninety-fifth percentile, not the anecdote?

Where does my audio go, how long is it kept, and is it used for training? What happens when transcription hears me wrong: can I see the words the system thinks I said and correct them before action is taken? And which parts of this are your engineering, versus three API calls I assembled this morning from a learning package?

14.6 Practice Exercises

Exercise 14.1 (Level 1). Run `voice_pieces.py`, then transcribe a phone recording of the same sentence. Diff the two transcripts and list what the room cost you.

Exercise 14.2 (Level 2). Run the typed voice chat for four turns. Note the first reply that would have been fine on screen and was painful aloud, and adjust the system prompt because of it.

Exercise 14.3 (Level 3). Time a full round trip with a recorded question. Attribute the seconds across transcription, thinking, and synthesis as best you can, then shorten the turn by one deliberate change and re-time.

Exercise 14.4 (Level 4). Feed the loop its own transcription errors on purpose: dictate names, numbers, and a homophone-rich sentence. Where did the chat model recover the meaning anyway, and where did it earnestly answer the wrong sentence?

Exercise 14.5 (Level 5). Spec a voice front end for your Chapter 6 document bot: input and output paths, latency budget per hop, the spoken style rules, what the user sees when transcription is uncertain, and the consent and retention terms for recordings. One page, ending with the turn length at which you would switch the user back to text.

14.7 Chapter Summary

- A voice agent is transcribe, think, speak: an STT call turns audio into text, the existing chat machinery answers, and a TTS call turns the reply into an audio file. All three are

OpenAI-compatible calls.

- Audio usually needs its own servers and models, configured separately with defaults falling back to the main server. `speak` returns a file path or an error string, and empty text is refused locally.
- Latency is the voice problem: every hop spends from a turn-taking budget, and long replies hurt twice, once to generate and once to hear. Voice system prompts demand brevity for engineering reasons.
- Transcription errors enter disguised as the user's words, and the system answers the sentence it heard. Show the transcript where it matters.
- Voice recordings are identifying data. Consent, retention, and refusing imitation are part of the build, not the brochure.

14.8 Reflection

Before moving on, ensure you:

- ☐ Ran both translators alone, on clean and on real audio
- ☐ Heard a `Conversation` reply spoken from your own script
- ☐ Timed a full turn and can name its slowest hop
- ☐ Caught the system answering a sentence you did not say, or deliberately failed to

14.9 Your Learning Journal

1. Which transcription error was funniest, and which would have been most expensive in a real task?
2. Rewrite one of your earlier chapters' system prompts for the ear. What did you cut?
3. Whose voice will you never feed to a hosted service, and where did that line come from?

14.10 Next Steps

The senses turned out to be adapters: a picture in the payload, translators at the edges. Every part of the book has used the same shape. The conclusion names that shape once, plainly, gathers the questions each chapter taught you to ask of a demo, and points at the doors out of the learning lab.

Part VI

Inside the Box (Optional)

Chapter 15

The Older AI in the Room

> Before the chat box, and still underneath much of the world, sit models fitted to tables of numbers. They are fast, inspectable, and frequently the right tool.

Notebook companion: `classical_ml.ipynb` in the package repository's `notebooks/` folder. It needs only scikit-learn, already installed as a package dependency, and runs in seconds with no model server of any kind.

> **Part VI opens the fence here**
>
> Parts I to V needed no training and almost no maths. This optional part relaxes both, in notebooks, at toy scale. It begins with the classical machine learning that predates LLMs, because the training chapters after it reuse its habits (features, splits, sealed tests) at every step. Skip the whole part and the book remains complete; read it and the title's umbrella word, AI, is one you have personally surveyed rather than taken on trust.

Misconception this chapter kills: "AI means LLMs now." The field is older and wider, and for a large class of tabular problems the older machinery is cheaper, clearer, and better behaved than a language model will ever be.

The build: a decision tree classifying iris flowers from four measurements, read back as plain rules and scored on sealed rows; then the labels thrown away and k-means left to find the groups alone.

15.1 The Concept First

AI is the umbrella: machines doing tasks we would call intelligent in a person. Under it sit several families with different ancestries. Symbolic AI reasons over rules and search. Classical machine learning fits models to tables: rows of examples, columns of numeric features, and, for supervised learning, a label to predict. Deep learning fits neural networks with gradient descent, the machinery the next chapter opens, and gave us both convolutional networks (CNNs) that learned to see images as spatial patterns and, eventually, the transformers behind LLMs. The first five parts of this book lived in that last twig of the family tree. This chapter visits the trunk it grew from, because the trunk still carries enormous weight: spam filtering, fraud scoring, demand forecasting, medical risk scores, and recommendation plumbing run overwhelmingly on classical models and their ensembles, as the 150-flower example below shows.

The classical workflow follows a rhythm: assemble a table, seal some rows away for testing before any fitting begins, fit a model on the rest, predict on the sealed rows, and evaluate against the truth you withheld: `fit`, `predict`, `evaluate`. Every later chapter in this part, including the adapter work with its sealed eval set, is that rhythm at greater expense.

15.2 Understanding Through Real Life

Think of a botanist's field key, the printed kind that walks you through a plant by measurements. If the petal is under two and a half centimetres, it is a setosa; otherwise check the width; otherwise check this. A good key is a decision tree written by a person over seasons of observation. The notebook's first build asks what key the data writes for itself when an algorithm chooses the thresholds to best separate the rows it was shown. The second build removes the

guide's species labels entirely and asks a different field question: given only measurements, do these flowers fall into natural groups at all? That is clustering, and its ancestor is every market segmentation and anomaly hunt running quietly in industry: find the lumps in the data first, argue about their names second.

15.3 The Build: a Key the Data Writes Itself

Open `classical_ml.ipynb` and run it top to bottom; this section narrates what to look at. The dataset is the iris table that ships inside scikit-learn, so nothing downloads: 150 flowers, four measurements each, three species in equal numbers. Thirty percent of rows are sealed away stratified by species, leaving 105 training rows and 45 test rows. Fitting is one line, `DecisionTreeClassifier(...).fit(X_train, y_train)`, and it is over before you can look up. After five parts of server round trips, token budgets, and timeouts, a model that trains in milliseconds on your own CPU and needs no account, key, or network is a useful reminder that "AI" did not always imply a data centre.

Then print the model. The learned key opens with the same observation a botanist would, that petal length alone separates setosa cleanly (`petal length (cm) <= 2.45`), and resolves the harder two species through a handful of further width thresholds, eight leaves deep at most. Every prediction the tree ever makes can be traced down those branches by a person with a ruler. Scored on the sealed rows, the run captured for this draft (seed fixed, so yours should match) reaches 0.933 accuracy: 42 of 45 correct, with all three mistakes calling borderline versicolor flowers virginica. Read the mistakes individually, as the notebook does. Their measurements sit where the two species genuinely overlap, which is a property of flowers rather than a scandal in the model, and deciding whether those three errors matter is a judgement no accuracy number makes for you, as Chapter 11's moderation sessions also show.

The second build deletes the answer key. K-means is asked for three clusters in the same measurement space, knowing nothing of species: pick three centres, assign each flower to the nearest,

move each centre to the middle of its assignees, repeat until settled. Against the true species it scores an adjusted Rand index of 0.73 in the captured run, where 1.0 is perfect agreement and 0 is chance. The composition table explains the decimal: one cluster is purely setosa, all fifty, the geometry having found unaided what the tree's first threshold found with labels; the other two clusters split the versicolor and virginica ranges untidily between them, because those species really do blur into each other in these measurements. Unsupervised evaluation argues from structure and usefulness rather than from a withheld answer key, a softer standard that the next chapter's sealed losses will stiffen again.

15.4 What Each Branch Is For

The visit teaches a routing skill, of the same kind Chapter 12 fenced around typed judgments: given a problem, which branch of the family earns the first attempt?

Your material looks like	Reach first for	Because
A table of numbers, a label to predict	Decision tree or its ensembles; SVM beside it	Fast, inspectable, strong baselines, honest error bars
A table of numbers, no labels	k-means and kin	Structure and segments before anyone names them
Images, audio, signals with spatial or time shape	CNNs and deep nets (next chapter's family)	Features must be learned from the raw grid, not hand-slotted
Language, instructions, messy documents, conversation	LLMs and this book's wrappers	Meaning arrives unslotted; that is their whole trick

A support vector machine, the SVM in the first row, earns its mention

as the classical classifier that draws the widest separating boundary it can between classes, the geometric opposite temperament to a tree's axis-aligned questions; the notebook's closing cell swaps it in on identical splits so you can compare held-out scores rather than temperaments in the abstract. Ensembles, forests of trees voting, are why the first row still wins tabular competitions against far larger machinery. The routing cuts both ways. Reaching for a language model to classify rows that already sit in clean numeric columns is usually an expensive way to get a worse, less inspectable answer. Conversely, hand-engineering feature columns for raw text is the 2010s version of the same mistake.

15.5 Where a Demo Can Mislead You

Two symmetrical cons run in this territory. The first relabels: a logistic regression or a gradient-boosted tree that has scored transactions well for a decade is presented, post-2023, as "our AI platform", with the LLM era's glamour borrowed for machinery that neither needs nor survives it. The second overreaches: an LLM demo classifies support tickets from raw text beautifully, and nobody mentions the tree over structured fields that would do most of the same job at a thousandth of the cost with rules a regulator can read. What model family is this, in the taxonomy rather than the brochure? Show me the held-out score and the mistakes, not the accuracy on data it trained on. If it is classical, which features carry the decision, and who chose them? If it is an LLM doing a table's job, what did the table baseline score, and what does the difference cost per million rows?

15.6 Practice Exercises

Exercise 15.1 (Level 1). Run the notebook untouched. From the printed tree alone, classify a made-up flower with petal length 5.0 and width 1.8, then check your hand-trace against `tree.predict`.

Exercise 15.2 (Level 2). Limit the tree to depth 2 and rerun the split. Record the accuracy and readability trade in one sentence

each, in both directions.

Exercise 15.3 (Level 3). Swap the classifier for `SVC` on identical splits. Compare held-out accuracy and the confusion pattern with the tree's. Which mistakes moved, and what does that say about the boundary's shape?

Exercise 15.4 (Level 4). Run k-means with two and with five clusters. Using the composition tables, argue which count the data itself prefers, and state what evidence would change your mind.

Exercise 15.5 (Level 5). Take a classification your workplace currently sends to an LLM (or does by hand). Spec the classical baseline: the table, the features, the label, the sealed split, the acceptance bar that would make the LLM unnecessary. Build it if the data is yours to touch.

15.7 Chapter Summary

- AI is the umbrella; classical ML, deep learning, and LLMs are branches. This book's first five parts occupy the LLM twig, and the title is earned narrowly and now explicitly.
- Supervised classical learning runs fit, predict, evaluate on a sealed split. A decision tree on iris reaches 0.933 held-out accuracy in milliseconds and prints as rules a person can trace.
- Unsupervised k-means finds geometric groups without labels; on iris it isolates setosa perfectly and blurs the overlapping pair, ARI 0.73.
- Routing by material is the skill: tables to trees and SVMs, unlabelled tables to clustering, raw signals to deep nets, language to LLMs. Each branch misused on another's material is a category error with an invoice attached.
- The evaluation discipline (seal first, read mistakes, distrust training scores) is one continuous habit from this chapter through adapters.

15.8 Reflection

Before moving on, ensure you:

- ☐ Traced one prediction down the printed tree by hand
- ☐ Read all the held-out mistakes individually, not only the accuracy
- ☐ Can explain what k-means was never told and how you measured it anyway
- ☐ Routed one problem from your own life to its branch, with reasons

15.9 Your Learning Journal

1. Which felt more like "AI" to you, the readable tree or the fluent chatbot, and what does that feeling track?
2. Where in your work is an LLM doing a table's job, or a spreadsheet doing a language model's?
3. What would your sealed test set be for a model whose errors you currently never count?

15.10 Next Steps

The tree chose its thresholds by testing splits, a tidy, countable search. The deep learning branch could not work that way: its models have millions of continuous knobs, and the only way to set them is the training loop, guess, measure surprise, nudge downhill, that the next chapter runs in miniature.

Chapter 16

Train a Tiny Model

> Training is not uploading knowledge. It is nudging a large pile of numbers until the model's next guess scores better, and a model small enough to train in seconds lets you watch every nudge.

Notebook companion: `train_tiny_model.ipynb` in the package repository's `notebooks/` folder. It runs on a laptop CPU in seconds at its defaults, or on a free Colab GPU if you prefer. This chapter narrates it; the notebook is the doing.

> ⚠ Part VI changes the rules, on purpose
>
> Everything before this part needed a model you could call and the Python in Appendix A. This part needs PyTorch (`pip install torch`, preinstalled on Colab) and introduces two ideas the earlier parts fenced out: loss and gradient descent, each explained in place without calculus. The builds are simplified teaching versions, in the open. Nothing here is how you would train for production, and the chapters say where the production road starts instead.

Misconception this chapter kills: "Training uploads knowledge into the model." Training adjusts weights so next-token guesses score better on examples. What looks like knowledge later is that

adjustment, generalised or memorised, and this chapter shows you the difference on a graph.

The build: a character-level model of 38,874 parameters, trained on ten sentences about this book, sampled before and after training, then overtrained on purpose.

16.1 The Concept First

Strip a language model to its training loop and four moves remain. Represent text as numbers, because arithmetic needs numbers: here, one integer per distinct character, the smallest workable tokenisation. Guess the next token everywhere in a batch of examples, with the current weights. Measure how wrong the guesses were with a loss, a single number that falls as guesses improve, computed here as cross-entropy, which you can read as average surprise at the true next character. Then nudge every weight a small step in the direction that would have reduced that loss on this batch, which is all gradient descent means here: follow the slope downhill, in small steps, many thousands of times.

Frontier training runs are those four moves at far larger scale, on trillions of tokens, with engineering beyond this book's scope. But scale changes the behaviour that emerges, not the identity of the moves. A model that has only ever done this, whose entire education is guessing the next character slightly less badly each step, is the same kind of object whose call you have been making since Chapter 1. When a large model answers from its training in Chapter 5's sense, this loop, on that text, is all the contact it ever had with the material.

16.2 Understanding Through Real Life

Think of learning a piano piece by playing it through against a score you can check note by note. Each pass, you hear where your fingers expected one note and the score had another, and your hands adjust a little. Play one short piece five hundred times and two different

things can happen. You might learn the piece so thoroughly that related music gets easier, the fingering patterns generalising. Or you might merely memorise this one sequence, brilliant at these forty bars and helpless at bar one of anything else. From outside, during practice, both feel like improvement. The test that separates them is a piece you have not practiced.

Training loss is your accuracy on the practiced piece. Validation loss, computed on held-out text the trainer never trains on, is the unpracticed piece. Watching only the first number is how overfitting goes unnoticed.

16.3 The Build: Ten Sentences and a Sealed Test

Open the notebook and run it cell by cell the first time. The corpus is ten sentences about this book ("the model proposes and the code disposes" and its friends), small enough that you can audit every word the model could possibly learn. Eight sentences form the training text; two are held out whole as the sealed validation set, the same discipline the LocoLabo capstone enforces on its training data. The notebook lets you prove why that whole-sentence split matters: split repeated text at a character offset instead and the same sentences appear on both sides, after which validation loss flatters you by measuring memorisation and calling it learning. Leakage of that kind, in larger datasets, is behind a share of published AI results.

Before training, the notebook samples from the fresh model. Keep the output. In the run captured for this draft, seeded for reproducibility, the untrained model produced this, starting from the characters `the`:

```
the ifolhpydttyctri.jfemckylhcr.dpkxu.fsdscouotlf
x deorxxxaoxdyilatyle
```

That is what near-random weights sound like: the alphabet available, no preferences worth the name. Then 600 steps of training, which

took about four seconds on the machine that captured this run:

```
step    0  train 3.322  val 3.350
step  100  train 1.500  val 3.027
step  200  train 0.606  val 4.477
step  300  train 0.413  val 5.260
step  400  train 0.332  val 5.553
step  500  train 0.248  val 5.862
step  600  train 0.233  val 6.154
```

At step 0 the model is equally surprised by everything, training and held-out text alike. Training loss then falls without pause, as expected. Validation loss rises almost from the start. The model is not learning English or even these sentences' patterns in any general sense; it is memorising eight sentences with increasing fidelity, at the expense of the two it has never trained on. Sample it and the memorisation is audible: near-verbatim corpus sentences at low temperature, degrading gracefully as temperature rises and the sampler is allowed to choose less probable characters. Temperature, met here as a dial between conservative and adventurous sampling from the same weights, matters when hosted models expose it as a setting: it changes the choosing without changing the knowledge.

The notebook then overtrains deliberately, four times more steps, and the finishing figures in the captured run (training loss 0.153, validation 7.984) show the same pattern further along. Nothing malfunctioned: the optimiser did its job on the objective it was given, examples it was shown, and a corpus far too small to generalise from. Scale accounts for most of the difference between this small run and the models in Part I. The loop, the loss, the need for sealed evaluation, and the ease of lying to yourself with the wrong split did not vary with scale.

16.4 What This Buys You as a Reader

You will likely never pretrain a model anyone uses, and neither will most professional builders of AI systems. The returns on this afternoon are conceptual, and they carry through the rest of Part

VI and back into the earlier parts. Training data is now a concrete thing you have assembled and audited by eye, which changes how you read claims about what a model "was trained on". Loss is a number whose two versions you have watched diverge, which is the permanent antidote to training-curve screenshots in vendor decks. And generation is demystified at the source: the fluent text of Part I is sampling from next-token scores at some temperature.

A one-layer, 38,874-parameter character model teaches the loop, not the capabilities: word-level tokenisation, depth, instruction tuning, reinforcement from feedback, and data curation at scale are each worlds this notebook gestures past. The notebook's closing cell points at Karpathy's *Zero to Hero* for the concepts taught properly and at nanochat, nanoGPT's successor codebase (nanoGPT itself is now deprecated by its author), for readers who want this chapter at full size. Appendix D carries the same pointers in the exit map.

16.5 Where a Demo Can Mislead You

"We trained our own model" spans everything from this notebook to a frontier run, and the sentence alone tells you nothing. Trained on what, how much of it, and who audited the corpus the way you just audited ten sentences? What was sealed for evaluation, and can I see validation beside training performance rather than a sample of outputs? Is this full training, continued training, or an adapter over someone else's base, which is the next chapter's distinction and the one most often elided in pitches? Look for answers that name data cards, splits, and held-out numbers.

16.6 Practice Exercises

Exercise 16.1 (Level 1). Run the notebook untouched. Save the before-training sample, the loss table, and both temperature samples. Explain the validation column to an imaginary colleague in three sentences.

Exercise 16.2 (Level 2). Repeat with the leakage split the notebook

warns about (one repeated text, cut at a character offset). Compare validation behaviour and state, in writing, which split you would trust for a result you intended to publish.

Exercise 16.3 (Level 3). Swap in your own small text, commit messages, a poem, your meeting openings. Hold out a sensible sealed slice for that genre and defend the split. What does the trained model capture, and what does it plainly not?

Exercise 16.4 (Level 4). Vary one setting at a time (model width, steps, learning rate) across three runs with the seed fixed. Record which moved validation loss and which only moved training loss. That table is your first hyperparameter intuition, earned cheaply.

Exercise 16.5 (Level 5). Write the data card for your tiny corpus as if for publication: source, size, split discipline, known biases, what a model trained on it should never be trusted to do. One page, suitable for attaching to the notebook.

16.7 Chapter Summary

- Training is four moves on repeat: tokenise, guess next tokens in batches, measure surprise as loss, nudge weights downhill. Scale changes what emerges, not what the moves are.
- A 38,874-parameter character model trained on eight sentences memorises them in seconds; sealed validation sentences expose the memorisation as training loss falls and validation loss rises.
- Splits are methodology, not bookkeeping. Splitting repeated text leaks the test into training and makes validation lie politely.
- Sampling temperature changes the choosing, not the knowledge.
- This notebook is a teaching instrument: real pretraining adds tokenisation, depth, data curation, and post-training this chapter only names.

16.8 Reflection

Before moving on, ensure you:

- ☐ Ran the notebook and kept the before-training noise as your baseline
- ☐ Can explain the diverging loss columns without the word "learning" doing any work
- ☐ Tried both split disciplines, or can state exactly how their validation numbers would differ
- ☐ Know which cell you would point a sceptic at to prove generation is sampling

16.9 Your Learning Journal

1. At what step did the samples stop being noise to your ear, and what had the losses done by then?
2. Describe a published AI result you now want to ask about the split.
3. What would your ten sentences be, if a tiny model's whole world had to be your writing?

16.10 Next Steps

You have now trained a model from nothing and watched it memorise. Almost nobody in practice starts from nothing: they start from a capable base model and adjust its behaviour with a comparatively tiny amount of additional training. The next chapter does that with a LoRA adapter, your own examples, and the same sealed evaluation, and asks whether it actually beat the base.

Chapter 17

Teach It a Dialect

> Fine-tuning teaches a model how to answer here: the voice, the format, the habits. Facts still belong in evidence you can point at.

Notebook companion: `finetune_adapter_colab.ipynb` in the package repository's `notebooks/` folder, distilled from the LocoLabo capstone training notebook your author supervises. It needs a free Colab GPU. Budget tens of minutes at this chapter's scale, not the capstone's hours.

> ⚠ Still the advanced arc
>
> This chapter needs the notebook machinery from the previous one plus Unsloth in Colab, and it will happily consume an afternoon on data preparation alone. That is not overhead around the lesson. At this scale, data preparation is most of the lesson. Simplified throughout, as flagged: a production fine-tuning programme adds data governance, regression suites, and serving decisions this chapter only names.

Misconception this chapter kills: "Fine-tuning teaches the model our facts." An adapter mostly learns from relatively few examples how answers should sound and behave here. Whether it beats the base model at anything is an empirical question with a sealed test,

and the honest answer is sometimes no.

The build: a LoRA adapter over Qwen3-4B teaching one plain-English answering dialect, exported to Ollama, then judged blind against its own base model.

17.1 The Concept First

Almost nobody retrains a capable model from scratch to specialise it, for the arithmetic reason Chapter 16 made vivid: full training touches every weight, with the memory and compute to match. Fine-tuning starts from a pretrained base and continues the same next-token training on a small, deliberate dataset: examples of the behaviour you want more of, formatted as the conversations the model should learn to continue. LoRA, low-rank adaptation, goes one step thriftier. The base weights are frozen, and a small set of extra weights wired alongside key layers is the only thing trained: in the notebook's configuration, well under one percent of the total parameter count. At serving time the adapter can be merged back into the base to produce one ordinary model file, which is what the notebook exports for Ollama.

The proportion governs what this technique can and cannot do. A fraction of a percent of movable weights, trained on dozens to hundreds of examples, is enough to shift voice, format discipline, refusal habits, and which of the base model's existing knowledge gets foregrounded in a domain. It is a poor place to store a policy manual. Facts planted by fine-tuning arrive with no citation, go stale silently, and compete with everything else in the base, while Chapter 6's retrieval supplies facts at question time with sources attached. The division of labour is therefore: retrieval for what is true, system prompts for what is cheap to state, and an adapter only for behaviour that examples teach better than instructions do.

17.2 Understanding Through Real Life

Think of accent coaching for an already fluent speaker joining a regional service desk. You do not reteach them the language. You give them fifty recorded calls that sound right for this desk: the greeting shape, the local terms, the calm way of saying no, and you drill until the habits are theirs under pressure. And you would not coach product prices into their accent, however many times they repeat them in training calls, because prices change next quarter and the information belongs on the screen in front of them, not in their muscle memory.

The measurement discipline matches too. You would not certify the coaching by replaying the practice calls. You would hold back calls the coach never used, have the newcomer handle them, and compare against how they sounded on day one, judged by the desk's own standards.

17.3 The Build: Data First, Because Data Is the Build

Open the companion notebook at Steps 3 and 4. The training format is JSONL, one conversation per line, user then assistant, and the notebook opens with a miniature built-in dataset teaching a single dialect, short plain-English answers, so the whole pipeline can be smoke-tested before your own data is at stake. Then come the checks: every line parses, turns alternate correctly, no assistant answer is empty, duplicate prompts are flagged, and training prompts are intersected with eval prompts to catch leakage by name. Read two examples by eye before any GPU wakes up, because the model will imitate the assistant column with total sincerity, typos, hedges, evasions, and all. In fine-tuning, the dataset's quality sets your ceiling.

Two scaling notes separate the book build from the capstone it distils. The capstone demands at least 500 training examples and 50 sealed eval examples before a proposal is even approved, because

its question is whether a domain adapter beats a 4B base in earnest. The notebook here builds 60 smoke-test records from four question-answer pairs repeated, which is enough to exercise every mechanism and nowhere near enough to claim a specialisation. Your real run sits between: 50 to 100 of your own examples with a sealed eval set you wrote before training, not after seeing results. Write the eval first. Chapter 11 required the same moderation before marks.

17.4 Train, Export, and Put It Back on the Bench

The middle of the notebook is the LocoLabo machinery at book scale: Unsloth loads Qwen3-4B in 4-bit so a free GPU can hold it, `get_peft_model` attaches the LoRA layer to the attention projections, examples are rendered through the model's own chat template (worth reading once, to see the exact markers the model trains on), and an SFT trainer runs three epochs at the recorded settings. Watch the loss with Chapter 16's eyes: on a set this small it will head toward memorisation quickly, and the training loss is the rehearsal number, of interest mainly when it misbehaves. Then the merge: the notebook exports a quantised GGUF, roughly two and a half gigabytes, and the instructions walk it out of Colab before the session deletes it, a failure mode with its own graveyard of lost evenings.

Back on your machine the artefact rejoins the world of Part I. A one-line Ollama Modelfile (`FROM ./your-file.gguf`) registers it under a name, `HANDS_ON_AI_MODEL` points your earlier chapters' scripts at it, and the department has a new intern whose only difference from the old one is the dialect you drilled. Nothing else in your stack changes: fine-tuning, done this way, is a change of model file, evaluated like any other model change from Chapter 4.

17.5 The Only Question That Matters: Did It Beat the Base?

Ask every sealed question of both models, the untouched base and your adapter, ideally with the answers shuffled and their sources

hidden from whoever marks. Mark them against the criteria your dialect claims: plainness, brevity, format, whatever the adapter was for, using Chapter 11's judge as the tireless marker and your own reading of a sample as the moderation. Report both scores and read some actual answers, not only the aggregate, because a point gained in style while losing a fact is visible only at reading distance.

At this scale, unfashionable outcomes are common. The adapter may win on voice and lose on substance, which says your examples taught manner at the expense of content. It may match the base everywhere, which usually says a system prompt would have bought the same shift for almost nothing, and Chapter 2 should be tried first next time. Or it may lose, through too few examples, too many epochs, or a dialect the base already spoke. The capstone this notebook descends from states its position in its brief, and this chapter adopts it whole: a negative result with a rigorous explanation is a better piece of work than a flattering number nobody can defend. Publish the training log either way; the log template travels with the notebook materials for that reason.

17.6 Extension: Learn from Preferences

Supervised fine-tuning imitates examples. Preference learning goes one step further and adjusts weights from comparisons: for the same prompt, this answer over that one. The simplest teachable form is DPO, direct preference optimisation, which consumes triples of prompt, chosen answer, and rejected answer and nudges likelihood toward the chosen style without a separate reward model to babysit. Your supply chain for pairs already exists in this book: generate two candidate answers per sealed-style prompt with your SFT model, rank them against your criteria (your marks first, the judge's second, per Chapter 11's order of trust), and file the winners and losers. The companion notebook's closing section sketches the additional cells; treat them as a second sitting, not a first run.

Beyond DPO sits online reinforcement learning proper, GRPO being the current teachable shape: sample several answers, score them with a reward, push weights toward the better-scoring behaviour,

repeat. It earns its complexity only where the reward can be computed: arithmetic checked by Python, formats validated by a parser, code judged by its tests. At inference time, simple reward-guided search exists, and you built it in Chapter 10: the ratchet changes no weights. In weights, the simple on-ramp is preferences offline and verifiable rewards online; beyond that lies a research programme. Each step up this ladder is judged by the same sealed set, untouched by SFT, DPO, and every threshold conversation in between.

17.7 Where a Demo Can Mislead You

"Fine-tuned on our data" is a claim about a process, not a result, and it demos beautifully because style is the easiest thing an adapter changes and the first thing an audience feels. Ask for the base model's score on the same sealed questions, the size and provenance of the training set, who wrote the assistant column and to what standard, and what the system does about the facts the adapter confidently absorbed wrong. Then ask the substitution question this chapter makes unavoidable: what here could retrieval or a system prompt not have done, cheaper and with citations? Look for answers that give behaviour examples and eval tables.

17.8 Practice Exercises

Exercise 17.1 (Level 1). Run the notebook's data cells only (no GPU work): build the miniature dataset, run the checks, and deliberately introduce one malformed record, one duplicate, and one leaked eval prompt. Confirm each is caught by name.

Exercise 17.2 (Level 2). Write your sealed eval set first: ten questions with the answers your dialect should produce, marked against criteria you state now. Date the file. Nothing in training may touch it.

Exercise 17.3 (Level 3). Complete the smoke-test training run and export. Register the result in Ollama beside the base and ask both models three training questions and three sealed ones. Record

where rehearsal and readiness diverge.

Exercise 17.4 (Level 4). Grow to your own 50-plus examples in a dialect you care about. Train, export, and run the blind comparison. Write the training log entry the capstone would expect, whichever way the result lands.

Exercise 17.5 (Level 5). Build preference pairs from your adapter's sealed-set answers, ranked by your marks with the judge as second reader. Spec the DPO run (pairs count, epochs, what would make you stop) and the exact result that would convince you it beat SFT alone. Run it if the spec survives your own reading.

17.9 Chapter Summary

- LoRA fine-tuning freezes the base and trains a small added layer, well under one percent of weights here, on conversation examples; merged and quantised, it serves as an ordinary Ollama model.
- Adapters teach voice, format, and habits. Facts belong in retrieval, cheap instructions belong in system prompts, and the substitution question comes before any training run.
- The dataset is the lesson: format checks, duplicate and leakage detection, and reading examples by eye precede the GPU work and matter more than the settings.
- Success is defined against a sealed eval, base versus adapter, marked blind. A negative result, rigorously shown, is a legitimate and publishable outcome.
- Preference learning extends the arc in order: ratchet search (no weights), DPO on ranked pairs, GRPO only with verifiable rewards. The sealed set judges all three.

17.10 Reflection

Before moving on, ensure you:

- ☐ Can state what your adapter is for in one sentence a system prompt could not achieve

- ☐ Wrote your sealed eval before training and kept it untouched
- ☐ Compared base and adapter blind, and read raw answers beside the scores
- ☐ Logged settings, loss, and outcome, especially if the answer was no

17.11 Your Learning Journal

1. What did your assistant column teach that you did not know you had written?
2. Where did the adapter beat the base honestly, and where did it merely sound more like your examples?
3. Which behaviour in your working life is currently bought with a long system prompt that fifty examples might teach better?

17.12 Next Steps

Part VI ends where the book's first page began, with a model call, except that you have now stood on both sides of it: directing pre-trained models through their front door, and adjusting weights yourself at toy scale. The conclusion gathers all six parts into the single shape they share. Readers going deeper from here have two roads: Karpathy's line for training itself, in Appendix D, and the LocoLabo capstone this chapter distilled, for adapter work done in teams on real domains.

Part VII

Looking Underneath

Chapter 18

The Shape Underneath

> You came for seventeen chapters of machinery and found one call, five things people wrap around it, and your own judgement at every seam.

Package companion: all of `hands-on-ai` 0.6.0, which you have now read in the pieces that matter.

Misconception this chapter kills: "I need to learn a framework." Frameworks will come and go above this shape. You now own the shape.

The build: none. This chapter assembles what you built into one picture and hands you the questions to carry out of the lab.

18.1 The shape, in six drawers

Almost every system in this book is one call to a model, wrapped in plain Python. The six parts were five families of wrapper plus, in the optional Part VI, a look at how the object inside the call is made. Each part killed the misconception that its wrapper was the intelligence:

Part	Wrapper added	The plain version
I. The Model	Instructions, transcript, settings	A payload, a system string, a resent list, three variables
II. Grounding	Evidence fetched before the call	Search, then the results pasted into the prompt
III. Action	Functions the model may request, in a loop	Your code executes, results return as messages
IV. Control	Stages, goals, scores, typed judgments	Folders of contracts, a stated goal, a calibrated marker, a schema
V. The Senses	Pictures in, translators at the edges	An image in the payload; speech to text and back
VI. Inside the Box (optional)	Weights adjusted in miniature	A training loop and an adapter, judged on sealed eval

Point at any AI product you use this week and the decomposition exercise from Chapter 1 now has six drawers to sort its behaviour into, the last one asking what the model itself was trained and adapted on. The chatbot is Part I with a budget. The document helper is Part II with better retrieval. The agent is Part III with a larger bench and higher stakes. The platform is Part IV with dashboards where your folders and logs were. The voice assistant is Part V wrapped around whichever of the earlier parts does its thinking. Products differ, sometimes enormously, in the quality and scale of their wrappers. Almost none differ in the underlying event: text, and now pictures and sound by conversion, assembled into a payload; a model call; the result checked, filed, or fed around again by code someone wrote on a Tuesday.

18.2 The questions to carry

Each chapter ended by slowing a demo down, and the questions collect into a single habit. What did this add around the plain call? Where does the evidence come from, and can I open it? Who executes, on what allowlist, and what happens on the worst input? Where are the stage outputs, the traces, the logs, and which exit ended this run? Who marked the quality, against what criteria, calibrated against whose reading? Where did my words, pictures, and voice travel, and how long are they kept?

Models will be replaced under you quarterly; several already have been while this book was written. The wrappers will be renamed by vendors twice a year. The questions survive both, because they attach to the seams where someone's code met a model call and a decision got made about evidence, authority, cost, or proof. Asking them of your own builds first is the difference between supervising AI systems and merely operating them, and it is the same difference this series' other titles circle from their own directions: *Conversation, Not Delegation* for the thinking discipline, *Speak Software* for directing agents without code, *Prove, Don't Persuade* for insisting on evidence over demonstration.

18.3 Leaving the lab

This package was built to be outgrown, and its own philosophy says so without embarrassment: a learning lab, small on purpose, trading completeness for clarity so the concepts stay visible. The moment your needs name a gap here, error handling with real retries and budgets, streaming at scale, a vector store, judged evaluation over thousands of cases, agents running tools concurrently, voice with interruption handling, you have reached the exit it was aimed at, carrying the concepts a production framework will assume you have. Appendix D maps each module to its grown-up counterparts, and *Ship Python, Orchestrate AI* in this series takes the professional road in full: the same shapes, built to be operated by strangers.

One graduation exercise is worth doing before you shelve the lab.

Pick the module you leaned on hardest and read its whole source in one sitting, pen in hand, answering for every function the question Chapter 1 taught you: what does this add to the plain call? The chat module will take an afternoon. So will the rest together. A reader who has done that once stops asking whether they are technical enough to judge an AI system, which was the quiet hope underneath the preface.

18.4 Practice Exercises

Exercise 18.1 (Level 1). Choose an AI product you used this week and sort its behaviour into the six drawers, one line per drawer, naming what you are inferring rather than observing.

Exercise 18.2 (Level 2). Revisit your earliest learning journal entries. Find one belief about AI systems that a specific chapter's build, not its prose, changed for you. Name the listing.

Exercise 18.3 (Level 3). Assemble two parts on one bench: a voice loop whose answers are grounded by your document index, or a judge watching your workflow's draft stage. Keep it small; the composition is the exercise.

Exercise 18.4 (Level 4). Write the one-page version of this book for a colleague: the shape, the five questions, and the single exercise from any chapter that would teach the most in an hour. Defend your choice of exercise.

Exercise 18.5 (Level 5). Capstone. Build one small thing for real use, from Appendix C or your own backlog, under this book's disciplines: stated criteria, a checkable goal, sources shown, errors visible, a budget. Ship it to one other person and record what their use taught you that your testing did not.

18.5 Chapter Summary

- Almost every system built here is one model call plus a wrapper: the call itself, grounding, action, control (typed judg-

ments included), and the senses. Part VI adds where the weights inside the call come from, and how little of them an adapter moves.
- The durable skill is seam questions: evidence, execution authority, traces and exits, calibrated quality, data travel and retention.
- The package is a learning lab designed to be outgrown. Reaching its limits with the concepts intact is success, and Appendix D is the exit map.

18.6 Reflection

Before you close the book, ensure you:

- ☐ Can decompose a product you use into the six drawers without notes
- ☐ Read one whole package module against the Chapter 1 question
- ☐ Know which appendix you will open next, and why

18.7 Your Learning Journal

1. Which misconception, of the seventeen, were you most reluctant to give up?
2. What will you now refuse to accept from a demo, including your own?
3. What is the first production-shaped problem you intend to point these habits at?

Appendix A

Just Enough Python

> This book uses about a dozen Python constructs. This appendix teaches all of them, and nothing else.

You do not need to read this appendix first. Each chapter names the Python it needs and points back here. Read the section you need, when you need it, and return to the chapter. Every example in this appendix is complete, runs without a model or a network, and prints exactly what is shown beneath it. Type them in rather than pasting.

If you want to learn Python itself, properly and for its own sake, that is a different book: *Code Python, Consult AI* in this series. After this appendix, no listing in this book will contain a construct you have not met.

A.1 Running a Script

A Python program in this book is a plain text file whose name ends in `.py`. You write it in any text editor, save it, and run it from a terminal:

```
python ask.py
```

Python reads the file from the top, line by line, and does what each line says, in order. When it reaches the end of the file, the program

is over. There is no separate compile step; the first line is the start.

Lines beginning with # are comments. Python ignores them. Comments are notes for human readers, and in this book they often carry the listing number and the file name to save the code as.

```
# This line is a comment. Python skips it.
print("This line runs.")
```

```
This line runs.
```

First used in Chapter 1.

A.2 print: Showing a Value

`print` writes a value to the terminal. Getting a value and showing a value are different acts, and Chapter 1 makes you delete a `print` once to feel the difference.

```
print("Hello")
print(42)
```

```
Hello
42
```

The words `Hello` are inside quotation marks, which makes them text. The `42` has no quotation marks, which makes it a number. That distinction, text versus number, is the next two sections.

First used in Chapter 1.

A.3 Variables: Named Values

A variable is a name attached to a value. The `=` sign attaches it. Read `answer = "yes"` as "the name answer now refers to the text yes".

```
name = "Sam"
age = 31
```

```
print(name)
print(age + 1)
```

```
Sam
32
```

A variable can be reassigned, and the old value is gone. A variable can also hold the result of a calculation or, throughout this book, the text a model returned. Choose names that say what the value is: `question`, `answer`, and `usage` carry you through most of Part I.

First used in Chapter 1.

A.4 Strings: Text

Text in Python is called a string: characters between quotation marks. Single and double quotes both work, and a string can contain the other kind of quote without trouble.

```
greeting = "It's working."
print(greeting)
print(len(greeting))
```

```
It's working.
13
```

`len` counts the characters, spaces and punctuation included. Two string operations appear constantly in this book. Adding strings with `+` joins them end to end. And an f-string, a string with the letter `f` before its opening quote, lets you drop values into the middle of text by naming them inside curly braces:

```
name = "Sam"
tokens = 53
print("Name: " + name)
print(f"{name} used {tokens} tokens.")
```

```
Name: Sam
Sam used 53 tokens.
```

Chapter 6 builds a prompt by assembling strings this way; `f"..."` there is an f-string like the one above.

First used in Chapter 1. F-strings first used in Chapter 6.

A.5 Imports and pip: Borrowing Code

Python code written by other people is organised into packages. `pip install` downloads a package onto your machine once:

```
pip install "hands-on-ai==0.6.0"
```

An `import` line inside your script then borrows named pieces of that package for that run:

```
from hands_on_ai.chat import get_response

print(type(get_response))
```

```
<class 'function'>
```

Read the import as a path: inside the package `hands_on_ai`, in its `chat` section, find the thing called `get_response`. After that line, your script can use the name `get_response` as if you had written it yourself. If the package is not installed, Python stops at the import line with a `ModuleNotFoundError`, which is the error message's way of saying the install step was skipped.

First used in Chapter 1.

A.6 Calling Functions

A function is a named piece of work that takes inputs, called arguments, and hands back a result. You call it by writing its name followed by parentheses holding the arguments:

```
words = "the model sends text back".split()
print(words)
print(len(words))
```

```
['the', 'model', 'sends', 'text', 'back']
5
```

Arguments can be given by position, as above, or by name. A named argument spells out which input it fills, and named arguments can often be left out, in which case the function uses a default value chosen by its author:

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}."

print(greet("Sam"))
print(greet("Sam", greeting="Welcome back"))
```

```
Hello, Sam.
Welcome back, Sam.
```

That example defines a function too, which is the next section. `get_response(question, system="...")` in Chapter 2 uses the same pattern: one argument by position, one by name, the rest left at their defaults.

First used in Chapter 1. Named arguments first used in Chapter 2.

A.7 Defining Functions: Naming Your Own Work

The word `def` begins a function definition. The indented lines beneath it are the function's body, and `return` sends a value back to the caller:

```
def add_tax(price):
    return price * 1.10

print(add_tax(20))
```

```
22.0
```

Indentation matters in Python. The four spaces before `return` are how Python knows the line belongs to the function. Most editors

insert them for you when you press Tab after a line ending in a colon. If Python complains about indentation, check that every line of the body lines up exactly.

Chapter 2 wraps a system prompt in a function to make a bot. The function there does no arithmetic: it calls `get_response` and returns whatever came back.

First used in Chapter 2.

A.8 Lists: Ordered Collections

A list holds values in order, inside square brackets, separated by commas:

```
roles = ["system", "user", "assistant"]
print(roles[0])
print(len(roles))
roles.append("user")
print(roles)
```

```
system
3
['system', 'user', 'assistant', 'user']
```

Counting positions in a list starts at 0, so `roles[0]` is the first item. `append` adds one item to the end. The message list in every payload in this book is a list, and `Conversation` in Chapter 3 spends its life appending to one.

First used in Chapter 1, where the printed payload's `messages` value is a list.

A.9 Dictionaries: Named Collections

A dictionary holds values under names, called keys, inside curly braces. Each entry is a key, a colon, and a value:

```
message = {"role": "user", "content": "What is my
name?"}
print(message["role"])
print(message["content"])
```

```
user
What is my name?
```

You read a value out by giving its key in square brackets. Each message in a payload is a dictionary with two keys, `role` and `content`, and the usage report from Chapter 1 is a dictionary with three. Environment variables in Chapter 4 live in a dictionary called `os.environ`, which your code reads and writes the same way:

```
import os

os.environ["HANDS_ON_AI_MODEL"] = "llama3"
print(os.environ["HANDS_ON_AI_MODEL"])
```

```
llama3
```

First used in Chapter 1. `os.environ` first used in Chapter 4.

A.10 Loops: Doing Something to Each Item

A `for` loop runs its indented body once per item in a list:

```
chunks = ["first chunk", "second chunk", "third
chunk"]
for chunk in chunks:
    print("-", chunk)
```

```
- first chunk
- second chunk
- third chunk
```

The name `chunk` is a variable that takes each item's value in turn. The body can do more than print. Chapter 6 loops over retrieved chunks to build one long prompt string, adding a line per chunk:

```
prompt = "Context:\n"
for chunk in ["alpha", "beta"]:
    prompt += f"- {chunk}\n"
print(prompt, end="")
```

```
Context:
- alpha
- beta
```

`\n` inside a string means a newline, the invisible character that ends a line. And `+=` means "add this to the existing value", so `prompt += f"- {chunk}\n"` grows the string a line at a time.

First used in Chapter 6.

A.11 Objects and Methods: Verbs After a Dot

Some values carry their own operations. You reach those operations with a dot, and an operation reached by a dot is called a method. You have already used two in this appendix: `split` on a string and `append` on a list.

Chapter 3 introduces the pattern at full size. `Conversation(system="...")` creates an object, a self-contained bundle holding a message list and a token total. The object is stored in a variable, usually `chat`. After that, `chat.ask(...)`, `chat.save(...)`, and `chat.reset()` call methods on that bundle, and `chat.total_tokens` reads a value stored inside it:

```
class Counter:
    def __init__(self):
        self.total = 0

    def add(self, amount):
        self.total += amount
        return self.total

counter = Counter()
print(counter.add(5))
```

```
print(counter.add(3))
print(counter.total)
```

```
5
8
8
```

That is a complete, if tiny, class. `Counter` is the template, `counter` is one object made from it, and the object remembers its total between calls because the total lives inside the object. This book asks you to use two or three classes written by the package. The `self` you see repeated is how the object refers to its own stored values; you can read past it safely.

First used in Chapter 3.

A.12 True, False, and None

Three special values appear throughout the book. `True` and `False` are Python's yes and no, produced by comparisons and consumed by settings like `return_usage=True`. `None` means the absence of a value: no usage reported, no model chosen yet, nothing found in the cache.

```
usage = None
print(usage is None)
print(3 > 2)
```

```
True
True
```

When a listing checks `if usage:` before reading token counts, it is guarding against `None`. Chapter 1 explains the two ordinary situations where usage is `None`, and neither is an error.

First used in Chapter 1.

A.13 Reading an Error Message

Sooner or later Python will stop and print a traceback: several lines naming the file and line where things failed, ending with the complaint. Read tracebacks from the bottom. The last line says what went wrong. The lines above it say where. Three complaints cover most of what beginners meet in this book:

Last line begins	Usually means	First thing to check
`ModuleNotFoundError`	A package is not installed	Run the `pip install` step
`NameError`	A name was used before it existed	Spelling, or a missing import
`SyntaxError` or `IndentationError`	Python could not parse the file	The line it points at: quotes, colons, indentation

Errors from the model call itself are different. As Chapter 1 explains, `get_response` returns them as strings beginning with □ or □ rather than stopping your program, and `handsonai doctor` is the first tool for those.

A.14 A Worked Example Using Most of It

This script uses an import, a list of dictionaries, a loop, an f-string, and a function definition. It is the payload from Chapter 1, rebuilt from the pieces in this appendix, with no model involved:

```
import json

def build_payload(model, question):
    messages = [
        {"role": "system", "content": "You are a
        helpful assistant."},
        {"role": "user", "content": question},
    ]
```

```
    return {"model": model, "messages": messages}

payload = build_payload("llama3", "What is my
name?")
for message in payload["messages"]:
    print(f'{message["role"]}:
    {message["content"]}')
print(json.dumps(payload))
```

```
system: You are a helpful assistant.
user: What is my name?
{"model": "llama3", "messages": [{"role":
"system", "content": "You are a helpful
assistant."}, {"role": "user", "content": "What is
my name?"}]}
```

If that script reads as ordinary to you, you have enough Python for this book. Return to the chapter that sent you here.

Appendix B

Model Access

> Every setting this book needs, in one place: which server answers, which models fill which roles, what each chapter spends, and what to check when nothing answers.

The package resolves its configuration in a fixed order, weakest first: built-in defaults, then the config file at `~/.hands-on-ai/config.json`, then environment variables, which win over everything. `handsonai doctor` prints the resolved result and probes the server. `handsonai config` shows and edits the file layer. `handsonai models` lists what your server reports. When in doubt, run the doctor before touching code; it reads the same configuration your scripts will, in the same order.

B.1 The Settings

Variable	Role	Default
`HANDS_ON_AI_SERVER`	Chat, embedding, and vision server address	`http://localhost:11434`
`HANDS_ON_AI_API_KEY`	Key for every server above	unset (local servers ignore it)
`HANDS_ON_AI_MODEL`	Chat model	`llama3`

Variable	Role	Default
`HANDS_ON_AI_EMBEDDING_MODEL`	Retrieval embeddings, Chapter 6	`nomic-embed-text`
`HANDS_ON_AI_VISION_MODEL`	Vision model, Chapter 13	`moondream`
`HANDS_ON_AI_STT_SERVER`	Speech-to-text server, Chapter 14	the main server
`HANDS_ON_AI_STT_MODEL`	Transcription model	`whisper-1`
`HANDS_ON_AI_TTS_SERVER`	Text-to-speech server, Chapter 14	the main server
`HANDS_ON_AI_TTS_MODEL`	Speech model	`kokoro`
`HANDS_ON_AI_CACHE`	Response cache directory	unset, caching off
`HANDS_ON_AI_LOG`	Set to `debug` for verbose logging	unset

Each row has a matching key in the config file (`server`, `api_key`, `model`, `embedding_model`, `vision_model`, `stt_server`, `stt_model`, `tts_server`, `tts_model`) for settings you want to survive terminal sessions. Environment variables still take precedence.

B.2 The Free Local Path

Ollama covers chat, embeddings, and vision from one install at `http://localhost:11434` with no account and no key. The pulls this book needs:

```
ollama pull llama3            # chat (Parts I to
IV)
ollama pull nomic-embed-text # retrieval (Chapter
6)
ollama pull moondream         # vision (Chapter 13)
```

Audio is the local exception. Transcription and speech typically come from a small separate server that speaks the same

`/v1/audio/` dialect: speaches or whisper.cpp serving a Whisper model for speech-to-text, Kokoro or Piper for text-to-speech. Point `HANDS_ON_AI_STT_SERVER` and `HANDS_ON_AI_TTS_SERVER` at them and Chapter 14 works untouched. Local everything is the private, offline, per-call-free route; its costs are your hardware's limits on model size and patience, felt most in Parts III and V.

B.3 Hosted Providers

Any service exposing the OpenAI-compatible chat and embeddings endpoints can drive most of the book through the three variables from Chapter 4. The package documentation carries worked examples; the shapes are uniform:

```
export HANDS_ON_AI_SERVER="https://api.openai.com"
# OpenAI
export
HANDS_ON_AI_SERVER="https://openrouter.ai/api"
# OpenRouter
export
HANDS_ON_AI_SERVER="https://api.groq.com/openai"
# Groq
export HANDS_ON_AI_API_KEY="your-key-here"
export
HANDS_ON_AI_MODEL="the-provider's-model-name"
```

Audio and vision coverage varies by provider, so check the endpoints you need before a Part V weekend. Free tiers come and go and are deliberately not enumerated here; the package documentation and the providers are the live sources. One classroom pattern: one shared local or hosted server behind a key, the three variables issued to students, and the cache (below) warmed before the lesson so a network failure cannot fail thirty exercises at once.

B.4 What Each Chapter Spends

Local calls cost time; hosted calls cost money per token. Either way, the shape of the spend per build is knowable in advance:

Build	Calls, roughly
Part I scripts	1 per line you run
`Conversation`	1 per turn, over a resent transcript that grows
Indexing, Chapter 6	1 embedding call for all chunks, then 1 per question plus its embedding
One agent run	1 per iteration, up to `max_iterations`
One judged loop iteration	1 step call + 1 judge call
One ratchet proposal	1 step call + 1 scoring call, plus 1 to score the start
One vision question	1, carrying a large payload, on a slow clock
One voice turn	1 transcription + 1 chat + 1 synthesis

Budgets in the chapters (`max_iters`, `max_iterations`) are counted in iterations, not calls. Multiply before you fund a run.

B.5 The Response Cache

Setting `HANDS_ON_AI_CACHE` to a directory turns on a plain-text reply cache for `get_response`: identical model, system prompt, and question return the stored answer without contacting a server, and with token usage reported as `None`. Warm it once for a classroom and reruns become reproducible, offline-capable, and free. Clear it by deleting the directory. Remember from Chapter 4 that it is also a folder of transcripts on disk, and that a model or prompt change does not invalidate old entries until the name inside the lookup

changes. Note its scope: the cache sits in `get_response`, so direct `Conversation` turns and vision calls bypass it.

B.6 Troubleshooting

Symptom	Likely cause	First check
`Error: Connection error.`	Server off, wrong address, blocked network	`handsonai doctor`; read the resolved server aloud
Error naming a model	Model never pulled, or name mismatch	`handsonai models`; compare with your settings exactly
401 or 403 from a hosted server	Missing or wrong key	`HANDS_ON_AI_API_KEY` present in *this* terminal
Chat works, Chapter 6 fails	Embedding model missing	`ollama pull nomic-embed-text`, or the embedding setting
Chapter 13 fails, chat fine	Vision model missing or not vision-capable	`vision_model` setting; pull `moondream`
Chapter 14 fails, all else fine	Audio needs its own server	`STT`/`TTS` server variables actually set
Answers never change however you edit	The cache is serving stored replies	Unset `HANDS_ON_AI_CACHE` or delete the directory
Agent replies look unstructured	Small model in the text format	Let `format="auto"` choose, as Chapter 8 discussed
Everything worked yesterday	Settings were terminal-session exports	Put durable choices in `~/.hands-on-ai/config.json`

One habit resolves most rows: run the doctor, read what it resolved, and argue with settings before arguing with scripts. The scripts in this book are short and seldom the problem; when they are, the error strings Part I taught you to read will usually say which line to look at.

Appendix C

The Project Gallery

> The package ships its own set of guided projects. This appendix maps them to the book's parts and suggests where each chapter's reader goes next.

The `hands-on-ai` documentation hosts the project guides at michael-borck.github.io/hands-on-ai (https://michael-borck.github.io/hands-on-ai/), organised by module. At version 0.6.0 there are 46 of them. Each guide assumes the concept and walks a build. The natural rhythm is chapter first, its named project second, then one or two neighbours from the same module while the ideas are warm.

C.1 After Part I: Chat

The book's companion builds were the five-line call, the personality bots, and the remembering conversation. The chat gallery is the largest, with 24 projects. Start with *Build Your Own Personality Bot* and *Knowledge Quiz Bot*, both direct extensions of Chapter 2's function-wrapping pattern. *Build Your Own Cache* reimplements Chapter 4's lesson from the inside. From there the gallery widens by appetite: tutors and journal bots for the patient, text adventures and debate panels for the playful, dashboards and converters for the practically minded. Any three will sharpen the system-prompt

judgement from Chapter 2.

C.2 After Part II: Retrieval

Four projects: *My First RAG Bot*, which Chapter 6 builds; *PDF Question-Answering Chatbot*, the same shape aimed at one document type; *Build Your Own Q&A CLI*, for readers who want the command-line tool rather than the library calls; and *Hybrid Reasoning Bot*, which starts mixing retrieval with the agent patterns of Part III. Do the first alongside the chapter and at least one more on documents you own.

C.3 After Part III: Agents

Twelve projects, from *Simple Calculator Agent* (Chapter 7's companion, with a longer treatment of safe expression evaluation) through *Build Your Own Tool* and *Custom Tool Creator*, to *Multi-Tool Problem Solver* and *Agent Evaluation Framework* at the deeper end. Heed Chapter 7's caution as you browse: the registry calls tools with one string, so read each guide's tool signatures before adopting them, and test tools directly before letting a model anywhere near them.

C.4 After Part IV: Workflow, Loop, Eval

Four projects across the three control modules, each named in its chapter: *Build a Pipeline* for workflows, *Improve Until Good* and *Build a Ralph Loop* for the two loop shapes, and *Judge Your Bot* for evaluation. These are short modules and the guides are close to the chapters; their value is repetition on material of your choosing, with the review gates, logs, and moderation habits intact.

C.5 After Part V: Vision and Voice

Two projects, one per sense: *The Talking Picture*, a beginner's version of Chapter 13's build, and *Build the World's Smallest Pipecat*, the

advanced bridge from Chapter 14's three calls to the production voice framework in Appendix D. Vision and voice age fastest in this book; treat both guides as starting points and expect model names to have moved.

C.6 Three Capstones

Pick by your reason for reading. Each composes at least three parts of the book.

For a personal tool. A document assistant over your own notes: Chapter 6 retrieval over a folder you maintain, Chapter 3 conversation for follow-ups, Chapter 11 criteria checking that answers stayed inside the documents. Success looks like a tool you open weekly whose sources you can click through to.

For a classroom demo. A judged improvement loop shown live: a rough student paragraph, Chapter 10's ratchet with its log projected where the class can argue with each keep and discard, and a Chapter 11 moderation table comparing the class's marks with the judge's. Success looks like students disputing criteria wording, unprompted.

For a portfolio piece. A small staged workflow with traces kept: a Chapter 9 pipeline whose middle stage uses Chapter 7 tools and whose final stage is scored by Chapter 11's judge, all outputs filed for inspection. Success looks like a stranger being able to replay the run, find the stage you are least proud of, and hear you explain its job card.

Appendix D

Going Deeper

> A map, not a manual. For each module you outgrow, the grown-up neighbourhood to move into, and the idea from this book to carry through the door.

You will outgrow this package at a named moment: a budget it cannot express, a volume it cannot serve, an error it returns as a string where production needs a policy. Frameworks and services change quickly enough that this appendix names neighbourhoods and the current well-known residents, and trusts you to check who lives there when you arrive. The concepts are the durable part.

D.1 The Call Itself (Part I)

Your next step is the providers' official SDKs, which add streaming, structured outputs, provider-native tool calling, and proper exceptions where this package returns error strings. If several providers must sit behind one interface in your own code, that is the job of routing layers such as LiteLLM, which generalise Chapter 4's three variables into a managed surface. Carry with you: the payload is still the truth; read what any SDK actually sends before trusting its abstractions, as Listing 1.2 taught you.

D.2 Grounding (Part II)

Production retrieval adds vector stores, hybrid keyword-plus-vector search, metadata filtering, incremental reindexing, and evaluation sets for retrieval quality. LlamaIndex is the neighbourhood's best-known framework address, with LangChain's retrieval pieces and every vector database vendor nearby. Carry with you: it is still chunk, fetch, answer. The engineering is in chunking policy, freshness, and proving with scores and sampled answers that the fetch deserves the prompt built on it.

D.3 Action (Part III)

Agent frameworks such as Pydantic AI and the SDKs' native agent tooling replace text-format parsing with validated function calling, add dependency injection for tools, retries with policy, and tracing as a first-class output. Carry with you: the model still only proposes. Your registry, your allowlists, and your worst-input discipline from Chapter 7 are the safety case, whatever the framework calls its decorators. Read *Ship Python, Orchestrate AI* in this series for this transition taken at professional length.

D.4 Control (Part IV)

Workflow engines and durable-execution platforms add scheduling, parallelism, persistence across crashes, and human-task queues where this book had folders and a person at a bench. Evaluation matures into dataset-driven evals: fixed case sets, regression gates in CI, judge ensembles with measured agreement. Carry with you: stage outputs you can inspect, goals stated before loops run, and moderation records for every judge. A platform that hides any of the three removes evidence you need. Typed judgments grow up into structured-output modes in the official SDKs, schema-constrained decoding in local servers such as Ollama (verify the current `format` behaviour before relying on it), and dedicated System One services such as TypeSafe's Jev where calibrated probabilities are worth

paying for.

D.5 The Senses (Part V)

Vision production work adds image pipelines, detection and OCR specialists beside the general VLM, and thinking about resolution and cost per image. Voice production is its own stack, and Pipecat is the front door the package's final project points at: streaming transcription and synthesis, interruption handling, and the latency budget fought hop by hop. Carry with you: translators around a text core, and the consent and retention questions, which do not get simpler at scale.

D.6 The Model Itself

One door remains if the black box itself is what you want opened. Andrej Karpathy's *Zero to Hero* series builds a small language model from first principles, at the cost of the mathematics Parts I to V promised to spare you: gradients, tensors, training loops. Its codebase line now runs from the deprecated nanoGPT to its successor nanochat, a full pretrain-to-chat pipeline kept deliberately hackable. Part VI opens this briefly, with toy training and a LoRA adapter; Karpathy covers it at full size. Nothing in Parts I to V requires either, and most builders of AI systems will never train a model.

Acknowledgments

This book exists because the companion package came first. `hands-on-ai` began as a teaching lab: eight small modules, each wrapping one call to a model in plain Python, with documentation pages and project guides for readers to build against. The book is the narrative the package leaves out, the why and the progression and the misconceptions each module is designed to kill. Every listing here was checked against the package source, and the package is better for the checking; several sharp edges found while drafting these chapters, including a tool description that its own function could not accept, were worth finding in a manuscript rather than in a classroom.

The listings were tested the way the book asks readers to test them: offline first, with scripted stand-ins where a live model would only add weather, and against local models where the call itself was the lesson. Students and classes who worked through early versions of these builds, in applied AI teaching at Curtin University and in the workshops around it, showed exactly where the explanations ran ahead of the code. Where a chapter now slows down to print a payload or read a trace against the grain, it is usually because a room full of capable people skimmed past that step and then could not say what had happened.

The intellectual debts are named where they are incurred: the ReAct paper for the reason-act-observe loop, the original retrieval-augmented generation paper for search-then-prompt, Jake Van Clief and David McDermott's Interpretable Context Methodology for the folder-as-agent architecture, Andrej Karpathy's teaching for the go-

deeper exits, and *Conversation, Not Delegation* for the argument that judging model output is the new literacy. The optional final part draws on the local fine-tuning community's tooling and notebooks, credited in place.

The open-source community behind Quarto, Pandoc, Python and GitHub made it possible to write, build and publish this book from text files the reader can inspect, alongside the package it narrates. The local-model ecosystem, Ollama in particular, is what makes a book like this runnable offline in a classroom at all.

This book was written using the machinery it describes. AI drafted, summarised, checked listings and proposed structures at every stage, and produced fluent errors on schedule, which is why every printed output that claims to be program output was verified by running the program. The machine made the work faster. The judgement, and the responsibility for every claim that remains, are the author's.

About the Author



Figure D.1: Photo of Michael Borck

Michael Borck is a software developer and educator working at the intersection of human expertise and artificial intelligence. He teaches applied AI and AI in business at Curtin University, and runs *The Human Edge* masterclasses through Curtin Executive Education. *Build AI, Understand AI* grew out of a simple classroom observation: students could use AI tools fluently long before they could say what those tools were doing, and the gap closed fastest when they built a tiny version themselves.

His earlier book, *Conversation, Not Delegation*, argues for treating AI as a thinking partner you converse with. *Speak Software* teaches readers to direct agents without writing code. This book opens the box those agents are built on: every chapter reduces one familiar AI system to a single model call wrapped in plain Python, built in miniature so it can be read in one sitting. The doing lives in

the companion package, `hands-on-ai` (`pip install hands-on-ai`), whose modules, documentation and project guides this book narrates; neither duplicates the other.

The bet underneath the book is that models change every few months and the ideas underneath them change slowly. A reader who has built chat, retrieval, an agent loop, a workflow and a judge by hand stops being fooled by demos, whatever the next model is called. Michael builds educational software and resources, and writes about the 80/20 principle in learning and professional judgement.

Connect

- GitHub projects (https://github.com/michael-borck): Software and educational resources
- Free book collection (https://books.borck.education): Current online editions and downloads
- 8020workshop.com (https://8020workshop.com): Passion projects and workshops
- LinkedIn (https://linkedin.com/in/michaelborck)

Other Books in This Series

Python Track:

- *Think Python, Direct AI: Computational Thinking for Beginners*
- *Code Python, Consult AI: Python Fundamentals for the AI Era*: the language this book talks to the machine in
- *Ship Python, Orchestrate AI: Professional Python in the AI Era*: workflows, loops and judges in production form
- *Converse Python, Partner AI: The Intentional Prompting Methodology*
- *Speak Software*: directing agents without code; the natural first book for the never-touch-a-keyboard reader

AI-Collaboration and Judgement Track:

- *Conversation, Not Delegation: How to Think With AI, Not Just Use It*: the thinking-with-AI framework behind this series
- *Prove, Don't Persuade: Why Proof Beats the Demo in Applied AI*
- *Substantiate, Don't Assume: Why the Auditor's Opinion Matters More Than Ever in the Age of AI*
- *Partner, Don't Police: AI in the Business Classroom*

Web Track:

- *Build Web, Guide AI: Business Web Development with AI*